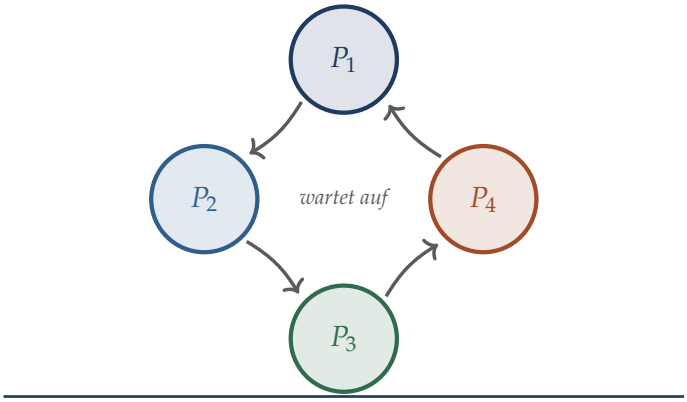
Informatik von Grund auf — Band 5

Der Wettlauf der Prozesse

Eine vollständige Einführung in Betriebssysteme und Nebenläufigkeit



Von der Unterbrechung bis zur Verklemmung

Jeder Schritt erklärt. Mit vollständig durchgerechneten Übungsaufgaben.

Informatik von Grund auf — Band 5
Der Wettlauf der Prozesse · Betriebssysteme und Nebenläufigkeit

Herausgeber und Urheber: Jürgen Maier
Entstanden 2026 in Zusammenarbeit mit Claude (Anthropic); Konzeption, Vorgaben, Prüfung und Herausgeberschaft: Jürgen Maier.

Lizenz
Dieses Werk ist lizenziert unter einer Creative-Commons-Lizenz **Namensnennung – Weitergabe unter gleichen Bedingungen 4.0 International (CC BY-SA 4.0)**. Sie dürfen das Werk vervielfältigen, verbreiten, bearbeiten und auch kommerziell nutzen, sofern Sie den Urheber nennen und Bearbeitungen unter derselben Lizenz weitergeben.
Lizenztext: `https://creativecommons.org/licenses/by-sa/4.0/legalcode.de`

Quellen und Fassungen
Quelltexte, Fehlermeldungen und aktuelle Fassungen aller Bände:
`https://github.com/juergenmaier2026/informatik-von-grund-auf`
Dauerhaft archiviert bei Zenodo: `https://doi.org/10.5281/zenodo.23773283`
Schwesterreihe: *Mathematik von Grund auf*, `https://doi.org/10.5281/zenodo.23698547`

Zitiervorschlag
Maier, J. (2026). *Informatik von Grund auf, Band 5: Der Wettlauf der Prozesse*. Fassung 1.0.1. `https://doi.org/10.5281/zenodo.23773283`

Satz: LaTeX mit Palatino. Fassung 1.0.1, Oktober 2026. 186 Seiten.

Stand der Forschung: August 2026

Inhaltsverzeichnis

Vorwort 1

I Der Kern 9

1 Was ein Betriebssystem ist 11

2 Unterbrechungen 23

II Prozesse 33

3 Prozesse und Fäden 35

4 Ablaufplanung 47

5 Speicherverwaltung 57

III Nebenläufigkeit 65

6 Wettläufe und wechselseitiger Ausschluss 67

7 Semaphore und Monitore 79

8 Verklemmungen 91

9 Die klassischen Aufgaben 99

IV Ein- und Ausgabe 109

10 Geräte und Treiber 111

11 Dateisysteme 121

12 Wenn der Strom ausfällt 131

V Größere Zusammenhänge **141**

13 Virtualisierung und Container **143**

14 Schutz und Isolation **153**

15 Der Stand im Jahr 2026 **163**

VI Anhang **169**

A Zahlen und Kenngrößen **171**

B Landkarte des Bandes **173**

C Glossar **175**

D Zeittafel **177**

E Kommentierte Quellen **179**

Vorwort: Der Fehler, der nur manchmal auftritt

Dieser Band handelt von zwei Dingen, die enger zusammengehören, als es zunächst scheint. *Das eine ist die Software, die einen Rechner verwaltet. Das andere ist die Frage, was geschieht, wenn mehrere Dinge gleichzeitig passieren.*

Ein Programm, das fast immer richtig rechnet

Zwei Fäden erhöhen denselben Zähler, jeder eine Million Mal. Am Ende sollten zwei Millionen dastehen.

```
int zaehler = 0;

void *arbeit(void *arg) {
  for (int i = 0; i < 1000000; i++)
zaehler++;
  return NULL; }
```

Zehn Läufe auf einem gewöhnlichen Rechner ergaben:

Lauf	Ergebnis	Lauf	Ergebnis
1	1 384 721	6	1 204 556
2	1 102 993	7	**2 000 000**
3	1 551 208	8	1 317 842
4	1 299 441	9	1 468 015
5	1 733 760	10	1 090 377

> *Kein einziger Lauf ist reproduzierbar, und einer ist zufällig richtig. Genau das macht solche Fehler so unangenehm: Sie verschwinden, sobald man hinsieht.*

Die Ursache steckt in der Zeile `zaehler++`. Sie sieht aus wie eine Anweisung und zerfällt in drei:

Schritt	Befehl	Wirkung
1	`mov eax,[zaehler]`	lesen
2	`add eax,1`	rechnen
3	`mov [zaehler],eax`	schreiben

Wird ein Faden zwischen Schritt 1 und 3 unterbrochen und erhöht der andere in dieser Zeit den Zähler, so überschreibt der erste das Ergebnis mit einem veralteten Wert. Eine Erhöhung geht verloren.

Warum das nicht durch Testen zu finden ist

Das Zeitfenster für den Fehler umfasst zwei Maschinenbefehle, also unter einer Nanosekunde. Ob eine Unterbrechung genau dort fällt, hängt von Dingen ab, die kein Programm beeinflusst.

Andere Last	Was sonst noch auf dem Rechner läuft
Cachezustand	Wie lange ein Zugriff dauert
Zeitgeber	Wann die nächste Unterbrechung fällt
Kernzahl	Auf einem Kern seltener als auf vieren

> *Ein Test, der tausendmal durchläuft, beweist gar nichts. Der Fehler kann beim tausendundersten Mal auftreten, oder erst beim Kunden, oder nur auf Maschinen mit mehr Kernen als der Testrechner hat.*

Bei nebenläufigen Programmen genügt es deshalb nicht, sie zu testen. Man muss zeigen können, dass sie richtig sind. Teil III dieses Bandes handelt davon, wie das geht.

Was ein Betriebssystem eigentlich tut

Die übliche Antwort lautet: Es verwaltet die Betriebsmittel. Das ist richtig und wenig aufschlussreich. Eine bessere Antwort besteht aus drei Punkten.

Es lügt	Jedes Programm bekommt vorgespiegelt, es habe den Rechner für sich allein: einen eigenen Adressraum, einen eigenen Prozessor, eigene Dateien
Es schützt	Kein Programm darf ein anderes stören, auch nicht durch Fehler oder böse Absicht
Es teilt zu	Wenn mehrere etwas wollen und es nicht für alle reicht, muss jemand entscheiden

Der erste Punkt ist derselbe Gedanke wie in Band 4. Dort hielt der Prozessor die Fiktion aufrecht, er führe einen Befehl nach dem anderen aus. Hier hält das Betriebssystem die Fiktion aufrecht, jedes Programm laufe allein.

Programm sieht	**tatsächlich**	**Mechanismus**
eigener Prozessor	geteilt	Ablaufplanung (Kap. 4)
eigener Speicher	geteilt	Seitentabellen (Kap. 5)
eigene Geräte	geteilt	Treiber (Kap. 10)
Dateien und Ordner	Blöcke auf Platten	Dateisystem (Kap. 11)

Wie in Band 4 gilt auch hier: Die Fiktion ist nicht dicht. Ein Programm merkt, dass es sich den Rechner teilt, sobald es auf die Uhr sieht.

Was dieses Buch will

Erstens: Die Mechanismen vom Zweck her erklären.
Ein Betriebssystem besteht aus einer großen Zahl von Datenstrukturen und Verfahren, die man auswendig lernen kann, ohne etwas verstanden zu haben. Ich versuche stattdessen, bei jedem Mechanismus zuerst zu sagen, welches Problem er löst und was ohne ihn schiefginge.

Zweitens: Nebenläufigkeit ernst nehmen.

> *Vier von fünfzehn Kapiteln handeln davon. Das ist mehr, als Lehrbücher üblicherweise vorsehen, und es entspricht dem Anteil, den dieses Thema in der Praxis an Schwierigkeiten verursacht.*

Seit der Wende zu Mehrkernprozessoren (Band 4, Kapitel 11) ist nebenläufiges Programmieren keine Spezialität mehr, sondern Alltag. Die Werkzeuge dafür stammen größtenteils aus den Jahren zwischen 1965 und 1978 und werden bis heute benutzt.

Drittens: Zeigen, wo die Schicht durchlässig ist.

Kapitel 15 sammelt die Stellen, an denen man das Betriebssystem bemerkt, obwohl man es nicht sollte: unerklärliche Verzögerungen, Programme, die auf einer anderen Maschine anders laufen, Sicherheitsgrenzen, die weniger dicht sind als gedacht.

Drei Warnungen

Erstens: Fast alles hier ist ein Kompromiss.
Bei der Ablaufplanung, bei der Speicherverwaltung, bei den Dateisystemen gibt es keine richtige Lösung, sondern Verfahren mit verschiedenen Stärken. Ich sage jeweils dazu, wofür ein Verfahren gedacht war und wo es versagt.
Zweitens: Die Beispiele beziehen sich auf Unix.

> *Das ist eine Entscheidung, keine Wertung. Unix und seine Nachkommen sind gut dokumentiert, quelloffen und weit verbreitet, und die Begriffe der Fachliteratur stammen überwiegend von dort. Wo Windows grundsätzlich anders vorgeht, sage ich es.*

Drittens: Sperrfreie Verfahren sind nichts für den Alltag.
Kapitel 7 zeigt, wie man ohne Sperren auskommt, und rät zugleich davon ab. Wer nicht sehr genau weiß, was er tut, schreibt Code, der auf der eigenen Maschine funktioniert und anderswo nicht.

Zum Aufbau

- **Teil I** (Kap. 1–2): Was ein Kern ist, wie man ihn aufruft, und wie Unterbrechungen funktionieren.
- **Teil II** (Kap. 3–5): Prozesse und Fäden, Ablaufplanung, Speicherverwaltung.
- **Teil III** (Kap. 6–9): Wettläufe, Sperren, Semaphore, Monitore, Verklemmungen und die klassischen Aufgaben.
- **Teil IV** (Kap. 10–12): Geräte, Dateisysteme und die Frage, wie Daten einen Stromausfall überstehen.
- **Teil V** (Kap. 13–15): Virtualisierung und Container, Isolation und Sicherheit, und der Stand im August 2026.

Ein Betriebssystem als Nebenprodukt

Bell Labs beteiligte sich in den 1960er Jahren zusammen mit MIT und General Electric an Multics, einem sehr ehrgeizigen Betriebssystemprojekt. Multics sollte alles können: mehrere Nutzer, dynamisches Nachladen, feingranulare Rechte, ein einheitlicher Adressraum für Speicher und Dateien.
Das Projekt geriet in Verzug. Bell Labs stieg 1969 aus.

> *Damit standen* KEN THOMPSON *und* DENNIS RITCHIE *ohne die Rechnerumgebung da, an die sie sich gewöhnt hatten. Thompson fand eine ausrangierte PDP-7 und begann in seiner Freizeit, ein sehr viel kleineres System dafür zu schreiben.*

Der Name war zunächst ein Wortspiel. BRIAN KERNIGHAN schlug „Unics“ vor, ein kastriertes Multics, und daraus wurde Unix.

	Multics	**Unix (1974)**
Entwicklungszeit	8 Jahre	wenige Monate
Kern	sehr groß	etwa 10 000 Zeilen
Sprache	PL/I	**C**
Systemaufrufe	über 200	etwa 30

Der entscheidende Schritt kam 1973, als Ritchie den Kern in C neu schrieb. Bis dahin galt es als selbstverständlich, dass ein Betriebssystem in Assembler entstehen muss.

> *Ein in einer Hochsprache geschriebener Kern ließ sich auf andere Rechner übertragen. Genau das geschah, und deshalb gibt es Unix heute noch, während Multics 2000 abgeschaltet wurde.*

Ritchie und Thompson beschrieben das System 1974 in einem Aufsatz von elf Seiten. Er ist bis heute lesenswert, weil er die Entwurfsgedanken nennt und nicht nur die Merkmale. Der bekannteste davon: Alles ist eine Datei.

Beide erhielten 1983 den Turing-Award. In seiner Dankesrede warnte Thompson vor einer Möglichkeit, einen Übersetzer so zu manipulieren, dass er sich selbst und eine Hintertür immer wieder erzeugt (Band 2, Kapitel 6).

Multics war übrigens keineswegs erfolglos. Viele seiner Ideen, etwa der virtuelle Speicher mit Segmenten und die feingranularen Zugriffsrechte, wurden später anderswo aufgegriffen. Es war ein System, das zu früh zu viel wollte.

Sechzig Jahre Betriebssysteme

1956	GM-NAA I/O	erstes Stapelverarbeitungssystem
1961	CTSS	**Zeitscheibenbetrieb**
1961	Atlas	virtueller Speicher
1964	Multics	Projektbeginn; Rechnen als Versorgungsleistung
1965	**Dijkstra**	**wechselseitiger Ausschluss**
1965	Dijkstra	Semaphore, THE-System
1968	**Dijkstra**	**„Cooperating Sequential Processes“**
1971	Coffman u. a.	vier Bedingungen für Verklemmungen
1974	**Ritchie, Thompson**	**Unix**
1974	**Hoare**	**Monitore**
1975	Saltzer, Schroeder	acht Entwurfsgrundsätze für Schutz
1978	Hoare	CSP
1978	**Lamport**	**Ordnung von Ereignissen**
1981	Peterson	einfache Lösung des Ausschlussproblems
1983	Lampson	„Hints for Computer System Design“
1987	MINIX	Lehrsystem von Tanenbaum
1991	Linux	Torvalds beginnt
1992	Rosenblum, Ousterhout	protokollstrukturiertes Dateisystem
1993	Windows NT	

1994	Bonwick	Slab-Zuteiler
1996	Ousterhout	„Warum Fäden eine schlechte Idee sind“
2000	FreeBSD Jails	frühe Container
2003	**Xen**	**Paravirtualisierung**
2005	Intel VT-x, AMD-V	Virtualisierung in Hardware
2007	cgroups	Ressourcengrenzen in Linux
2013	**Docker**	**Container im Massengebrauch**
2014	Kubernetes	Verwaltung vieler Container
2018	Spectre, Meltdown	Isolation wird teuer (Band 4)
2019	io_uring	neue Ein-/Ausgabeschnittstelle in Linux
2020	eBPF	Programme im Kern ausführen
2026	—	Kerne in Rust; Isolation als Hauptthema

Zwei Zeilen sind für diesen Band besonders wichtig.
1965, Dijkstra. EDSGER DIJKSTRA veröffentlichte eine einseitige Notiz mit der ersten korrekten Lösung des Problems, mehreren Prozessen abwechselnd Zugang zu einer gemeinsamen Ressource zu geben, ohne Hardwareunterstützung. Der Aufsatz begründete ein ganzes Teilgebiet.
1974, Hoare. TONY HOARE schlug den Monitor vor, eine Sprachkonstruktion, die den wechselseitigen Ausschluss vom Programmierer weg in den Übersetzer verlagert. Praktisch jede moderne Programmiersprache hat eine Form davon, auch wenn sie anders heißt.

Ein nebenläufiges Programm zu testen ist wie zu prüfen, ob ein Heuhaufen eine Nadel enthält, indem man zwanzig Mal hineingreift.

— sinngemäß eine verbreitete Redensart unter Systemprogrammierern. Ihr Ursprung ist unklar; der Sachverhalt ist es nicht.

Part I

Der Kern

1 Was ein Betriebssystem ist

Zwei Betriebsarten

Der Schutz, von dem im Vorwort die Rede war, beruht auf einer einzigen Vorrichtung im Prozessor: einem Bit, das sagt, ob gerade der Kern läuft oder ein gewöhnliches Programm.

	Kernbetrieb	Benutzerbetrieb
Alle Befehle	ja	**nein**
Seitentabelle ändern	ja	nein
Unterbrechungen sperren	ja	nein
Geräte ansprechen	ja	nein
Zugriff auf Kernspeicher	ja	nein

Versucht ein Programm im Benutzerbetrieb einen dieser Befehle, so löst der Prozessor eine Ausnahme aus und übergibt die Kontrolle an den Kern. Das Programm kann sich also nicht selbst Rechte verschaffen.

> *Die ganze Sicherheit eines Betriebssystems ruht darauf, dass dieser Übergang nur an vom Kern festgelegten Stellen möglich ist. Wer eine Möglichkeit findet, ihn anderswo auszulösen, hat das System übernommen.*

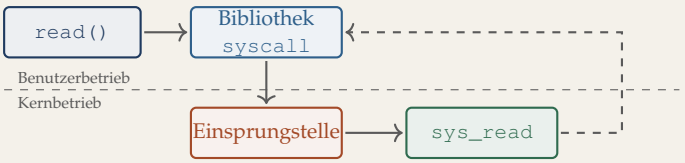
Wie ein Programm den Kern erreicht.
Es gibt genau drei Wege in den Kernbetrieb.

Anlass	ausgelöst durch	Beispiel
Systemaufruf	das Programm selbst	Datei öffnen
Ausnahme	das Programm, ungewollt	Division durch null
Unterbrechung	ein Gerät	Tastendruck

Die ersten beiden sind synchron, also durch den laufenden Befehl verursacht. Die dritte ist asynchron und kann jederzeit eintreffen. Kapitel 2 behandelt alle drei ausführlich.

Systemaufrufe

Ein Systemaufruf sieht im Programm aus wie ein gewöhnlicher Funktionsaufruf und ist etwas ganz anderes.



1. Das Programm legt die Nummer des gewünschten Dienstes und die Argumente in bestimmte Register.
2. Es führt einen besonderen Befehl aus (`syscall` auf x86-64, `svc` auf ARM).
3. Der Prozessor wechselt in den Kernbetrieb und springt an eine **feste, vom Kern eingetragene** Adresse.
4. Der Kern prüft die Argumente, erledigt die Arbeit und kehrt zurück.

Punkt 3 ist der Kern der Sache. Das Programm bestimmt nicht, wohin gesprungen wird. Es kann nur eine Nummer angeben, und der Kern entscheidet, was er damit tut.

Was ein Systemaufruf kostet.

Vorgang	Takte	Bemerkung
Funktionsaufruf	2–5	zum Vergleich
Systemaufruf, minimal	≈ 150	`getpid`
mit Spectre-Gegenmaßnahmen	≈ 700	seit 2018 (Band 4)
Prozesswechsel	2000–10 000	mit Cachewirkung

Der Sprung von der zweiten zur dritten Zeile hat weitreichende Folgen. Programme, die viele kleine Systemaufrufe machen, wurden 2018 spürbar langsamer, und ein Teil der Entwicklung seither zielt darauf, ihre Zahl zu senken.

Wie man Systemaufrufe spart

Puffern	Statt jedes Byte einzeln zu schreiben, sammelt die Bibliothek und schreibt blockweise
Bündeln	Ein Aufruf erledigt viele Vorgänge, etwa `writev` oder `io_uring`
vDSO	Häufige, harmlose Auskünfte werden ohne Betriebsartwechsel beantwortet
Einblenden	Dateien werden in den Adressraum eingeblendet und wie Speicher benutzt

Der dritte Punkt verdient eine Erläuterung. Der Aufruf `gettimeofday` muss nur eine Zahl liefern, die der Kern ohnehin regelmäßig aktualisiert. Statt eines vollen Betriebsartwechsels blendet der Kern eine kleine Seite mit Code und Daten in jeden Adressraum ein, das **virtuelle dynamische geteilte Objekt**.

Aufruf	ohne vDSO	mit vDSO
`gettimeofday`	≈ 700 Takte	≈ 25 Takte
`clock_gettime`	≈ 700 Takte	≈ 30 Takte

> *Das ist ein Beispiel dafür, wie eine Schutzgrenze aufgeweicht wird, weil sie zu teuer ist. Der Kern gibt einen kleinen Teil seiner Daten preis, weil sie ohnehin harmlos sind und der Gewinn erheblich ist.*

Der vierte Punkt, `io_uring`, geht noch weiter. Programm und Kern teilen sich zwei Ringpuffer im Speicher: einen für Aufträge, einen für Ergebnisse. Im günstigsten Fall entfällt der Systemaufruf ganz, weil ein Kernfaden die Aufträge von sich aus abholt.

Wie groß darf ein Kern sein?

Über diese Frage wurde in den 1990er Jahren heftig gestritten, und die Antwort steht bis heute nicht fest.

	Monolith	Mikrokern
Im Kern	alles: Treiber, Dateisysteme, Netz	nur Speicher, Fäden, Nachrichten
Rest läuft	—	als gewöhnliche Prozesse
Ein Treiberfehler	legt alles lahm	trifft nur ihn
Aufwand je Dienst	Funktionsaufruf	**Nachricht, zwei Wechsel**
Beispiele	Linux, Windows, BSD	QNX, seL4, MINIX 3

> *Der Mikrokern ist der sauberere Entwurf, und er hat sich außerhalb von Nischen nicht durchgesetzt. Der Grund ist die vierte Zeile: Was im Monolithen ein Funktionsaufruf ist, wird zu einer Nachricht mit zwei Betriebsartwechseln.*

Die Nischen sind allerdings bemerkenswert. QNX steckt in vielen Fahrzeugen, seL4 ist ein Mikrokern mit maschinell geprüftem Korrektheitsbeweis, und Apples Kern hat einen Mach-Mikrokern in seinem Inneren, um den herum ein monolithischer BSD-Teil gebaut ist.

Der Streit von 1992

Im Januar 1992 schrieb ANDREW TANENBAUM in die Newsgruppe `comp.os.minix` einen Beitrag mit der Überschrift „Linux is obsolete“. Sein Argument: Monolithische Kerne seien ein Entwurf der 1970er Jahre. Ein neues System im Jahr 1991 monolithisch zu bauen, sei ein Rückschritt. Außerdem sei Linux zu eng an die Intel-Architektur gebunden.

> LINUS TORVALDS *antwortete am selben Tag, teils sachlich, teils schnippisch. Er räumte ein, dass MINIX der sauberere Entwurf sei, hielt aber dagegen, dass Linux funktioniere und benutzt werde.*

Die Diskussion zog sich über Wochen und ist vollständig archiviert. Sie gilt als eine der bekanntesten Auseinandersetzungen der Informatikgeschichte.

Tanenbaum lag richtig	Monolithische Kerne sind schwer zu warten; ein Treiberfehler kann das System zum Absturz bringen
Torvalds lag richtig	Ein System, das läuft und wächst, schlägt einen Entwurf, der eleganter ist

> *Interessant ist, was seither geschah. Linux hat viele Mikrokerneigenschaften nachgerüstet: Treiber im Benutzerbetrieb über FUSE und UIO, ladbare Module, und mit eBPF sogar eine Möglichkeit, geprüfte Programme im Kern auszuführen, ohne ihn neu zu übersetzen.*

Tanenbaum hat später gesagt, er habe die Debatte im Rückblick eher gewonnen als verloren, weil sich die Ideen durchgesetzt hätten, wenn auch nicht sein System. Torvalds hat gesagt, er sei damals fünfundzwanzig gewesen und würde manches heute höflicher formulieren.

Aufgabe 1.1.

(a) Verfolgen Sie einen Systemaufruf vollständig und bestimmen Sie seine Kosten.
(b) Rechnen Sie aus, wann sich das Puffern von Ausgaben lohnt.
(c) Vergleichen Sie Monolith und Mikrokern quantitativ.

Lösung Schritt für Schritt.

(a) Ein Systemaufruf im Detail.
Wir verfolgen `read(fd, puffer, 4096)` unter Linux auf x86-64.

Schritt 1 – Im Benutzerbetrieb.

```
mov rax, 0      Nummer von read
mov rdi, fd
mov rsi, puffer
mov rdx, 4096
syscall
```

Schritt 2 – Was der Prozessor tut.

1. Rücksprungadresse nach `rcx`, Statusregister nach `r11` retten.
2. Betriebsart auf Kern setzen.
3. Zum Wert im Register `IA32_LSTAR` springen, den der Kern beim Hochfahren eingetragen hat.

Schritt 3 – Was der Kern tut.

1. Auf den Kernstapel dieses Fadens umschalten.
2. Alle Register retten.
3. Seitentabelle wechseln (seit KPTI, siehe Band 4, Kapitel 14).
4. Nummer prüfen und in der Sprungtabelle nachschlagen.
5. **Argumente prüfen**: Ist `fd` offen? Liegt `puffer` im erlaubten Bereich?
6. Arbeit erledigen.
7. Alles rückwärts, dann `sysret`.

Punkt 5 ist sicherheitskritisch

Der Kern darf den übergebenen Zeiger keinesfalls einfach benutzen. Ein Programm könnte eine Kernadresse angeben und den Kern dazu bringen, dorthin zu schreiben.

> *Deshalb gibt es eigene Funktionen für Zugriffe auf Benutzerspeicher, in Linux* `copy_from_user` *und* `copy_to_user`*. Sie prüfen den Bereich und behandeln den Fall, dass die Seite gar nicht vorhanden ist.*

Ein direkter Zugriff auf Benutzerspeicher wird von neueren Prozessoren sogar in Hardware unterbunden, mit der Vorrichtung SMAP. Der Kern muss sie für einen Zugriff ausdrücklich kurz abschalten. Das fängt eine ganze Klasse von Fehlern ab.

Schritt 4 – Die Kosten.

Anteil	Takte	Bemerkung
`syscall/sysret`	≈ 100	Hardware
Register retten	≈ 30	
Seitentabellenwechsel	≈ 300	KPTI
TLB-Verluste danach	≈ 200	indirekt
Prüfungen	≈ 50	
ohne die Arbeit selbst	≈ 680	

Bei 4 GHz sind das etwa 170 Nanosekunden. Wer eine Million Aufrufe je Sekunde macht, verbringt damit 17 Prozent seiner Zeit.

(b) Wann sich Puffern lohnt.
Der Fall. Ein Programm schreibt 10^7 Zeilen zu je 40 Byte in eine Datei.
Fassung 1 – Jede Zeile einzeln.

$$10^7 \text{ Aufrufe} \cdot 680 \text{ Takte} = 6{,}8 \cdot 10^9 \text{ Takte} = 1{,}7 \text{ s}.$$

Fassung 2 – Puffer von 64 KB.

$$\frac{10^7 \cdot 40}{65\,536} = 6104 \text{ Aufrufe}.$$

$$6104 \cdot 680 = 4{,}2 \cdot 10^6 \text{ Takte} = 0{,}001 \text{ s}.$$

Puffergröße	Aufrufe	Zeit im Kern
keiner (40 B)	10^7	1,70 s
4 KB	97 656	0,017 s
64 KB	6104	0,001 s
1 MB	382	0,00007 s

Der Sprung von keinem Puffer auf 4 KB bringt Faktor 100. Der Sprung von 64 KB auf 1 MB bringt fast nichts mehr, weil die Aufrufkosten dann ohnehin verschwunden sind.

Genau deshalb puffert die C-Bibliothek standardmäßig, und zwar mit Blockgrößen zwischen 4 und 64 KB. Ein Programm, das `printf` benutzt, macht viel weniger Systemaufrufe, als es Zeilen ausgibt.

Puffern hat einen Preis

Ein gepufferter Schreibvorgang ist nicht abgeschlossen, wenn die Funktion zurückkehrt. Bei einem Absturz gehen die noch nicht geschriebenen Daten verloren.

`stdout` an Terminal	zeilenweise gepuffert; man sieht jede Zeile sofort
`stdout` in Datei	blockweise gepuffert; bei Absturz fehlt der Rest
`stderr`	**ungepuffert**; Fehlermeldungen sollen ankommen

Die zweite Zeile führt zu einer verbreiteten Überraschung: Ein Programm, dessen Ausgabe auf dem Bildschirm vollständig erscheint, verliert beim Umleiten in eine Datei die letzten Zeilen, wenn es abstürzt.

Und selbst ein erfolgreicher Systemaufruf bedeutet noch nicht, dass die Daten auf der Platte sind. Dafür braucht es `fsync`, und Kapitel 12 handelt davon.

(c) Monolith gegen Mikrokern.
Der Fall. Ein Programm liest eine Datei. Im Monolithen ist das Dateisystem Teil des Kerns, im Mikrokern ein eigener Prozess.
Monolith.

Schritt	Takte
Systemaufruf hinein	680
Funktionsaufruf ins Dateisystem	5
Arbeit	—
zurück	—
Aufschlag	**685**

Mikrokern.

Schritt	Takte	
Nachricht an den Kern	680	
Wechsel zum Dateisystemprozess	2000	**Adressraumwechsel**
Antwort an den Kern	680	
Wechsel zurück	2000	
Aufschlag	**5360**	

$$\text{Faktor} = \frac{5360}{685} = 7{,}8.$$

Bei einem großen Lesevorgang von einem Megabyte fällt das nicht ins Gewicht. Bei vielen kleinen Zugriffen sehr wohl, und genau die machen bei Dateisystemen den Großteil aus.

Was dagegen spricht, es dabei zu belassen.
JOCHEN LIEDTKE zeigte in den 1990er Jahren mit seinem Kern L4, dass der Nachrichtenaustausch sehr viel billiger sein kann als angenommen. Er brachte ihn von etwa 5000 auf unter 200 Takte, indem er ihn konsequent für die Hardware optimierte.

System	Nachricht	Jahr
Mach	≈ 5000 Takte	1990
L4	≈ 150 Takte	1995
seL4	≈ 200 Takte	2009, mit Beweis

Damit fällt das Hauptargument gegen Mikrokerne weitgehend weg. Dass sie sich trotzdem nicht durchgesetzt haben, liegt eher daran, dass Linux und Windows vorhanden sind und funktionieren.

seL4 verdient eine gesonderte Erwähnung. Für diesen Kern existiert ein maschinell geprüfter Beweis, dass die Implementierung der Spezifikation entspricht. Der Beweis umfasst etwa 200 000 Zeilen und brauchte mehr als zwanzig Personenjahre, für einen Kern von rund 10 000 Zeilen C-Code.

Das Verhältnis von zwanzig Beweiszeilen je Codezeile zeigt, warum dieses Vorgehen die Ausnahme bleibt. Für sicherheitskritische Systeme lohnt es sich trotzdem.

Arpaci-Dusseau und Arpaci-Dusseau[2] sind der beste frei verfügbare Einstieg; Tanenbaum und Bos[28] behandeln die Kernarchitekturen ausführlich; Ritchie und Thompson[24] lohnen sich im Original, und wer den Gegenentwurf sehen will, dem Unix seinen Namen verdankt, lese Corbató und Vyssotsky[8].

2 Unterbrechungen

Warum es Unterbrechungen gibt

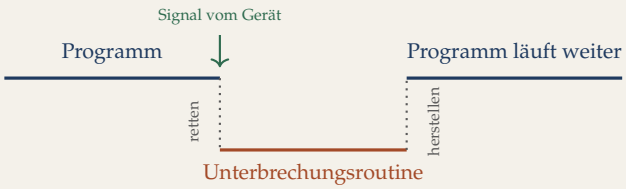
Ein Gerät braucht Zeit. Eine Platte antwortet nach Millisekunden, eine Tastatur wenn jemand tippt, eine Netzkarte wenn ein Paket eintrifft. Der Prozessor könnte in dieser Zeit etwas anderes tun.

	Abfragen	**Unterbrechung**
Ablauf	in einer Schleife nachsehen	das Gerät meldet sich
Prozessor	beschäftigt	frei für anderes
Verzögerung	sehr klein	**einige hundert Takte**
Sinnvoll bei	sehr schnellen Geräten	allem Übrigen

Die dritte Zeile erklärt, warum das Abfragen nicht ausgestorben ist. Bei einer schnellen Netzkarte, die Millionen Pakete je Sekunde liefert, sind Unterbrechungen teurer als das Nachsehen.

Moderne Netztreiber tun deshalb beides: Sie lassen sich die erste Meldung anzeigen, schalten dann die Unterbrechungen ab und fragen ab, bis nichts mehr kommt. Das Verfahren heißt in Linux NAPI und ist ein gutes Beispiel dafür, dass die eine Antwort selten die richtige ist.

Der Ablauf



1. Das Gerät meldet sich beim Unterbrechungssteuerwerk.
2. Der Prozessor beendet den laufenden Befehl, rettet Befehlszähler und Status.
3. Er schlägt in einer Tabelle nach, welche Routine zuständig ist, und springt dorthin.
4. Die Routine rettet die übrigen Register, erledigt das Nötige, stellt alles wieder her.
5. Ein besonderer Rücksprungbefehl stellt Befehlszähler und Betriebsart wieder her.

Das unterbrochene Programm merkt nichts. Es sieht nach der Unterbrechung genau denselben Zustand wie vorher, abgesehen davon, dass Zeit vergangen ist.

Warum das schwerer ist, als es aussieht.
Der Prozessor muss die Unterbrechung an einer Stelle annehmen, an der sich ein sauberer Zustand herstellen lässt. Bei einer Maschine mit Umordnung und Spekulation (Band 4, Kapitel 6) ist das nicht selbstverständlich, und genau dafür gibt es den Rückordnungspuffer.

Was in einer Unterbrechungsroutine verboten ist

Nicht blockieren	Es gibt keinen Faden, der schlafen könnte. Wer wartet, blockiert den ganzen Kern
Nicht auf Benutzerspeicher zugreifen	Der aktuelle Adressraum gehört einem beliebigen Prozess, nicht dem, für den die Unterbrechung gedacht ist
Keine gewöhnliche Sperre nehmen	Hält der unterbrochene Code sie schon, entsteht eine Verklemmung mit sich selbst
Kurz bleiben	Währenddessen sind meist weitere Unterbrechungen gesperrt

Die dritte Zeile verdient ein Beispiel. Ein Faden nimmt eine Sperre. Eine Unterbrechung trifft ein. Die Routine will dieselbe Sperre nehmen. Sie wartet auf einen Faden, der nicht weiterlaufen kann, solange die Routine läuft.

> *Deshalb gibt es in Kernen eigene Sperrformen, die beim Nehmen die Unterbrechungen abschalten. In Linux heißen sie* `spin_lock_irqsave`*, und wer sie verwechselt, erzeugt einen Fehler, der sich unter Last einmal am Tag zeigt.*

Die Zweiteilung.
Weil Unterbrechungsroutinen kurz sein müssen, teilt man die Arbeit auf.

	oberer Teil	unterer Teil
Läuft	sofort, mit gesperrten Unterbrechungen	später, unterbrechbar
Tut	Daten abholen, Gerät quittieren	auswerten, weiterreichen
Darf blockieren	nein	meist ja
Linux	`irq handler`	`softirq`, `tasklet`, `workqueue`

Ausnahmen

Eine Ausnahme wird nicht von einem Gerät ausgelöst, sondern vom laufenden Befehl selbst.

Art	Beispiel	danach
Fehler	Seitenfehler	Befehl wird **wiederholt**
Falle	Haltepunkt, Systemaufruf	es geht danach weiter
Abbruch	Speicherfehler der Hardware	keine Fortsetzung

Der Unterschied zwischen den ersten beiden Zeilen ist wichtig. Bei einem Seitenfehler hat der Befehl nichts bewirkt, und nachdem der Kern die Seite besorgt hat, wird er einfach nochmal ausgeführt. Bei einer Falle war der Befehl erfolgreich.

Der Seitenfehler ist die häufigste und wichtigste Ausnahme. Er ist kein Fehler im üblichen Sinn, sondern der normale Weg, auf dem Speicher zugeteilt wird.

Kleiner Seitenfehler	Die Seite ist im Speicher, nur nicht in dieser Seitentabelle eingetragen. Kostet Mikrosekunden
Großer Seitenfehler	Die Seite muss von der Platte geholt werden. Kostet Millisekunden
Schutzverletzung	Der Zugriff ist unzulässig. Das Programm bekommt ein Signal und stirbt meist

*Wenn ein Programm Speicher anfordert, bekommt es zunächst nur einen Eintrag in einer Liste. Die Seiten werden erst zugeteilt, wenn es sie wirklich anfasst, und zwar einzeln über Seitenfehler. Man nennt das **Zuteilung bei Bedarf**.*

Aufgabe 2.1.

(a) Vergleichen Sie Abfragen und Unterbrechungen rechnerisch.
(b) Untersuchen Sie den Seitenfehler beim Kopieren bei Bedarf.
(c) Analysieren Sie, was bei einem Unterbrechungssturm geschieht.

Lösung Schritt für Schritt.

(a) Abfragen gegen Unterbrechungen.
Die Größen.

Größe	Wert	
Kosten einer Unterbrechung	2000 Takte	mit Cachewirkung
Kosten einer Abfrage	50 Takte	ein Registerzugriff
Prozessortakt	4 GHz	

Fall 1 – Tastatur, 10 Anschläge je Sekunde.

Verfahren	Takte/s	Anteil
Unterbrechung	$2 \cdot 10^4$	0,0005 %
Abfrage (alle 10 ms)	$5 \cdot 10^3$	0,0001 %

Beides ist vernachlässigbar. Man nimmt Unterbrechungen, weil sie schneller reagieren.

Fall 2 – Netzkarte, 10^7 Pakete je Sekunde.

Verfahren	Takte/s	Anteil
Unterbrechung je Paket	$2 \cdot 10^{10}$	500 %
Abfrage	$5 \cdot 10^8$	12,5 %

Der Prozessor bräuchte fünfmal so viel Zeit, wie überhaupt vorhanden ist, allein für die Unterbrechungen. Er käme nie dazu, ein Paket zu verarbeiten.

Diese Lage heißt **Receive Livelock**: Das System ist vollständig damit beschäftigt, Unterbrechungen anzunehmen, und kommt zu keiner Nutzarbeit mehr. Sie wurde 1996 von MOGUL und RAMAKRISHNAN beschrieben.

Die Zwischenlösung.

1. Die erste Unterbrechung wird angenommen.
2. Der Treiber schaltet die Unterbrechungen dieses Geräts ab.
3. Er fragt in einer Schleife ab und verarbeitet, was da ist.
4. Ist nichts mehr da, schaltet er die Unterbrechungen wieder ein.

Bei geringer Last verhält sich das System wie ein unterbrechungsgesteuertes und reagiert schnell. Bei hoher Last verhält es sich wie ein abfragendes und bleibt arbeitsfähig. Das ist ein Entwurfsmuster, das man in Betriebssystemen häufig wiederfindet.

(b) Kopieren bei Bedarf.

Das Problem. Der Aufruf `fork` erzeugt eine Kopie des laufenden Prozesses. Bei einem Prozess mit 4 GB Speicher wäre das teuer.

Die Beobachtung. Meist folgt auf `fork` sofort ein `exec`, das den ganzen Adressraum verwirft. Die Kopie wäre umsonst gewesen.

Die Lösung.

1. Beide Prozesse bekommen Seitentabellen, die auf **dieselben** Rahmen zeigen.
2. Alle Einträge werden auf **nur lesen** gesetzt, auch die eigentlich schreibbaren.
3. Schreibt einer, so gibt es eine Schutzverletzung.
4. Der Kern erkennt die Lage, kopiert **diese eine** Seite, setzt beide Einträge auf schreibbar.
5. Der Befehl wird wiederholt und gelingt.

Die Rechnung.

Fall	**Kosten**	**Bemerkung**
vollständig kopieren	4 GB $\approx$ 0,4 s	
Tabellen kopieren	$\approx$ 1 MB $\approx$ 0,1 ms	bei 4 GB
je Schreibzugriff	$\approx 3\,\mu s$	Fehler + Kopie

Wenn der Kindprozess vor dem `exec` nur zehn Seiten anfasst:

$$0{,}1\ \text{ms} + 10 \cdot 3\,\mu\text{s} = 0{,}13\ \text{ms} \quad \text{statt} \quad 400\ \text{ms}.$$

Faktor 3000. Und der ganze Mechanismus beruht darauf, dass man eine Schutzverletzung absichtlich herbeiführt und im Kern behandelt, statt sie als Fehler zu melden.

Die unangenehme Folge

Ein Prozess kann an einem Speicherzugriff scheitern, der längst erfolgreich war. Wenn der Kern beim Kopieren keine freie Seite mehr findet, muss er etwas tun, und ein Rückgabewert steht nicht zur Verfügung.

> *Linux beantwortet das mit dem **OOM Killer**: Wenn der Speicher ausgeht, sucht der Kern einen Prozess aus und beendet ihn. Welchen, entscheidet eine Punktbewertung, in die Größe, Alter und einige Einstellungen eingehen.*

Das ist umstritten und die Folge einer früheren Entscheidung, nämlich der Überzeichnung: Der Kern sagt zu, mehr Speicher zu liefern, als er hat, weil die meisten Programme ihre Zusagen nicht ausschöpfen.

Überzeichnung	Anforderungen gelingen fast immer; im Ernstfall stirbt ein zufälliger Prozess
Keine Überzeichnung	Anforderungen schlagen ehrlich fehl; dafür scheitern Programme, die viel anfordern und wenig nutzen

Beide Wege haben Nachteile. Datenbanksysteme empfehlen meist, die Überzeichnung abzuschalten, weil sie lieber eine ehrliche Fehlermeldung bekommen als einen unerwarteten Tod.

(c) Der Unterbrechungssturm.
Der Fall. Ein defektes Gerät meldet sich ununterbrochen, oder ein Treiber quittiert die Meldung nicht richtig.

Symptom	Das System reagiert nicht mehr, obwohl die Prozessorlast im Werkzeug niedrig aussieht
Grund	Die Zeit in Unterbrechungsroutinen wird keinem Prozess zugerechnet
Erkennung	Unter Linux zeigt `/proc/interrupts` die Zähler je Quelle

Die Gegenmaßnahmen.

Abschalten	Der Kern zählt und deaktiviert eine Quelle, die zu oft meldet, mit einer Meldung im Protokoll
Drosselung	Das Gerät wird angewiesen, höchstens n Meldungen je Sekunde zu senden
Bündelung	Die Netzkarte sammelt mehrere Pakete und meldet einmal

Die dritte Zeile ist bei Netzkarten Standard und heißt **Interrupt Coalescing**. Sie ist ein Kompromiss zwischen Durchsatz und Verzögerung.

Bündelung	Durchsatz	Verzögerung
keine	niedrig	sehr klein
50 µs	hoch	+25 µs im Mittel
500 µs	sehr hoch	+250 µs

> *Wer Börsengeschäfte abwickelt, schaltet die Bündelung ab und nimmt den Durchsatzverlust hin. Wer Dateien überträgt, macht es umgekehrt. Dasselbe Gerät, entgegengesetzte Einstellung.*

Arpaci-Dusseau und Arpaci-Dusseau[2] behandeln Unterbrechungen und Seitenfehler ausführlich; Love[18] beschreibt die Umsetzung in Linux; Bovet und Cesati[6] gehen bis auf die Quelltextebene.

Part II

Prozesse

3 Prozesse und Fäden

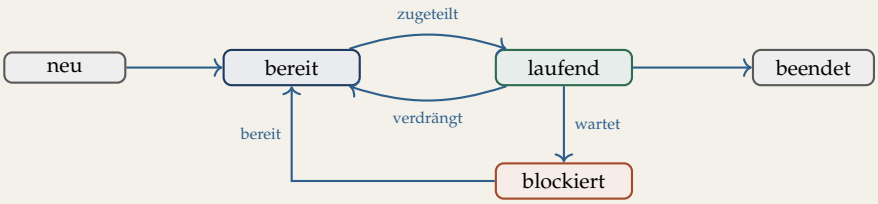
Was ein Prozess ist

Ein Prozess ist ein Programm in Ausführung, zusammen mit allem, was dazugehört.

Adressraum	Code, Daten, Halde, Stapel; eigene Seitentabelle
Registerinhalte	Befehlszähler, Stapelzeiger, alles Übrige
Offene Dateien	Tabelle der Dateideskriptoren
Rechte	Benutzer, Gruppe, Fähigkeiten
Verwandtschaft	Elternprozess, Kinder
Buchführung	verbrauchte Zeit, Speicher, Signale

Der Kern führt darüber eine Datenstruktur, die in Unix **Prozesskontrollblock** heißt und in Linux `task_struct`. Sie ist mit über tausend Zeilen eine der größten Strukturen des Kerns.

Die Zustände.



Der Übergang von „laufend“ nach „bereit“ ist der interessante. Er geschieht ohne Zutun des Prozesses, weil der Zeitgeber eine Unterbrechung auslöst. Ohne ihn könnte ein Programm den Rechner beliebig lange belegen.

Systeme, in denen dieser Übergang fehlt, heißen **kooperativ**. Ein Prozess gibt dort den Prozessor freiwillig ab. Windows tat das bis Version 3.11, das klassische Mac OS bis Version 9. Ein einziges hängengebliebenes Programm legte das ganze System lahm.

Fäden

Ein Faden ist ein Ausführungsstrang innerhalb eines Prozesses. Alle Fäden eines Prozesses teilen sich den Adressraum.

	je Prozess	**je Faden**
Adressraum	✓	
Offene Dateien	✓	
Rechte	✓	
Register		✓
Stapel		✓
Befehlszähler		✓

Der geteilte Adressraum ist der Grund für den Nutzen und für die Gefahr. Fäden können ohne Aufwand Daten austauschen, und sie können sich ohne Aufwand gegenseitig zerstören.

Vorgang	**Kosten**	**Bemerkung**
Prozess erzeugen	≈ 200 µs	`fork` + `exec`
Faden erzeugen	≈ 20 µs	
Prozesswechsel	≈ 3 µs	Seitentabelle, TLB
Fadenwechsel (gleicher Prozess)	≈ 1 µs	nur Register
Umschalten im Benutzerbetrieb	≈ 0,05 µs	Koroutinen

Die letzte Zeile ist der Grund für die leichten Fäden, die viele Sprachen heute anbieten: Goroutinen in Go, Fasern in Java seit Version 21, `async` in Rust und Python.

Leichte Fäden

Die Idee: Man führt viele Ausführungsstränge auf wenigen Betriebssystemfäden aus und schaltet im Benutzerbetrieb um.

	Kernfäden	**leichte Fäden**
Anzahl praktikabel	einige tausend	**Millionen**
Stapel	8 MB reserviert	2–8 KB, wächst
Umschalten	Kern	Laufzeitsystem
Blockierender Aufruf	blockiert einen Kernfaden	**wird umgelenkt**
Echte Parallelität	ja	nur über die Kernfäden

Die vierte Zeile ist der Kunstgriff. Wenn ein leichter Faden auf das Netz wartet, meldet das Laufzeitsystem den Wunsch bei einer Ereignisschnittstelle an und schaltet auf einen anderen Faden um. Der Kernfaden bleibt beschäftigt.

> *Damit lassen sich Programme im Stil „ein Faden je Verbindung“ schreiben, die intern wie ereignisgesteuerte Programme arbeiten. Man bekommt die Lesbarkeit des einen und die Leistung des anderen.*

Der Preis ist, dass ein leichter Faden, der einen blockierenden Systemaufruf umgeht oder rechenintensiv arbeitet, ohne abzugeben, seinen Kernfaden festhält. Go hat deshalb 2020 die Verdrängung leichter Fäden nachgerüstet.

Mittlere Datentechnik: Nixdorf und NIROS

Zwischen den Großrechnern und den späteren Personalcomputern lag eine Klasse von Maschinen, für die sich der deutsche Ausdruck **mittlere Datentechnik** eingebürgert hat. Sie waren für Betriebe gedacht, die keinen Rechenzentrumsbetrieb unterhalten wollten und trotzdem Fakturierung, Buchhaltung und Lagerhaltung auf dem Rechner führten.

Heinz Nixdorf baute darauf ab 1968 ein Unternehmen auf, das zeitweise zu den größten Computerherstellern Europas gehörte. Die Rechner standen beim Kunden im Betrieb, nicht in einem klimatisierten Saal, und wurden von Angestellten bedient, nicht von Operateuren.

Der Betriebsart nach waren es Mehrplatzsysteme: Ein Rechner, daran eine Reihe von Bildschirmarbeitsplätzen, an denen mehrere Personen gleichzeitig arbeiteten. Genau die Aufgabenstellung dieses Bandes, nur in kleinerem Maßstab als bei den Großrechnern.

Das Betriebssystem der Baureihe 8870 hieß NIROS. Programmiert wurde überwiegend in einem Business-BASIC, und Nixdorf lieferte mit COMET eine umfangreiche kaufmännische Anwendungssoftware dazu.

Zielgruppe	mittelständische Betriebe, Handwerk, Handel
Betriebsart	mehrere Bildschirmplätze an einem Rechner
Sprache	Business BASIC, auf kaufmännische Aufgaben zugeschnitten
Anwendungen	Auftragsbearbeitung, Fakturierung, Finanzbuchhaltung

Für diesen Band ist an dieser Welt zweierlei bemerkenswert.
Erstens der Zuschnitt der Sprache. Ein Business BASIC ist kein allgemeines BASIC. Es bringt Dateiverwaltung, Bildschirmmasken und Rechnen mit dezimalen Festkommazahlen als Sprachmittel mit. Wer eine kaufmännische Anwendung schrieb, brauchte keine Bibliothek, sondern nur die Sprache.

Dezimales Rechnen ist dabei kein Detail. Bei Geldbeträgen ist die binäre Gleitkommarechnung aus Band 0 unbrauchbar, weil sich 0,10 *nicht exakt darstellen lässt. Kaufmännische Systeme rechnen deshalb seit jeher dezimal, und der Befehlssatz mancher dieser Maschinen unterstützte das unmittelbar.*

Zweitens die Nebenläufigkeit. Sobald mehrere Bildschirmplätze auf dieselben Stammdaten zugreifen, stellt sich genau die Frage, um die es in Teil III geht: Was geschieht, wenn zwei Bedienende denselben Datensatz gleichzeitig ändern wollen? Kapitel 6 behandelt die Antwort, die diese Systeme darauf gefunden haben.

Nixdorf wurde 1990 von Siemens übernommen; aus der Verbindung entstand Siemens Nixdorf Informationssysteme. Das Heinz Nixdorf MuseumsForum in Paderborn bewahrt Geräte und Unterlagen dieser Zeit[12].

Die mittlere Datentechnik verschwand nicht, weil sie schlecht gewesen wäre, sondern weil vernetzte Personalcomputer und später Standardsoftware dieselben Aufgaben billiger erledigten. Die Aufgabenstellung selbst ist geblieben, und wer heute eine Warenwirtschaft betreut, löst dieselben Probleme mit anderen Mitteln.

Aufgabe 3.1.

(a) Verfolgen Sie `fork` und `exec` vollständig.
(b) Bestimmen Sie, wann Fäden und wann Prozesse die richtige Wahl sind.
(c) Untersuchen Sie, was ein Prozesswechsel tatsächlich kostet.

Lösung Schritt für Schritt.

(a) `fork` und `exec`.
Der ungewöhnliche Rückgabewert.

```
pid_t p = fork();
if (p == 0) { /* Kind */
execl("/bin/ls", "ls", NULL); }
else if (p > 0) { /* Eltern */
waitpid(p, &status, 0); }
else { /* Fehler */ }
```

Ein Aufruf, zwei Rückkehren. Der Elternprozess bekommt die Nummer des Kindes, das Kind bekommt null. Das ist ungewöhnlich und hat sich als überraschend brauchbar erwiesen.

Was der Kern bei `fork` tut.

1. Neue `task_struct` anlegen und die alte hineinkopieren.
2. Neue Prozessnummer vergeben.
3. Seitentabellen kopieren, alle Einträge auf nur lesen setzen (Kap. 2).
4. Die Tabelle der offenen Dateien kopieren; die Deskriptoren zeigen auf dieselben Einträge.
5. Rückgabewert im Kind auf null setzen.
6. Das Kind in die Warteschlange der bereiten Prozesse einhängen.

Punkt 4 hat eine überraschende Folge

Eltern und Kind teilen sich den **Dateizeiger**, nicht nur die Datei. Schreibt das Kind, so bewegt sich der Zeiger auch für die Eltern.

> *Genau darauf beruht die Umleitung in der Kommandozeile. Die Shell öffnet die Zieldatei, ruft `fork`, und das Kind erbt den Deskriptor. Es muss nur noch die Nummer 1 darauf setzen und dann `exec` rufen.*

Dasselbe gilt für Röhren. Ein `a | b` entsteht so: Röhre anlegen, zweimal `fork`, im ersten Kind das Schreibende auf Deskriptor 1 legen, im zweiten das Leseende auf Deskriptor 0.

Die ganze Kommandozeilenumgebung von Unix lässt sich aus `fork`, `exec`, `dup2` und `pipe` aufbauen. Das ist der Grund, warum sie so alt und so lebendig ist.

Was `exec` tut.

1. Den alten Adressraum vollständig abbauen.
2. Die neue Datei prüfen und ihren Kopf lesen.
3. Code- und Datenbereiche einblenden, ohne sie zu laden.
4. Stapel anlegen, Argumente und Umgebung hineinschreiben.
5. Befehlszähler auf den Einsprungpunkt setzen.

Punkt 3 ist bemerkenswert. Das Programm wird nicht geladen, sondern nur in den Adressraum abgebildet. Die Seiten kommen einzeln durch Seitenfehler, während es läuft.

> *Ein Programm von 100 MB startet deshalb ähnlich schnell wie eines von 1 MB, solange beide anfangs nur wenige Seiten anfassen.*

Warum `fork` und `exec` getrennt sind.
Zwischen beiden Aufrufen ist das Kind ein vollwertiger Prozess, der beliebige Vorbereitungen treffen kann: Deskriptoren umbiegen, Rechte ablegen, das Arbeitsverzeichnis wechseln.

	Unix	Windows
Aufrufe	`fork` + `exec`	`CreateProcess`
Vorbereitung	beliebiger Code	**Parameterstruktur**
Flexibilität	hoch	begrenzt
Aufwand	Adressraum duplizieren	gering

Der Windows-Aufruf hat über ein Dutzend Parameter, weil alles, was unter Unix als Code zwischen `fork` und `exec` steht, dort als Angabe übergeben werden muss.

Für Programme, die nur ein anderes starten wollen, gibt es unter Unix inzwischen `posix_spawn`, das beides zusammenfasst und den Umweg über die Kopie spart.

(b) Fäden oder Prozesse.

	Fäden	Prozesse
Datenaustausch	unmittelbar	über Röhren, Speicher, Netz
Isolation	keine	vollständig
Absturz eines Teils	trifft alle	trifft nur ihn
Kosten	gering	höher
Fehleranfälligkeit	hoch	geringer

Ein Beispiel für jede Wahl.

Anwendung	Wahl	Grund
Numerische Rechnung	Fäden	gemeinsame Datenfelder
Webbrowser	**Prozesse**	eine Seite darf keine andere gefährden
Datenbankserver	gemischt	Fäden je Verbindung, Prozesse für Hilfsdienste
Kommandozeile	Prozesse	natürliche Grenze

Die zweite Zeile ist eine Entwicklung der Jahre nach 2008. Browser benutzten zuvor Fäden, weil das billiger war. Nach einer Reihe von Sicherheitsvorfällen wechselte man zu einem Prozess je Herkunftsseite und nimmt den Speicherverbrauch in Kauf.

Ousterhouts Einwand

JOHN OUSTERHOUT hielt 1996 einen Vortrag mit dem Titel „Why Threads Are a Bad Idea (for most purposes)“. Sein Argument in Kurzform:

1. Fäden verlangen Synchronisation, und Synchronisation ist schwer richtig hinzubekommen.
2. Die Fehler treten selten und nicht reproduzierbar auf.
3. Für die meisten Aufgaben genügt ein ereignisgesteuertes Programm mit einem einzigen Faden.
4. Echte Parallelität braucht man nur bei rechenintensiven Aufgaben.

Der Vortrag traf einen Nerv und wurde viel diskutiert. Ousterhout lag richtig darin, dass Fäden gefährlich sind. Er lag darin daneben, dass ereignisgesteuerte Programme mit vielen Rückrufen ihre eigene Unübersichtlichkeit erzeugen.

Die heutige Antwort versucht beides zu vereinen: leichte Fäden, die sich wie Fäden schreiben lassen und intern ereignisgesteuert arbeiten. Ob das die Synchronisationsprobleme löst, ist eine andere Frage. Es löst sie nicht; es macht nur den Umgang mit blockierenden Aufrufen billiger.

(c) Was ein Prozesswechsel kostet.
Die unmittelbaren Kosten.

Schritt	Takte	
Register retten	≈ 50	
Ablaufplanung	≈ 200	Auswahl des nächsten
Seitentabelle wechseln	≈ 300	
Register herstellen	≈ 50	
unmittelbar	≈ 600	0,15 µs

Die mittelbaren Kosten.
Hier liegt der eigentliche Aufwand. Der neue Prozess findet Caches und TLB vor, die mit den Daten des alten gefüllt sind.

Struktur	Wiederauffüllen	Kosten
L1-Datencache	≈ 500 Zugriffe	≈ 7000 Takte
L2-Cache	teilweise	≈ 3000 Takte
TLB	≈ 100 Einträge	≈ 2000 Takte
Sprungvorhersager	neu lernen	≈ 1000 Takte
mittelbar		≈ 13 000 Takte

Die mittelbaren Kosten sind zwanzigmal so hoch wie die unmittelbaren. Sie tauchen in keiner Messung des Wechsels selbst auf, sondern verteilen sich über die ersten Mikrosekunden danach.

Die Folgerung für die Ablaufplanung.
Bei einer Zeitscheibe von 1 ms bei 4 GHz stehen $4 \cdot 10^6$ Takte zur Verfügung. Der Wechsel kostet etwa 14 000, also 0,35 Prozent. Das ist tragbar.

Zeitscheibe	Verlust	Reaktionszeit
0,1 ms	3,5 %	sehr gut
1 ms	0,35 %	gut
10 ms	0,035 %	spürbar
100 ms	0,0035 %	unbrauchbar

Kapitel 4 zeigt, wie man diesen Kompromiss auflöst, ohne sich auf eine feste Zeitscheibe festlegen zu müssen. Arpaci-Dusseau und Arpaci-Dusseau[2] behandeln Prozesse und Fäden ausführlich; Silberschatz und Mitautoren[27] geben die breite Übersicht; Ousterhouts Vortrag[22] ist als Folienfassung frei verfügbar und in zwanzig Minuten gelesen.

4 Ablaufplanung

Widersprüchliche Ziele

Der Ablaufplaner entscheidet, welcher bereite Prozess als Nächstes den Prozessor bekommt. Die Ziele, die er dabei verfolgen soll, widersprechen einander.

Durchsatz	Möglichst viele Aufträge je Zeiteinheit
Antwortzeit	Möglichst schnell auf eine Eingabe reagieren
Gerechtigkeit	Jeder soll seinen Anteil bekommen
Termintreue	Bestimmte Aufgaben müssen rechtzeitig fertig sein
Wirkungsgrad	Der Planer selbst soll wenig Zeit verbrauchen

> *Durchsatz und Antwortzeit stehen unmittelbar gegeneinander. Wer selten wechselt, verliert wenig Zeit an Wechsel und lässt interaktive Programme warten. Wer oft wechselt, reagiert schnell und verschwendet Zeit.*

Deshalb gibt es keinen besten Ablaufplaner, sondern Verfahren, die auf verschiedene Lasten zugeschnitten sind.

Die Grundverfahren

Verfahren	Regel	Problem
FIFO	wer zuerst kommt	langer Auftrag blockiert alle
Kürzester zuerst	kürzeste Rechenzeit	Rechenzeit ist unbekannt
Rundlauf	feste Zeitscheibe reihum	Scheibenlänge zu wählen
Prioritäten	höchste Priorität	**Verhungern**
Vielstufig	Prioritäten mit Rückkopplung	viele Stellschrauben

Warum „kürzester zuerst“ optimal und unbrauchbar ist.
Man kann beweisen, dass dieses Verfahren die mittlere Wartezeit minimiert. Es setzt voraus, dass man die Rechenzeit im Voraus kennt, und das tut man nie.

> *Der übliche Ausweg besteht darin, die Zukunft aus der Vergangenheit zu schätzen. Ein Prozess, der bisher oft blockiert hat, ist vermutlich interaktiv und bekommt eine höhere Priorität.*

Verbraucht seine Scheibe	Vermutlich rechenintensiv; Priorität sinkt, Scheibe wird länger
Blockiert vorher	Vermutlich interaktiv; Priorität steigt, Scheibe wird kürzer

Damit bekommt ein Texteditor, der auf Tastendrücke wartet, hohe Priorität und reagiert sofort. Ein Übersetzungslauf bekommt niedrige Priorität und lange Scheiben, was ihn effizient macht.

Prioritäten führen zum Verhungern

Bei strengen Prioritäten kommt ein Prozess niedriger Priorität nie an die Reihe, solange ein höherer bereit ist.

> *Die übliche Abhilfe heißt **Alterung**: Die Priorität eines wartenden Prozesses steigt mit der Wartezeit. Irgendwann überholt er alle anderen.*

Ein bekannter Fall trat 1997 auf dem Mars auf. Der Rover Pathfinder setzte sich mehrfach selbst zurück, weil ein Prozess niedriger Priorität eine Sperre hielt, die ein Prozess hoher Priorität brauchte, und ein mittlerer Prozess den niedrigen verdrängte.

*Man nennt das **Prioritätsumkehr**. Kapitel 7 behandelt sie und ihre Behebung, die im Fall Pathfinder aus der Ferne eingespielt wurde.*

Der Linux-Planer

Linux hat seinen Planer mehrfach ausgetauscht.

Jahr	Verfahren	Bemerkung
1991	einfache Liste	$\mathcal{O}(n)$ je Entscheidung
2001	O(1)-Planer	konstante Zeit, viele Heuristiken
2007	**CFS**	**Rot-Schwarz-Baum,** $\mathcal{O}(\log n)$
2023	EEVDF	Nachfolger von CFS

Der Grundgedanke von CFS.
Statt Zeitscheiben zu verteilen, führt der Planer für jeden Prozess Buch über die verbrauchte Rechenzeit, gewichtet mit seiner Priorität. Er wählt immer denjenigen mit dem kleinsten Wert.

$$\text{vruntime} \mathrel{+}= \Delta t \cdot \frac{\text{Gewicht}_0}{\text{Gewicht}_i}.$$

> *Ein Prozess mit doppeltem Gewicht sammelt die halbe virtuelle Zeit je verbrauchter echter Zeit und kommt deshalb doppelt so oft an die Reihe. Die Prioritäten wirken damit multiplikativ und nicht als strenge Rangfolge.*

nice-Wert	Gewicht	Anteil bei zwei
-5	3121	75 %
0	1024	50 %
$+5$	335	25 %
$+19$	15	1,5 %

Ein neu erzeugter oder aufgewachter Prozess bekommt die virtuelle Zeit des am wenigsten fortgeschrittenen Prozesses, damit er nicht sofort alles an sich reißt und nicht ewig hinten steht.

Die Zeitscheibe verschwindet

CFS hat keine feste Zeitscheibe. Er verteilt eine **Zielverzögerung** auf alle bereiten Prozesse.

$$\text{Scheibe}_i = \text{Zielverzögerung} \cdot \frac{\text{Gewicht}_i}{\sum_j \text{Gewicht}_j}.$$

Prozesse	Scheibe	Bemerkung
2	12 ms	bei 24 ms Zielverzögerung
8	3 ms	
24	1 ms	**Mindestwert erreicht**
100	1 ms	Zielverzögerung wird überschritten

> *Unterhalb einer Mindestscheibe von etwa einer Millisekunde hört der Planer auf, weiter zu teilen, weil sonst die Wechselkosten überhandnehmen. Bei sehr vielen Prozessen verlängert sich dann die Reaktionszeit.*

Das Nachfolgeverfahren EEVDF geht einen Schritt weiter und gibt jedem Prozess eine *Frist*. Es kann damit Prozesse bevorzugen, die kurze Bearbeitungen brauchen, ohne die Gerechtigkeit über längere Zeiträume aufzugeben.

Aufgabe 4.1.

(a) Rechnen Sie mittlere Wartezeiten für verschiedene Verfahren aus.
(b) Untersuchen Sie den Konvoieffekt.
(c) Analysieren Sie die Ablaufplanung bei mehreren Kernen.

Lösung Schritt für Schritt.

(a) Wartezeiten im Vergleich.
Die Aufträge.

Auftrag	Ankunft	Rechenzeit
A	0	8
B	1	4
C	2	9
D	3	5

Verfahren 1 – FIFO.

Auftrag	Beginn	Ende	Wartezeit
A	0	8	0
B	8	12	7
C	12	21	10
D	21	26	18

$$\overline{W} = \frac{0+7+10+18}{4} = 8{,}75.$$

Verfahren 2 – Kürzester zuerst, ohne Verdrängung.
Bei Zeitpunkt 8 sind B (4), C (9), D (5) da. Reihenfolge: A, B, D, C.

Auftrag	Beginn	Ende	Wartezeit
A	0	8	0
B	8	12	7
D	12	17	9
C	17	26	15

$$\overline{W} = \frac{0+7+9+15}{4} = 7{,}75.$$

Verfahren 3 – Kürzeste Restzeit, mit Verdrängung.

Zeit	läuft	Grund
0–1	*A*	einziger
1–5	*B*	Rest $4 < 7$
5–10	*D*	Rest $5 < 7$
10–17	*A*	Rest $7 < 9$
17–26	*C*	

Auftrag	Ende	Verweilzeit	Wartezeit
A	17	17	9
B	5	4	0
C	26	24	15
D	10	7	2

$$\overline{W} = \frac{9 + 0 + 15 + 2}{4} = 6{,}5.$$

Verfahren 4 – Rundlauf, Scheibe 2.
Nach dem Durchspielen der Reihenfolge ergibt sich:

Auftrag	Ende	Verweilzeit	Wartezeit
A	24	24	16
B	14	13	9
C	26	24	15
D	22	19	14

$$\overline{W} = \frac{16 + 9 + 15 + 14}{4} = 13{,}5.$$

Der Vergleich.

Verfahren	$\overline{W}$	Reaktionszeit
FIFO	8,75	schlecht
Kürzester zuerst	7,75	mittel
Kürzeste Restzeit	**6,50**	gut
Rundlauf	13,50	**sehr gut**

Der Rundlauf hat die schlechteste mittlere Wartezeit und die beste Reaktionszeit. Kein Auftrag wartet länger als drei Scheiben, bis er zum ersten Mal an die Reihe kommt. Für einen interaktiven Betrieb ist genau das die wichtige Größe.

(b) Der Konvoieffekt.
Der Fall. Ein rechenintensiver Prozess und mehrere Prozesse, die viel auf Geräte warten, teilen sich einen Rechner.

Prozess	**Rechnen**	**Warten**
R	1000 ms	0
$E_1 \ldots E_5$	1 ms	10 ms Platte

Mit FIFO.
Sobald R läuft, warten alle anderen 1000 ms. Die Platte steht dabei still.

Größe	**Wert**	
Zyklusdauer	1005 ms	
Plattenauslastung	$\approx 5\,\%$	steht meistens still
Vorgänge je Sekunde	≈ 5	

Mit Rundlauf, Scheibe 10 ms.
Die wartenden Prozesse geben nach 1 ms freiwillig ab, weil sie blockieren. Sie kommen also fast sofort wieder dran.

Größe	**Wert**	
Plattenauslastung	$\approx 100\,\%$	durchgehend
Vorgänge je Sekunde	≈ 450	
R verliert	$\approx 45\,\%$	

Faktor 90 beim Durchsatz der wartenden Prozesse, für 45 Prozent Verlust beim rechenintensiven. Das ist fast immer ein guter Tausch, denn die Platte ist teuer und soll ausgelastet sein.

Die Bevorzugung wartender Prozesse ist deshalb in jedem allgemeinen Betriebssystem eingebaut. Sie ist auch der Grund, warum ein Rechner unter voller Rechenlast

noch bedienbar bleibt: Der Texteditor blockiert ständig auf Tastatureingaben und behält deshalb hohe Priorität.

(c) Ablaufplanung bei mehreren Kernen.
Die neue Frage. Nicht nur *wann*, sondern auch *wo*.

	eine Warteschlange	**je Kern eine**
Lastausgleich	automatisch	muss betrieben werden
Sperrkonflikte	alle Kerne, eine Sperre	keine
Cachewirkung	schlecht	gut
Skalierung	bis etwa 8 Kerne	darüber hinaus

Alle heutigen Systeme benutzen Warteschlangen je Kern und gleichen periodisch aus.
Warum die Cachewirkung so wichtig ist.
Ein Prozess, der auf einem anderen Kern weiterläuft, findet dort einen kalten Cache vor.

Fortsetzung auf	**Zusatzkosten**	**Grund**
demselben Kern	≈ 0	Cache noch warm
Nachbarkern, gleicher L3	$\approx 10\,000$ Takte	L1, L2 kalt
anderer Sockel	$\approx 100\,000$ Takte	**auch der Speicher ist fern**

> *Der Planer bevorzugt deshalb den zuletzt benutzten Kern und verschiebt einen Prozess nur, wenn die Last erheblich ungleich verteilt ist. Man nennt das* ***Kernaffinität****.*

Wann man von Hand eingreift.

Festbinden	Ein Prozess wird auf bestimmte Kerne festgelegt (`taskset`, `sched_setaffinity`)
Kerne freihalten	Bestimmte Kerne werden dem allgemeinen Betrieb entzogen (`isolcpus`)
Echtzeitklassen	`SCHED_FIFO` und `SCHED_RR` verdrängen alles Übrige

Echtzeitprioritäten sind gefährlich

Ein Prozess in der Klasse `SCHED_FIFO` läuft, bis er freiwillig abgibt oder blockiert. Eine Endlosschleife in einem solchen Prozess macht den Kern unbedienbar.

> *Linux hat deshalb eine Drossel eingebaut: Echtzeitprozesse dürfen standardmäßig höchstens 95 Prozent jeder Sekunde bekommen. Das rettet das System und macht die Echtzeitgarantie zugleich unvollständig.*

Wer echte Termingarantien braucht, kommt mit einem allgemeinen Betriebssystem an Grenzen. Die Antworten heißen Echtzeitbetriebssysteme wie QNX oder VxWorks, oder eine Erweiterung wie PREEMPT_RT, die 2024 nach über zwanzig Jahren Entwicklung in den Linux-Hauptzweig aufgenommen wurde.

> *Deren Ziel ist nicht Geschwindigkeit, sondern eine obere Schranke für die Verzögerung. Ein Echtzeitsystem darf langsamer sein, solange es berechenbar ist.*

Arpaci-Dusseau und Arpaci-Dusseau[2] entwickeln die Verfahren mit vielen Beispielen; Love[18] beschreibt CFS im Detail; Anderson und Dahlin[1] behandeln die Mehrkernseite.

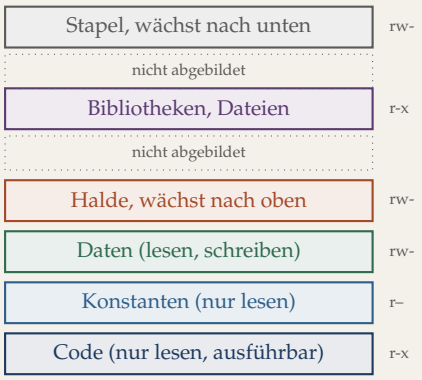
5 Speicherverwaltung

Was der Kern über den Speicher weiß

Band 4, Kapitel 9 hat die Seitentabellen aus Sicht der Hardware beschrieben. Hier geht es darum, wer sie füllt und nach welchen Regeln.

Physisch	Welche Rahmen sind frei, welche belegt, wer benutzt sie
Virtuell	Welche Bereiche hat jeder Prozess angefordert und wofür
Verdrängung	Welche Seite muss weichen, wenn es eng wird

Der Adressraum eines Prozesses besteht nicht aus einer durchgehenden Fläche, sondern aus einer Liste von Bereichen mit verschiedenen Eigenschaften.



> *Die nicht abgebildeten Lücken sind Absicht. Ein Zeiger, der versehentlich dorthin zeigt, löst sofort einen Fehler aus, statt stillschweigend fremde Daten zu verändern.*

Ganz unten liegt immer eine nicht abgebildete Seite, damit ein Zugriff über einen Nullzeiger zuverlässig abstürzt statt Unsinn zu tun.

Adressraumverwürfelung.
Seit den 2000er Jahren werden die Anfangsadressen der Bereiche bei jedem Start zufällig verschoben. Das erschwert Angriffe, die eine feste Adresse brauchen.

Bereich	Zufallsbits	Möglichkeiten
Stapel	≈ 30	10^9
Halde	≈ 28	$2 \cdot 10^8$
Bibliotheken	≈ 28	$2 \cdot 10^8$

Das ist keine Sicherheit im strengen Sinn, sondern eine Erschwernis. Wer eine Adresse aus dem Programm herausbekommt, hat die ganze Verschiebung. Kapitel 14 kommt darauf zurück.

Verdrängungsverfahren

Wenn kein Rahmen mehr frei ist, muss eine Seite weichen. Die Frage ist, welche.

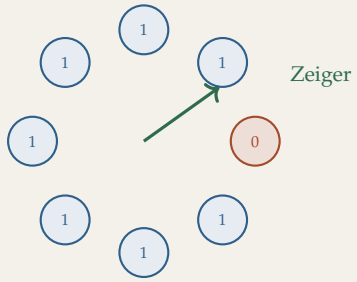
Verfahren	Regel	Bewertung
Optimal	die am längsten nicht gebrauchte	nicht umsetzbar
FIFO	die älteste	schlecht
LRU	die längst nicht benutzte	gut, teuer
Zweite Chance	FIFO mit Nutzungsbit	**gute Näherung**
Arbeitsmenge	alles, was zuletzt gebraucht wurde	aufwendig

Das optimale Verfahren setzt Kenntnis der Zukunft voraus. Es dient als Maßstab, an dem man die anderen misst.

Warum LRU nicht direkt umsetzbar ist.
Man müsste bei jedem Speicherzugriff einen Zeitstempel aktualisieren, also bei jedem einzelnen Befehl. Das kostet mehr, als die Verdrängung einspart.

Die Hardware bietet stattdessen ein einziges Bit je Seite, das bei jedem Zugriff gesetzt wird. Der Kern liest und löscht es regelmäßig und weiß dadurch, ob eine Seite im letzten Zeitraum benutzt wurde.

Der Uhrzeigeralgorithmus.



1. Der Zeiger läuft im Kreis über alle Rahmen.
2. Ist das Nutzungsbit 1, so wird es auf 0 gesetzt und weitergegangen.
3. Ist es 0, so wird diese Seite verdrängt.

Eine Seite bekommt also eine zweite Chance, wenn sie seit der letzten Umrundung benutzt wurde. Das nähert LRU erstaunlich gut an und kostet fast nichts.

Linux benutzt eine verfeinerte Fassung mit zwei Listen: einer für aktive und einer für inaktive Seiten. Eine Seite muss zweimal benutzt werden, um in die aktive Liste zu kommen, und das verhindert, dass ein einmaliger Durchlauf durch eine große Datei den ganzen Cache verdrängt.

Speicher im Kern zuteilen

Der Kern selbst braucht ebenfalls Speicher, und zwar in sehr verschiedenen Größen.

Bedarf	**Größe**	**Zuteiler**
Seitenrahmen	4 KB und mehr	Kumpelverfahren
Kernstrukturen	32–512 B	**Slab-Zuteiler**
Große Blöcke	beliebig	`vmalloc`

Das Kumpelverfahren.
Der Speicher wird in Blöcke von 2^k Seiten eingeteilt. Wird ein kleinerer Block gebraucht, so halbiert man einen größeren; werden zwei benachbarte Hälften frei, so verschmelzen sie wieder.



Der Vorteil ist, dass sich benachbarte Blöcke sehr schnell finden lassen: Die Adresse des Kumpels ergibt sich durch Umkehren eines einzigen Bits. Der Nachteil ist innerer Verschnitt von bis zu fünfzig Prozent.

Der Slab-Zuteiler.
JEFF BONWICK beschrieb 1994 für Solaris ein Verfahren, das auf einer einfachen Beobachtung beruht: Der Kern legt immer wieder Strukturen desselben Typs an und gibt sie wieder frei.

1. Für jeden Typ gibt es einen eigenen Vorrat aus ganzen Seiten.
2. In jeder Seite liegen möglichst viele Strukturen dieses Typs.
3. Freigegebene Strukturen bleiben **initialisiert** liegen.
4. Beim nächsten Anfordern entfällt die Initialisierung.

Punkt 3 ist der eigentliche Einfall. Bei einer Struktur mit eingebauten Sperren und Listen kostet das Aufsetzen mehr als das Zuteilen. Wenn man sie im aufgesetzten Zustand aufbewahrt, spart man das.

Vorgang	**Takte**	**Bemerkung**
Allgemeiner Zuteiler	≈ 400	Suche, Initialisierung
Slab, mit Vorrat	≈ 30	Listenkopf abhängen
Slab, Vorrat leer	≈ 400	neue Seite holen

Hinzu kommt ein zweiter Vorteil, der erst bei mehreren Kernen zählt: Jeder Kern hat einen eigenen kleinen Vorrat und braucht dafür keine Sperre.

Aufgabe 5.1.

(a) Vergleichen Sie Verdrängungsverfahren an einer Zugriffsfolge.
(b) Untersuchen Sie das Belady-Phänomen.
(c) Analysieren Sie, was beim Seitenflattern geschieht.

Lösung Schritt für Schritt.

(a) Verdrängungsverfahren im Vergleich.
Zugriffsfolge: $7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2$ bei drei Rahmen.
FIFO.

Zugriff	7	0	1	2	0	3	…
R1	7	7	7	**2**	2	2	
R2		0	0	0	0	**3**	
R3			1	1	1	1	
Fehler	✓	✓	✓	✓	—	✓	

Über die ganze Folge: **9 Fehler**.
LRU.
Bei jedem Fehler weicht die am längsten nicht benutzte Seite.

Zugriff	2	0	3	0	4	2	3
Inhalt	7,0,1→ 2,0,1	2,0,1	2,0,3	2,0,3	4,0,3	4,0,2	4,3,2
Fehler	✓	—	✓	—	✓	✓	✓

Über die ganze Folge: **8 Fehler**.
Optimal.
Verdrängt wird die Seite, die am spätesten wieder gebraucht wird.

Über die ganze Folge: **7 Fehler**.

Der Vergleich.

Verfahren	**Fehler**	**Abstand zum Optimum**
Optimal	7	—
LRU	8	+14 %
Zweite Chance	≈ 8	+14 %
FIFO	9	+29 %

Bei realen Programmen ist der Abstand größer, aber die Rangfolge bleibt. LRU und seine Näherungen liegen typischerweise zehn bis dreißig Prozent über dem Optimum, FIFO deutlich schlechter.

(b) Das Belady-Phänomen.
Die Behauptung. Mehr Rahmen führen zu weniger Fehlern. Das klingt selbstverständlich und ist bei FIFO falsch.
Das Gegenbeispiel. Folge 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5.
Mit drei Rahmen.

Zugriff	1	2	3	4	1	2	5	1	2
Fehler	✓	✓	✓	✓	✓	✓	✓	—	—

Insgesamt: **9 Fehler**.
Mit vier Rahmen.

Zugriff	1	2	3	4	1	2	5	1	2
Fehler	✓	✓	✓	✓	—	—	✓	✓	✓

Insgesamt: **10 Fehler**.

> *Ein zusätzlicher Rahmen erzeugt einen zusätzlichen Fehler.* László Bélády *beschrieb das 1969, und es wirkte zunächst wie ein Rechenfehler.*

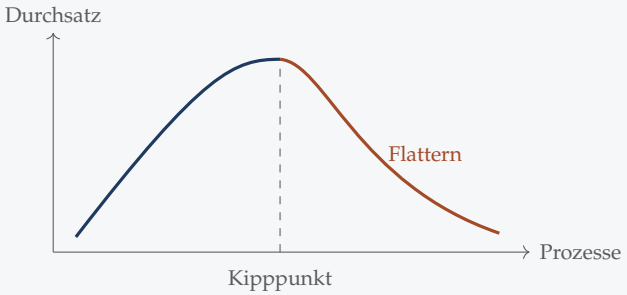
Warum LRU davon verschont bleibt.
LRU gehört zu den **Stapelverfahren**. Bei ihnen gilt: Der Inhalt bei n Rahmen ist stets eine Teilmenge des Inhalts bei $n + 1$ Rahmen.

> *Daraus folgt unmittelbar, dass jeder Treffer bei n Rahmen auch bei $n + 1$ ein Treffer ist. Mehr Speicher kann also nicht schaden.*

FIFO hat diese Eigenschaft nicht, weil die Verdrängungsreihenfolge nicht von der Nutzung abhängt und sich bei anderer Rahmenzahl völlig anders entwickelt.

Praktisch spielt das Phänomen keine große Rolle, weil kaum ein System reines FIFO benutzt. Als Warnung ist es trotzdem lehrreich: Auch offensichtliche Eigenschaften sollte man nachweisen.

(c) Seitenflattern.
Der Zustand. Die Summe der Arbeitsmengen aller Prozesse übersteigt den vorhandenen Speicher. Jeder verdrängt die Seiten der anderen, und alle warten auf die Platte.



Der Durchsatz bricht nicht allmählich ein, sondern stürzt ab. Jeder zusätzliche Prozess macht die Lage schlechter, weil er allen anderen Speicher wegnimmt.

Die Rechnung.

Fehlerrate	Mittlere Zugriffszeit	Verlangsamung
10^{-6}	65 ns	1,0
10^{-4}	75 ns	1,2
10^{-3}	165 ns	2,5
10^{-2}	1065 ns	**16**
10^{-1}	10,1 µs	155

(mit 65 ns für einen Speicherzugriff und 100 µs für einen Seitenwechsel von SSD)

Was man dagegen tut.

Arbeitsmengen-modell	Der Kern schätzt, wie viele Seiten jeder Prozess braucht, und lässt nur so viele Prozesse zu, wie hineinpassen
Auslagern ganzer Prozesse	Statt einzelner Seiten wird ein ganzer Prozess ausgelagert und später zurückgeholt
Grenzen setzen	Jeder Prozess oder jede Gruppe bekommt eine Obergrenze (cgroups)
Beenden	Der OOM-Killer aus Kapitel 2

Wenn Flattern besser wäre als Beenden

Linux neigt dazu, lange zu flattern, bevor es einen Prozess beendet. Für einen Anwender ist das die schlechteste aller Möglichkeiten: Das System reagiert minutenlang nicht, statt einen Fehler zu melden.

> *Es gibt deshalb Zusatzdienste, die den Speicherdruck überwachen und früher eingreifen, etwa* `earlyoom` *oder* `systemd-oomd`*. Sie benutzen die Druckanzeige des Kerns, die seit 2018 verfügbar ist.*

Diese Anzeige, unter Linux `/proc/pressure`, gibt an, welcher Anteil der Zeit Prozesse auf Speicher, Ein-/Ausgabe oder Prozessor gewartet haben. Sie ist eine der nützlichsten Messgrößen für die Frage, warum ein System langsam ist, und wird erstaunlich wenig benutzt.

Arpaci-Dusseau und Arpaci-Dusseau[2] behandeln die Verdrängungsverfahren mit vielen Beispielen; Bonwicks Arbeit[5] über den Slab-Zuteiler ist kurz und lesenswert; Bovet und Cesati[6] beschreiben die Umsetzung in Linux.

Part III

Nebenläufigkeit

6 Wettläufe und wechselseitiger Ausschluss

Das Problem genau gefasst

Ein **Wettlauf** liegt vor, wenn das Ergebnis davon abhängt, in welcher Reihenfolge nebenläufige Zugriffe stattfinden. Das Beispiel aus dem Vorwort ist der einfachste Fall.

Definition 6.1. Ein **kritischer Abschnitt** ist ein Codestück, das nicht gleichzeitig von mehreren Ausführungssträngen durchlaufen werden darf.

Eine Lösung des Ausschlussproblems muss vier Eigenschaften haben.

Ausschluss	Höchstens einer ist gleichzeitig im kritischen Abschnitt
Fortschritt	Ist niemand drin und wollen welche hinein, so kommt einer hinein
Begrenztes Warten	Niemand wartet unbegrenzt lange
Keine Annahmen	Über die relative Geschwindigkeit der Stränge wird nichts vorausgesetzt

Die vierte Bedingung schließt die naheliegende Lösung aus, einfach lange genug zu warten. Ein Verfahren, das nur funktioniert, wenn der eine Strang schneller ist als der andere, ist keine Lösung.

Warum das schwerer ist, als es aussieht

Der erste Versuch sieht meist so aus:

```
while (besetzt) { /* warten */ }
besetzt = 1;
/* kritischer Abschnitt */
besetzt = 0;
```

Das ist falsch, und zwar aus demselben Grund wie das Zählerbeispiel: Zwischen der Prüfung und dem Setzen liegt ein Zeitfenster. Zwei Stränge können beide die Null sehen und beide hineingehen.

> *Das Verschieben des Setzens vor die Prüfung hilft auch nicht. Dann setzen beide und beide sehen die Eins, und niemand kommt hinein.*

Die Schwierigkeit liegt darin, dass „prüfen und setzen“ zwei Vorgänge sind. Man braucht entweder ein sehr sorgfältiges Protokoll mit mehreren Variablen oder eine Hardwareunterstützung, die beides zu einem Vorgang verbindet.

Petersons Verfahren

GARY PETERSON veröffentlichte 1981 eine Lösung für zwei Stränge, die mit gewöhnlichen Lese- und Schreibzugriffen auskommt. Sie ist die einfachste bekannte und ersetzte Dijkstras kompliziertere Fassung von 1965.

```
bool will[2] = {false, false};
int dran = 0;
```

```
/* Strang i, der andere ist j = 1-i */
will[i] = true;
dran = j;      Höflichkeit: der andere zuerst
while (will[j] && dran == j) { /*
warten */ }
/* kritischer Abschnitt */
will[i] = false;
```

Der Einfall steckt in der Zeile `dran = j`. Jeder Strang erklärt seinen Willen und tritt dann ausdrücklich zurück. Wenn beide gleichzeitig eintreten, überschreibt der zweite die Variable, und damit gewinnt der erste.

Der Beweis des Ausschlusses.
Angenommen, beide sind gleichzeitig im kritischen Abschnitt. Dann haben beide ihre Schleife verlassen, also galt für Strang 0

$$\neg \text{will}[1] \ \vee \ \text{dran} \neq 1$$

und für Strang 1 entsprechend

$$\neg \text{will}[0] \ \vee \ \text{dran} \neq 0.$$

Beide haben ihr `will` auf `true` gesetzt und vor dem Eintritt nicht zurückgenommen. Also müssen die jeweils zweiten Bedingungen gelten:

$$\text{dran} \neq 1 \quad \text{und} \quad \text{dran} \neq 0.$$

Die Variable hat aber nur diese beiden Werte. Widerspruch.

Auf heutigen Rechnern funktioniert es trotzdem nicht

Der Beweis setzt voraus, dass die Zugriffe in der aufgeschriebenen Reihenfolge sichtbar werden. Genau das garantiert kein Prozessor (Band 4, Kapitel 11).

Der Prozessor	darf das Schreiben von `dran` hinter das Lesen von `will[j]` schieben
Der Übersetzer	darf die Variablen in Register laden und die Schleife wegoptimieren

Man braucht also Speicherbarrieren, in C und C++ ausgedrückt durch `std::atomic` mit passender Ordnung. Dann funktioniert Petersons Verfahren wieder, ist aber langsamer als eine Sperre aus einem atomaren Befehl.

Der praktische Wert liegt heute darin, dass es zeigt, was ohne Hardwareunterstützung möglich ist. Benutzen sollte man es nicht.

Atomare Befehle

Jeder heutige Prozessor bietet Befehle, die Lesen und Schreiben untrennbar verbinden.

Befehl	Wirkung	Rückgabe
Test-and-Set	setzt auf 1	alter Wert
Fetch-and-Add	addiert	alter Wert
Compare-and-Swap	tauscht, falls erwarteter Wert	Erfolg
Load-Linked / Store-Conditional	zwei Befehle mit Wächter	Erfolg

Compare-and-Swap ist der mächtigste der drei ersten. Man kann alles andere darauf zurückführen, und praktisch alle sperrfreien Verfahren beruhen darauf.

```
bool CAS(int *ort, int erwartet, int
neu) {
  /* atomar:  */
  if (*ort == erwartet) { *ort = neu;
return true; }
  else return false; }
```

Die einfachste Sperre.

```
void sperren(int *s) { while (!CAS(s,
0, 1)) { } }
void freigeben(int *s) { *s = 0; }
```

Diese **Drehsperre** ist korrekt und in dieser Form schlecht. Sie erzeugt bei jedem Versuch einen Schreibzugriff, und die Cachezeile wandert zwischen allen wartenden Kernen hin und her (Band 4, Kapitel 11).

Fassung	Zeit bei 8 Kernen	Bemerkung
CAS in der Schleife	4,8 s	Zeile wandert ständig
erst lesen, dann CAS	1,1 s	**Test-and-Test-and-Set**
mit Zurückweichen	0,7 s	wachsende Pausen
Warteschlange (MCS)	0,25 s	jeder dreht lokal

Die zweite Zeile ist eine Änderung von wenigen Zeichen und bringt Faktor vier. Man prüft mit einem gewöhnlichen Lesezugriff, ob die Sperre frei aussieht, und versucht erst dann den atomaren Befehl. Solange sie belegt ist, bleibt die Zeile im Zustand „gemeinsam“.

Die Satzsperre in der kaufmännischen Datenverarbeitung

Das Ausschlussproblem trat außerhalb der Betriebssystemforschung zuerst dort auf, wo mehrere Bildschirmplätze an einem Rechner hingen: in der mittleren Datentechnik (Kapitel 3).
Der Fall ist alltäglich. Zwei Bedienende rufen denselben Kundendatensatz auf, ändern verschiedene Felder und speichern. Ohne Vorkehrung überschreibt der zweite die Änderung des ersten.

Zeit	Platz 1	Platz 2
t_1	liest Satz 4711	
t_2		liest Satz 4711
t_3	ändert Anschrift	
t_4		ändert Kreditgrenze
t_5	schreibt zurück	
t_6		schreibt zurück

*Nach t_6 ist die neue Anschrift verschwunden, weil Platz 2 den ganzen Satz mit seinem älteren Stand überschrieben hat. Man nennt das die **verlorene Aktualisierung**, und es ist derselbe Fehler wie beim Zähler aus dem Vorwort, nur mit Minuten statt Nanosekunden dazwischen.*

Die Antwort dieser Systeme war die **Satzsperre**. Beim Lesen zur Änderung wird der Datensatz gesperrt und erst beim Zurückschreiben oder beim Abbruch wieder freigegeben. Der zweite Platz bekommt eine Meldung, dass der Satz gerade bearbeitet wird.

Sperrgegenstand	nicht die Datei, sondern der einzelne Satz
Dauer	über die ganze Bearbeitung, also Sekunden bis Minuten
Konflikt	wird der Bedienenden gemeldet, nicht im Programm behandelt

Die Feinheit dieser Lösung liegt in der Wahl der Sperrgröße. Eine ganze Datei zu sperren wäre einfach und würde den Mehrplatzbetrieb unmöglich machen. Einzelne Felder zu sperren wäre feiner und den Aufwand nicht wert.

*Der Datensatz ist die Einheit, in der kaufmännisch gedacht wird, und deshalb die richtige Sperrgröße. Diese Überlegung gilt unverändert, und heutige Datenbanksysteme führen dieselbe Abwägung unter dem Namen **Sperrgranularität**.*

Ein Unterschied zum Rest dieses Kapitels verdient Beachtung. Eine Sperre, die über eine Benutzereingabe hinweg gehalten wird, ist etwas anderes als eine Sperre über zwanzig Maschinenbefehle.

	im Kern	**über eine Maske**
Dauer	Nanosekunden	**Minuten**
Warten	drehen	Meldung an den Benutzer
Wer sie hält	ein Faden	**ein Mensch**
Risiko	Verklemmung	**Mittagspause**

Die letzte Zeile ist ernst gemeint. Wer einen Satz zur Bearbeitung öffnet und zu Tisch geht, blockiert ihn für alle. Die Systeme brauchten deshalb Zeitschranken und eine Möglichkeit für den Systembetreuer, eine Sperre aufzuheben.

Genau diese Erfahrung führte später zu den **optimistischen** Verfahren, bei denen man nicht sperrt, sondern beim Zurückschreiben prüft, ob sich der Satz inzwischen geändert hat. Kapitel 12 kommt darauf zurück.

Aufgabe 6.1.

(a) Analysieren Sie den Wettlauf am Zähler vollständig.
(b) Vergleichen Sie Drehsperren und blockierende Sperren rechnerisch.
(c) Untersuchen Sie, wie man Wettläufe findet.

Lösung Schritt für Schritt.

(a) Der Wettlauf im Detail.
Die Verschränkung, die den Fehler erzeugt.

Zeit	Faden A	Faden B	Zähler
t_1	liest $\rightarrow$ 100		100
t_2		liest $\rightarrow$ 100	100
t_3	rechnet 101		100
t_4		rechnet 101	100
t_5	schreibt 101		101
t_6		schreibt 101	**101**

Zwei Erhöhungen, eine Wirkung. Eine ist verlorengegangen.
Wie viele Erhöhungen verlorengehen.
Das Fenster umfasst die drei Befehle. Bei einer Schleife von etwa fünf Befehlen je Durchlauf ist es also etwa 60 Prozent der Zeit offen.

Bedingung	Verlustrate	Bemerkung
ein Kern, Zeitscheiben	sehr gering	Wechsel selten
zwei Kerne	$\approx 30\,\%$	
acht Kerne	$\approx 80\,\%$	

Die erste Zeile ist der Grund, warum solche Fehler jahrelang unentdeckt bleiben können. Auf einem Einkernrechner tritt der Fehler nur auf, wenn die Zeitscheibe genau in das Dreibefehlsfenster fällt, also etwa einmal in einer Million Durchläufe.

Die Lösungen im Vergleich.

Fassung	Zeit (2 Fäden)	richtig?
`zaehler++`	0,004 s	nein
mit Mutex	0,31 s	ja
`atomic_fetch_add`	0,09 s	ja
lokale Zähler	0,003 s	**ja**

Die letzte Zeile ist die richtige Antwort und zugleich die schnellste. Jeder Faden zählt in einer eigenen Variablen, und am Ende wird einmal addiert. Kein gemeinsamer Zustand, kein Problem.

(b) Drehen oder blockieren.
Die Größen.

Größe	Wert	
Kosten des Drehens	$\approx$ 1 Takt je Runde	
Kosten des Blockierens	$\approx$ 4000 Takte	hin und zurück
Länge des kritischen Abschnitts	L Takte	

Die Entscheidungsregel.

Drehen lohnt, wenn $L < 4000$ Takte, also unter 1 µs.

L	Drehen	Blockieren	besser
50	50	4050	drehen
1000	1000	5000	drehen
4000	4000	8000	drehen
10^5	10^5	$1{,}04 \cdot 10^5$	etwa gleich
10^7	10^7	4000	blockieren

Die letzte Zeile stimmt nur, wenn ein anderer Faden die Zeit nutzen kann. Beim Blockieren wird der Prozessor frei; beim Drehen wird er verbrannt. Auf einem einzigen Kern ist Drehen sogar tödlich, weil der Sperrhalter nie an die Reihe kommt.

Die übliche Lösung.
Ein **adaptiver Mutex** dreht zunächst eine begrenzte Zeit und blockiert dann. Er nimmt damit den besten Fall mit und begrenzt den schlechtesten.

1. Ein paar Runden drehen, meist einige hundert.
2. Prüfen, ob der Sperrhalter überhaupt gerade läuft. Wenn nicht, sofort blockieren.
3. Sonst weiterdrehen bis zu einer Schranke, dann blockieren.

Punkt 2 ist die wichtigste Verfeinerung. Wenn der Sperrhalter selbst verdrängt wurde, ist Drehen sinnlos, denn die Sperre kann sich erst lösen, wenn er wieder läuft. Linux nutzt dafür eine Angabe, die der Planer bereitstellt.

(c) Wettläufe finden.
Die Werkzeuge.

Werkzeug	Verfahren	Kosten
ThreadSanitizer	Vorher-Beziehung verfolgen	5–15-fach
Helgrind	dasselbe, in Valgrind	20–100-fach
Modellprüfer	alle Verschränkungen	nur kleine Modelle
Stresstest	viel Last, Daumen drücken	gering, unzuverlässig

Wie ThreadSanitizer arbeitet.
Es verfolgt die Beziehung „geschieht vor“ zwischen Ereignissen, die LESLIE LAMPORT 1978 für verteilte Systeme eingeführt hat[16]. Zwei Zugriffe auf dieselbe Speicherstelle, von denen mindestens einer schreibt und zwischen denen keine solche Beziehung besteht, sind ein Wettlauf.

Das Verfahren findet auch Wettläufe, die im konkreten Lauf gar nicht schiefgegangen sind. Es genügt, dass die beiden Zugriffe stattfanden und keine Synchronisation dazwischenlag.

Eigenschaft	**Bewertung**	
Falsche Meldungen	selten	gut
Übersehene Wettläufe	möglich	nur auf ausgeführten Pfaden
Verlangsamung	5–15-fach	tragbar für Tests
Speicherbedarf	5–10-fach	

Die zweite Zeile ist die Einschränkung. Das Werkzeug sieht nur, was ausgeführt wird. Ein Wettlauf in einem Zweig, den der Test nicht betritt, bleibt unentdeckt.

Was man tun sollte, statt Wettläufe zu suchen

Die zuverlässigste Vorgehensweise besteht darin, gemeinsamen veränderlichen Zustand von vornherein zu vermeiden.

Nichts teilen	Jeder Faden arbeitet auf eigenen Daten; Ergebnisse am Ende zusammenführen
Unveränderlich	Geteilte Daten werden nie geändert, sondern ersetzt
Eigentum	Zu jedem Zeitpunkt gehört ein Datum genau einem Faden; die Sprache prüft das
Nachrichten	Statt gemeinsamen Speichers werden Werte verschickt

Die dritte Zeile beschreibt den Weg, den Rust geht. Der Übersetzer verfolgt, wem ein Wert gehört, und lehnt Programme ab, in denen zwei Fäden gleichzeitig schreibend zugreifen könnten.

> *Damit wird eine ganze Fehlerklasse zu einem Übersetzungsfehler statt zu einem seltenen Laufzeitfehler. Das ist der stärkste Fortschritt auf diesem Gebiet seit den Monitoren von 1974.*

Die vierte Zeile beschreibt den Weg von Erlang, Go und CSP (Kapitel 9). Die Go-Regel dazu lautet: Kommuniziere nicht durch das Teilen von Speicher, sondern teile Speicher durch Kommunikation.

Dijkstras Arbeiten[9,10] stehen am Anfang; Peterson[23] gab die einfachste Lösung; Herlihy und Shavit[13] behandeln atomare Befehle und Sperren gründlich.

7 Semaphore und Monitore

Der Semaphor

EDSGER DIJKSTRA führte 1965 ein Werkzeug ein, das mehr kann als eine Sperre. Ein Semaphor ist ein Zähler mit zwei Operationen.

Operation	Wirkung	ursprünglicher Name
`wait`	warten bis > 0, dann -1	**P** (*passeren*)
`signal`	$+1$, einen Wartenden wecken	**V** (*vrijgeven*)

Beide Operationen sind unteilbar. Die niederländischen Namen stehen für „passieren“ und „freigeben“; Dijkstra hat später eingeräumt, dass P und V für Nichtniederländer wenig aussagen.
Zwei Verwendungsweisen.

	Anfangswert	Zweck
Binärer Semaphor	1	wechselseitiger Ausschluss
Zählender Semaphor	n	n gleiche Betriebsmittel
Signalsemaphor	0	**Reihenfolge erzwingen**

Die dritte Zeile ist die interessanteste und wird oft übersehen. Ein Semaphor mit Anfangswert null zwingt einen Faden zu warten, bis ein anderer ihm ein Zeichen gibt.

Faden A	Faden B
`ergebnis = rechne();`	`wait(fertig);`
`signal(fertig);`	`benutze(ergebnis);`

Damit ist sichergestellt, dass B das Ergebnis erst nach A liest, gleichgültig welcher Faden zuerst startet. Der Semaphor drückt hier keine Ausschließung aus, sondern eine Reihenfolge.

Semaphore sind fehleranfällig

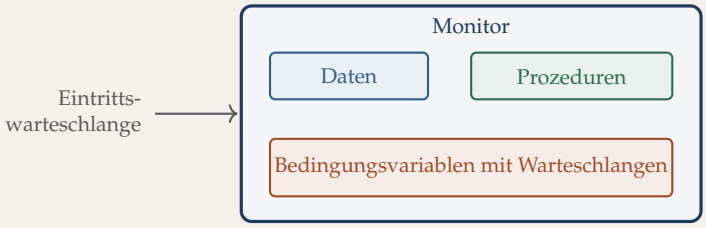
`signal` vergessen	Alle warten für immer
`wait` vergessen	Der Ausschluss ist wirkungslos, und niemand merkt es
Reihenfolge vertauscht	Bei zwei Semaphoren entsteht eine Verklemmung (Kap. 8)
Falscher Semaphor	Der Übersetzer sieht keinen Zusammenhang zwischen Semaphor und geschützten Daten

Die letzte Zeile ist der grundsätzliche Einwand. Ein Semaphor ist eine Variable, die zufällig neben den Daten liegt, die sie schützt. Es gibt keine Verbindung, die eine Sprache prüfen könnte.

Genau daran setzte Hoare 1974 an.

Der Monitor

Ein **Monitor** ist eine Sprachkonstruktion. Er fasst Daten und die darauf arbeitenden Prozeduren zusammen und garantiert, dass höchstens ein Faden gleichzeitig darin ist.



Der Ausschluss ist damit nicht mehr Sache des Programmierers, sondern des Übersetzers. Wer eine Monitorprozedur aufruft, nimmt die Sperre; wer sie verlässt, gibt sie frei. Vergessen ist nicht möglich.

Bedingungsvariablen.
Der reine Ausschluss genügt nicht. Man braucht auch das Warten auf einen Zustand, etwa „der Puffer ist nicht mehr leer“.

`wait(c)`	Gibt die Monitorsperre frei und legt sich schlafen. Beim Aufwachen wird die Sperre wieder genommen
`signal(c)`	Weckt einen Wartenden, falls einer da ist. Sonst geschieht nichts
`broadcast(c)`	Weckt alle Wartenden

Der erste Punkt ist entscheidend und leicht zu übersehen: `wait` gibt die Sperre frei. Andernfalls käme niemand herein, um die Bedingung wahr zu machen, und es entstünde eine Verklemmung.

Der Unterschied zum Semaphor.

	Semaphor	**Bedingungsvariable**
Gedächtnis	**ja**, zählt	**nein**
`signal` ohne Wartende	erhöht den Zähler	verpufft

Deshalb muss man eine Bedingungsvariable immer mit einer eigenen Zustandsvariablen kombinieren und die Bedingung in einer Schleife *prüfen, nicht in einem `if`.*

falsch	**richtig**
`if (!bereit) wait(c);`	`while (!bereit) wait(c);`

Warum die Schleife nötig ist

Es gibt drei Gründe, und jeder allein genügt.

Mehrere Wartende	Bei `broadcast` wachen alle auf, aber nur für einen ist die Bedingung erfüllt
Dazwischengekommen	Zwischen dem Wecken und dem Wiedererlangen der Sperre kann ein Dritter die Bedingung wieder zerstören
Grundloses Aufwachen	POSIX erlaubt ausdrücklich, dass `pthread_cond_wait` ohne Anlass zurückkehrt

Der dritte Punkt überrascht viele. Die Norm erlaubt das, weil es die Implementierung erheblich vereinfacht und weil eine korrekt geschriebene Schleife ohnehin damit umgeht.

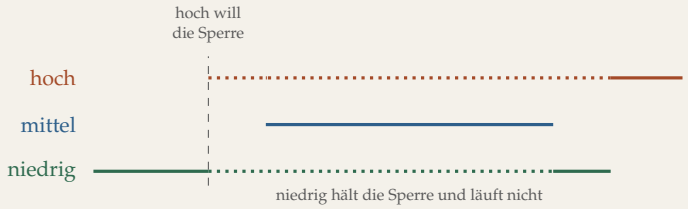
Der zweite Punkt hängt an einer Entwurfsentscheidung, die Hoare und seine Nachfolger verschieden getroffen haben.

	Hoare-Semantik	**Mesa-Semantik**
Nach `signal`	der Geweckte läuft **sofort**	der Wecker läuft weiter
Bedingung gilt	garantiert	**muss geprüft werden**
Umsetzung	aufwendig	einfach
Verbreitung	selten	**überall**

Praktisch alle heutigen Systeme benutzen die Mesa-Semantik, benannt nach der Sprache Mesa von Xerox PARC. Sie ist der Grund, warum die Schleife unverzichtbar ist.

Prioritätsumkehr

Ein Faden hoher Priorität wartet auf eine Sperre, die ein Faden niedriger Priorität hält. Das ist unvermeidlich und kurz. Unangenehm wird es, wenn ein dritter Faden mittlerer Priorität den niedrigen verdrängt.



Der Faden hoher Priorität wartet nun beliebig lange, obwohl er der wichtigste ist. Der mittlere hat ihn faktisch verdrängt, ohne je mit ihm in Berührung zu kommen.

Die Abhilfen.

Prioritätsvererbung	Wer eine Sperre hält, auf die ein wichtigerer wartet, erbt dessen Priorität für die Dauer
Prioritätsobergrenze	Jede Sperre hat eine feste Priorität; wer sie nimmt, wird darauf angehoben

Der Mars Pathfinder, Juli 1997

Der Rover war gelandet, sendete Daten und setzte sich in unregelmäßigen Abständen selbst zurück. Jedes Mal gingen die Daten des laufenden Tages verloren.
Die Ursache lag in einem Bus, über den mehrere Aufgaben Daten austauschten, geschützt durch einen Mutex.

Aufgabe	Priorität	Rolle
Busverwaltung	hoch	muss regelmäßig laufen
Kommunikation	mittel	lang laufend
Wetterdaten	niedrig	schreibt in den Bus

Wenn die Wetterdatenaufgabe die Sperre hielt und die Kommunikationsaufgabe sie verdrängte, kam die Busverwaltung nicht an die Reihe. Ein Wachhund stellte fest, dass sie ihre Frist versäumt hatte, und setzte das System zurück.

> *Das Team am Jet Propulsion Laboratory reproduzierte den Fehler auf einem Duplikat am Boden, nachdem es die Ablaufverfolgung eingeschaltet hatte. Der Fehler trat dort nach etwa achtzehn Stunden auf.*

Das benutzte Betriebssystem VxWorks bot Prioritätsvererbung an, sie war aber für diesen Mutex nicht eingeschaltet. Die Behebung bestand darin, ein einziges Kennzeichen zu setzen.

Der Patch wurde zum Mars gesendet und eingespielt. Der Rover arbeitete danach ohne weitere Rücksetzungen.

GLENN REEVES, der Leiter der Softwareentwicklung, hat den Vorgang später ausführlich beschrieben. Sein wichtigster Punkt: Der Fehler war nicht deshalb behebbar, weil man ihn vorhergesehen hatte, sondern weil das System eine Ablaufverfolgung aus der Ferne erlaubte.

> *Das ist die Lehre, die über den Einzelfall hinausgeht. Bei nebenläufigen Systemen muss man damit rechnen, Fehler im Betrieb suchen zu müssen, und die Möglichkeit dazu von vornherein einbauen.*

Aufgabe 7.1.

(a) Bauen Sie einen beschränkten Puffer mit Semaphoren und mit einem Monitor.
(b) Zeigen Sie, dass die Reihenfolge der Semaphoroperationen wesentlich ist.
(c) Untersuchen Sie Lese-Schreib-Sperren und ihre Tücken.

Lösung Schritt für Schritt.

(a) Der beschränkte Puffer.
Die Aufgabe. Erzeuger legen Werte in einen Ringpuffer der Größe N, Verbraucher nehmen sie heraus. Der Puffer darf weder überlaufen noch leer gelesen werden.
Fassung mit Semaphoren.

```
semaphore leer = N;      freie Plätze
semaphore voll = 0;      belegte Plätze
semaphore mutex = 1;
```

```
Erzeuger:                Verbraucher:
wait(leer);              wait(voll);
wait(mutex);             wait(mutex);
einfuegen(x);            x =
entnehmen();
signal(mutex);           signal(mutex);
signal(voll);            signal(leer);
```

Warum drei Semaphore.

`leer`	Zählt die freien Plätze. Ein Erzeuger wartet, wenn keiner frei ist
`voll`	Zählt die belegten. Ein Verbraucher wartet, wenn nichts da ist
`mutex`	Schützt die Zeiger des Ringpuffers vor gleichzeitigem Ändern

Die beiden Zählsemaphore ergänzen sich zu N: `leer` + `voll` + „gerade in Bearbeitung“ = N. Diese Invariante ist der Kern der Korrektheit.

Fassung mit Monitor.

```
monitor Puffer {
  int anzahl = 0; condition nichtvoll,
nichtleer;
  procedure einfuegen(x) {
    while (anzahl == N)
wait(nichtvoll);
    ablegen(x); anzahl++;
    signal(nichtleer); }
  procedure entnehmen() {
    while (anzahl == 0)
wait(nichtleer);
    x = holen(); anzahl−;
    signal(nichtvoll); return x; } }
```

	Semaphore	**Monitor**
Ausschluss	von Hand	automatisch
Bedingung	im Zähler versteckt	steht da
Lesbarkeit	mittel	gut
Fehlerrisiko	hoch	geringer

In der Monitorfassung steht die Bedingung wörtlich im Code: „solange der Puffer voll ist, warte“. In der Semaphorfassung muss man erst erschließen, dass `wait(leer)` *genau das bedeutet.*

(b) Die Reihenfolge der Operationen.
Die vertauschte Fassung.

```
Erzeuger:    wait(mutex);  wait(leer);
...
```

Was geschieht.

Zeit	Erzeuger	Verbraucher
t_1	`wait(mutex)` gelingt	
t_2	`wait(leer)`: Puffer voll, **blockiert**	
t_3		`wait(voll)` gelingt
t_4		`wait(mutex)`: **blockiert**

Der Erzeuger hält den Mutex und wartet auf einen freien Platz. Der Verbraucher könnte einen schaffen, braucht dafür aber den Mutex. Beide warten für immer.

Das ist eine Verklemmung, erzeugt durch das Vertauschen zweier Zeilen. Kapitel 8 behandelt die allgemeine Form.

Die Regel.

> *Man nimmt nie eine Sperre, während man auf eine Bedingung wartet, die nur ein anderer erfüllen kann, der dieselbe Sperre braucht. Kurz: Erst die Zählsemaphore, dann den Mutex.*

Bei einem Monitor kann dieser Fehler nicht auftreten, weil `wait` die Sperre freigibt. Das ist der praktische Hauptvorteil der Konstruktion.

(c) Lese-Schreib-Sperren.
Die Idee. Viele dürfen gleichzeitig lesen, aber Schreiben schließt alles andere aus.

	frei	Leser	Schreiber
Leser will	ja	ja	nein
Schreiber will	ja	nein	nein

Die naive Umsetzung und ihr Fehler.

```
Leser:    wait(mutex); leser++;
          if (leser == 1)
wait(schreibsperre);
          signal(mutex); ...  lesen ...
```

Solange ständig neue Leser kommen, sinkt der Zähler nie auf null, und der Schreiber wartet für immer. Das ist kein Fehler im engen Sinn, sondern Verhungern.

Die drei möglichen Vorrangregeln.

Regel	Wirkung	Problem
Leser zuerst	höchster Durchsatz	Schreiber verhungern
Schreiber zuerst	Aktualität	Leser verhungern
Warteschlange	niemand verhungert	etwas langsamer

Die dritte Regel gewinnt in der Praxis. Wer ankommt, stellt sich an, und Leser, die hintereinander ankommen, dürfen gemeinsam eintreten.

Wann sich Lese-Schreib-Sperren lohnen

Sie sind aufwendiger als ein einfacher Mutex, weil sie mehr Zustand verwalten.

Sperre nehmen	Takte
Mutex, frei	≈ 20
Lese-Schreib-Sperre, lesend	≈ 60

Der Gewinn entsteht erst, wenn der kritische Abschnitt lang ist und weit überwiegend gelesen wird. Bei kurzen Abschnitten ist ein einfacher Mutex schneller, auch wenn er Leser unnötig ausschließt.

Lohnt sich	Lange Lesevorgänge, Verhältnis über 10:1, wenige Schreiber
Lohnt sich nicht	Kurze Abschnitte, häufiges Schreiben
Besser	RCU, wenn die Daten selten geändert werden

Die letzte Zeile verweist auf ein Verfahren, das PAUL MCKENNEY für den Linux-Kern entwickelt hat. Leser brauchen dabei überhaupt keine Sperre; der Schreiber baut eine neue Fassung der Datenstruktur auf und tauscht sie mit einem einzigen Zeigerzugriff ein. Die alte Fassung wird erst freigegeben, wenn sicher kein Leser sie mehr benutzt.

Der Lesevorgang kostet damit null zusätzliche Takte. Der Preis ist, dass Leser für eine kurze Zeit eine veraltete Fassung sehen, und dass die Freigabe kompliziert wird.

Dijkstras Aufsatz von 1968[10] führt Semaphore ein; Hoare[14] beschreibt den Monitor; Brinch Hansen[11] setzte ihn erstmals in einer Sprache um; McKenney[19] behandelt RCU ausführlich.

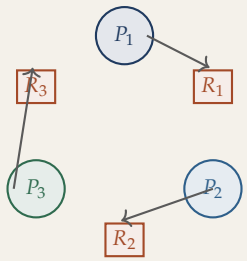
8 Verklemmungen

Die vier Bedingungen

EDWARD COFFMAN, MICHAEL ELPHICK und ARIE SHOSHANI zeigten 1971, dass eine Verklemmung genau dann möglich ist, wenn vier Bedingungen zugleich gelten.

Ausschluss	Ein Betriebsmittel kann nur einer zugleich benutzen
Halten und Warten	Wer etwas hat, kann zusätzlich auf mehr warten
Kein Entzug	Ein Betriebsmittel kann nicht weggenommen werden
Zyklisches Warten	Es gibt einen Ring von Wartenden

Das Bemerkenswerte an dieser Aufstellung ist, dass sie eine Handlungsanweisung enthält. Wer eine der vier Bedingungen bricht, kann keine Verklemmung mehr bekommen.



jeder hält eines
und will das nächste

Vier Strategien

Strategie	Vorgehen	Praxis
Verhindern	eine Bedingung ausschließen	üblich
Vermeiden	vor jeder Zuteilung prüfen	selten
Erkennen	zulassen und auflösen	Datenbanken
Ignorieren	nichts tun	**die meisten Systeme**

Die letzte Zeile ist keine Faulheit, sondern eine Abwägung. Verklemmungen sind selten, die Verhinderung ist teuer, und ein Neustart ist billiger als ein dauerhaft langsameres System. Man nennt das nach TANENBAUM *den Vogel-Strauß-Ansatz.*

Wie man verhindert.

Ausschluss	Kaum zu brechen; manche Betriebsmittel sind nun einmal exklusiv
Halten und Warten	Alles auf einmal anfordern. Führt zu schlechter Auslastung
Kein Entzug	Bei Misserfolg alles zurückgeben und neu versuchen. Kann zu Endlosschleifen führen
Zyklus	**Sperren immer in derselben Reihenfolge nehmen**

Die letzte Zeile ist die einzige Regel, die in der Praxis durchgängig angewandt wird. Man legt eine Ordnung über alle Sperren fest und verlangt, dass sie in aufsteigender Reihenfolge genommen werden.

Ein Zyklus setzt voraus, dass irgendwo eine Sperre mit kleinerer Nummer nach einer mit größerer genommen wird. Wenn das nie geschieht, kann es keinen Zyklus geben.

Wie Linux das prüft

Der Kern hat einen eingebauten Prüfer namens `lockdep`. Er merkt sich zur Laufzeit, in welcher Reihenfolge Sperren genommen wurden, und baut daraus einen Graphen.

1. Sperren werden nach Entstehungsort zu Klassen zusammengefasst.
2. Bei jedem Nehmen wird eine Kante von den bereits gehaltenen zur neuen eingetragen.
3. Entsteht dabei ein Zyklus, gibt es eine Warnung mit vollständiger Ablaufverfolgung.

Der Prüfer meldet die Gefahr, sobald die problematische Reihenfolge einmal *vorkommt, auch wenn es nie zu einer tatsächlichen Verklemmung kommt. Das ist der entscheidende Vorteil gegenüber dem Abwarten.*

Er verlangsamt den Kern um etwa den Faktor zwei und ist deshalb nur in Entwicklungsübersetzungen eingeschaltet. In dieser Rolle hat er über die Jahre eine große Zahl von Fehlern gefunden, die sonst unentdeckt geblieben wären.

Aufgabe 8.1.

(a) Konstruieren Sie eine Verklemmung und lösen Sie sie durch Sperrordnung auf.
(b) Führen Sie den Bankiersalgorithmus an einem Beispiel durch.
(c) Untersuchen Sie, wie Datenbanken mit Verklemmungen umgehen.

Lösung Schritt für Schritt.

(a) Verklemmung und Auflösung.
Der Fall. Eine Überweisung sperrt beide Konten.

```
void ueberweisen(Konto *von, Konto
*nach, int betrag) {
  sperren(&von->m);
  sperren(&nach->m);
  von->stand -= betrag; nach->stand +=
betrag;
  freigeben(&nach->m);
freigeben(&von->m); }
```

Die Verklemmung.

Zeit	Faden 1: A → B	Faden 2: B → A
t_1	sperrt A	
t_2		sperrt B
t_3	will B: **wartet**	
t_4		will A: **wartet**

Alle vier Coffman-Bedingungen sind erfüllt. Der Zyklus hat die Länge zwei.

Die Auflösung durch Ordnung.

```
Konto *erst = (von->nr < nach->nr) ?
von :  nach;
Konto *dann = (von->nr < nach->nr) ?
nach :  von;
sperren(&erst->m);  sperren(&dann->m);
...   freigeben(&dann->m);
freigeben(&erst->m);
```

Beide Fäden nehmen jetzt zuerst das Konto mit der kleineren Nummer. Faden 2 wartet auf A, ohne B zu halten, und Faden 1 kann durchlaufen.

Der Sonderfall.
Bei einer Überweisung von einem Konto auf sich selbst würde man dieselbe Sperre zweimal nehmen und sich selbst verklemmen. Das muss man gesondert abfangen:

```
if (von == nach) return;
```

*Solche Selbstverklemmungen sind ein häufiger Fehler und der Grund, warum manche Bibliotheken **wiedereintrittsfähige** Mutexe anbieten, die derselbe Faden mehrfach nehmen darf.*

(b) Der Bankiersalgorithmus.
DIJKSTRA entwarf ihn 1965 nach dem Vorbild eines Bankiers, der Kredite nur vergibt, wenn er alle Kunden noch bedienen kann.
Die Ausgangslage. Drei Prozesse, ein Betriebsmittel mit 12 Einheiten.

Prozess	**Höchstbedarf**	**hat**	**braucht noch**
P_1	10	5	5
P_2	4	2	2
P_3	9	2	7
Summe		9	**frei: 3**

Ist dieser Zustand sicher?
Gesucht ist eine Reihenfolge, in der alle Prozesse fertig werden können.

Schritt	**frei**	**wer kann**	**danach frei**
1	3	P_2 braucht 2	$3+2+2=7$
2	7	P_1 braucht 5	$7+5+5=12$
3	12	P_3 braucht 7	12

Sichere Folge: $\langle P_2, P_1, P_3 \rangle$. ✓

Eine Anforderung prüfen.
P_3 fordert eine weitere Einheit. Der Bankier tut so, als hätte er sie gegeben:

Prozess	**hat**	**braucht noch**
P_1	5	5
P_2	2	2
P_3	3	6
		frei: 2

Schritt	frei	wer kann	danach frei
1	2	P_2 braucht 2	$2+2+2=6$
2	6	P_1 braucht 5	$6+5+5=12$
3	12	P_3 braucht 6	12

Der Zustand bleibt sicher. Die Anforderung wird bewilligt. ✓

Eine Anforderung, die abgelehnt wird.
Stattdessen fordert P_1 zwei Einheiten. Dann bleiben nur 1 frei, P_1 braucht noch 3, P_2 noch 2, P_3 noch 7.

Keiner kann mit einer freien Einheit fertig werden. Der Zustand wäre unsicher, und die Anforderung wird abgelehnt, obwohl genug da wäre.

Warum das Verfahren nicht benutzt wird

Höchstbedarf	Muss im Voraus bekannt sein. Kaum ein Programm weiß das
Prozesszahl	Ändert sich ständig; Prozesse kommen und gehen
Aufwand	$\mathcal{O}(n^2 m)$ bei jeder einzelnen Anforderung
Zu vorsichtig	Lehnt Anforderungen ab, die tatsächlich unproblematisch wären

Der Algorithmus ist ein Lehrstück und kein Werkzeug. Sein Wert liegt darin, dass er den Unterschied zwischen „genug vorhanden“ und „sicher“ klarmacht.

In eingeschränkter Form findet man den Gedanken trotzdem wieder, etwa bei Datenbanken, die eine Transaktion abweisen, wenn sie den Sperrvorrat erschöpfen würde.

(c) Datenbanken und Verklemmungen.
Warum sie hier erkennen statt verhindern.
Eine Transaktion kann nicht im Voraus angeben, welche Sätze sie sperren wird, weil sich das erst aus den gelesenen Daten ergibt.

1. Das System führt einen Wartegraphen mit einem Knoten je Transaktion.
2. Wartet T_1 auf eine Sperre von T_2, so wird eine Kante eingetragen.
3. Regelmäßig, etwa jede Sekunde, wird auf Zyklen geprüft.
4. Bei einem Zyklus wird ein **Opfer** gewählt und zurückgerollt.

Die Wahl des Opfers.

Jüngste	Hat am wenigsten Arbeit verloren
Wenigste Sperren	Rollt am billigsten zurück
Wenigste Rücksetzungen	Verhindert, dass immer dieselbe stirbt

Die dritte Zeile ist wichtig, weil sonst ein Verhungern entstehen kann: Eine große Transaktion wird immer wieder als Opfer gewählt und kommt nie zum Abschluss.

Was das für die Anwendung bedeutet.
Ein Programm, das Datenbanktransaktionen benutzt, muss damit rechnen, dass eine Transaktion mit einem Verklemmungsfehler abgebrochen wird, und sie dann wiederholen.

```
for (versuch = 0; versuch < 5;
versuch++) {
  begin();
  if (arbeit() == OK) { commit();
break; }
  rollback();
  warte(zufaellig(10 « versuch)); /*
exponentiell */ }
```

Die zufällige, wachsende Wartezeit ist wesentlich. Wiederholen beide Beteiligten sofort, so laufen sie unter Umständen erneut in dieselbe Verklemmung, und das beliebig oft.

Dieses Muster, exponentiell wachsendes zufälliges Zurückweichen, taucht in der Informatik an sehr vielen Stellen auf: bei Ethernet, bei Netzanfragen, bei Drehsperren. Es ist eine der brauchbarsten Faustregeln überhaupt.

Wie oft das vorkommt.

Lastart	Verklemmungen	Bemerkung
Wenige kurze Transaktionen	$< 0{,}01\,\%$	vernachlässigbar
Gemischt, mittlere Last	0,1–1 %	Wiederholung nötig
Lange Transaktionen, hohe Last	$> 5\,\%$	Entwurf überdenken

Bei der letzten Zeile hilft kein besseres Erkennungsverfahren, sondern nur eine Änderung der Zugriffsmuster: kürzere Transaktionen, einheitliche Reihenfolge beim Zugriff auf Tabellen, oder ein optimistisches Verfahren ohne Sperren. Coffman und Mitautoren[7] stellten die vier Bedingungen auf; Silberschatz und Mitautoren[27] behandeln den Bankiersalgorithmus ausführlich; Tanenbaum und Bos[28] haben den Vogel-Strauß-Ansatz benannt.

9 Die klassischen Aufgaben

Warum es diese Aufgaben gibt

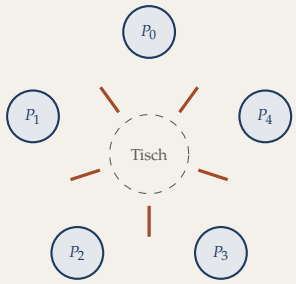
Dijkstra und seine Nachfolger formulierten eine Reihe von Aufgaben, die je eine typische Schwierigkeit in Reinform zeigen. Sie sind bis heute die übliche Prüfmenge für jedes neue Synchronisationswerkzeug.

Erzeuger-Verbraucher	Pufferung und Flusssteuerung
Leser-Schreiber	Ungleiche Zugriffsarten, Verhungern
Philosophen	Verklemmung durch zyklische Anforderung
Schlafender Friseur	Warteraum begrenzter Größe
Rauchende Zigaretten	Verteilung ungleicher Ressourcen

Wer ein neues Werkzeug vorschlägt, sollte zeigen, dass sich diese Aufgaben damit sauber lösen lassen. Umgekehrt entdeckt man beim Lösen die Schwächen des Werkzeugs.

Die speisenden Philosophen

Dijkstra stellte die Aufgabe 1965 als Prüfungsfrage. Fünf Philosophen sitzen an einem runden Tisch, zwischen je zweien liegt eine Gabel. Zum Essen braucht jeder beide Gabeln neben sich.



Die naive Lösung und ihr Fehler.

```
while (true) {
  denken();
  nimm(links);  nimm(rechts);
  essen();
  leg_ab(links);  leg_ab(rechts); }
```

Wenn alle fünf gleichzeitig die linke Gabel nehmen, hält jeder eine und wartet auf die andere. Ein Zyklus der Länge fünf, und alle vier Coffman-Bedingungen sind erfüllt.

Vier Lösungen, je eine Bedingung brechend.

Lösung	bricht	Bewertung
Höchstens vier gleichzeitig	Zyklus	einfach, wirksam
Beide Gabeln oder keine	Halten und Warten	braucht eine Sperre für den Tisch
Einer greift andersherum	Zyklus	**elegant**
Bei Misserfolg zurücklegen	Kein Entzug	kann verhungern

Die dritte Lösung im Detail.
Man nummeriert die Gabeln und verlangt, dass jeder zuerst die mit der kleineren Nummer nimmt. Für vier Philosophen ist das die linke, für einen die rechte.

Das ist genau die Sperrordnung aus Kapitel 8, angewandt auf ein anschauliches Beispiel. Der Zyklus wird unmöglich, weil er ein absteigendes Paar enthalten müsste.

Warum die vierte Lösung nicht taugt.
Wer die zweite Gabel nicht bekommt, legt die erste zurück und versucht es später erneut. Das verhindert die Verklemmung.

*Es kann aber geschehen, dass alle immer wieder gleichzeitig zugreifen, gleichzeitig scheitern und gleichzeitig zurücklegen. Niemand ist blockiert, und niemand isst. Man nennt das **Livelock**, im Unterschied zur Verklemmung.*

	Verklemmung	Livelock
Zustand	alle blockiert	**alle aktiv**
Prozessorlast	null	hoch
Fortschritt	keiner	keiner
Erkennung	Wartegraph	schwierig

Ein Livelock ist unangenehmer, weil er von außen wie Arbeit aussieht. Die Abhilfe ist wieder das zufällige Zurückweichen: Wer scheitert, wartet eine zufällige Zeit.

Kommunikation statt gemeinsamem Speicher

TONY HOARE schlug 1978 einen anderen Zugang vor. Statt gemeinsame Daten zu schützen, tauschen Prozesse Nachrichten aus und teilen gar nichts.

	Gemeinsamer Speicher	Nachrichten
Geteilt wird	Daten	**nichts**
Schutz durch	Sperren	Bauart
Fehlerarten	Wettläufe, Verklemmungen	**Verklemmungen**
Verteilbar	nein	ja
Kosten	sehr gering	Kopieren

> *Wettläufe verschwinden, weil es keinen gemeinsamen veränderlichen Zustand gibt. Verklemmungen bleiben möglich, denn zwei Prozesse können aufeinander warten.*

Der Ansatz heißt CSP und hat mehrere Sprachen geprägt: Occam in den 1980er Jahren, später Erlang und Go.

```
// Go:  ein Kanal statt einer Sperre
c := make(chan int)
go func() { c <- rechne() }()
ergebnis := <-c
```

Der Kanal übernimmt beides: den Datentransport und die Synchronisation. Der Empfänger wartet, bis etwas da ist, und muss dafür keine Bedingungsvariable anfassen.

Erlang und das Prinzip „lass es abstürzen“

Erlang entstand ab 1986 bei Ericsson für Telefonvermittlungen, die jahrelang ohne Unterbrechung laufen müssen.

Sehr viele Prozesse	Hunderttausende, jeder wenige hundert Byte
Kein geteilter Speicher	Nachrichten werden kopiert
Überwachungs-bäume	Ein Prozess wacht über andere und startet sie neu
Kein Abfangen	Fehler werden nicht behandelt, sondern führen zum Neustart

Die letzte Zeile widerspricht dem üblichen Vorgehen. Der Gedanke dahinter: Ein Prozess, der in einen unerwarteten Zustand geraten ist, kann nicht wissen, wie er sich retten soll. Ein sauberer Neustart aus einem bekannten Zustand ist zuverlässiger als eine Reparatur.

Weil kein Speicher geteilt wird, kann ein abgestürzter Prozess auch nichts hinterlassen haben, das andere stört. Genau das macht den Ansatz erst möglich.

Das Vermittlungssystem AXD301 erreichte damit eine Verfügbarkeit, die mit neun Neunen angegeben wurde, also etwa dreißig Millisekunden Ausfall im Jahr. Die Zahl ist umstritten; die Größenordnung ist bemerkenswert.

Aufgabe 9.1.

(a) Lösen Sie das Leser-Schreiber-Problem ohne Verhungern.
(b) Behandeln Sie den schlafenden Friseur vollständig.
(c) Vergleichen Sie Kanäle und Sperren an derselben Aufgabe.

Lösung Schritt für Schritt.

(a) Leser und Schreiber ohne Verhungern.
Der Kunstgriff. Ein zusätzlicher Semaphor, an dem sich alle Ankommenden anstellen. Er wird nur kurz gehalten und stellt die Reihenfolge her.

```
semaphore reihe = 1, ressource = 1,
zaehlmutex = 1;
int leser = 0;
```

```
Leser:
wait(reihe);
  wait(zaehlmutex);
    if (++leser == 1) wait(ressource);
  signal(zaehlmutex);
signal(reihe);     sofort wieder frei
...  lesen ...
wait(zaehlmutex);
  if (-leser == 0) signal(ressource);
signal(zaehlmutex);
```

```
Schreiber:
wait(reihe);  wait(ressource);
signal(reihe);
...  schreiben ...
signal(ressource);
```

Warum das Verhungern verschwindet.

> *Ein wartender Schreiber hält* `reihe`*. Jeder neu ankommende Leser bleibt dort stehen, statt sich beim Zähler einzureihen. Die gerade lesenden Leser laufen aus, der Schreiber kommt dran.*

Zeit	Ereignis	leser	Zustand
t_1	L_1 kommt	1	liest
t_2	L_2 kommt	2	lesen
t_3	S kommt	2	wartet, hält `reihe`
t_4	L_3 kommt	2	**steht bei `reihe`**
t_5	L_1, L_2 fertig	0	S schreibt
t_6	S fertig	0	L_3 liest

Niemand wartet länger als eine Runde. Die Lösung ist gerecht in dem Sinn, dass die Ankunftsreihenfolge gewahrt bleibt, und lässt gleichzeitig ankommende Leser trotzdem zusammen eintreten.

(b) Der schlafende Friseur.
Die Aufgabe. Ein Friseur, ein Stuhl, N Warteplätze. Ist niemand da, schläft der Friseur. Kommt ein Kunde und sind alle Plätze belegt, geht er wieder.

```
semaphore kunden = 0;      wartende Kunden
semaphore friseur = 0;     bereiter Friseur
semaphore mutex = 1;
int wartend = 0;
```

```
Friseur:
while (true) {
  wait(kunden);      schlafen, bis einer kommt
  wait(mutex); wartend-;
signal(mutex);
  signal(friseur);
  schneiden(); }
```

```
Kunde:
wait(mutex);
if (wartend < N) {
  wartend++;
  signal(kunden);
  signal(mutex);
  wait(friseur);
  haare_schneiden_lassen(); }
else { signal(mutex); weggehen(); }
```

Die drei Fälle.

Lage	**Friseur**	**Kunde**
Laden leer	schläft bei `wait(kunden)`	weckt ihn
Friseur arbeitet	schneidet	stellt sich an
Warteraum voll	schneidet	**geht weg**

Worauf es bei der Reihenfolge ankommt.

Der Kunde gibt den Mutex frei, bevor *er auf den Friseur wartet. Täte er es nicht, könnte der Friseur den Zähler nicht verändern, und es entstünde die Verklemmung aus Kapitel 7, Aufgabe (b).*

Was die Aufgabe eigentlich lehrt.
Sie ist das Modell für jeden Dienst mit begrenzter Warteschlange: ein Webserver mit einer Höchstzahl offener Verbindungen, ein Fadenvorrat mit begrenztem Auftragspuffer, eine Netzkarte mit Empfangsring.

Warteraum zu klein	Kunden werden abgewiesen, obwohl der Dienst gleich frei wäre
Warteraum zu groß	Kunden warten sehr lange und geben selbst auf; die Arbeit war umsonst

Die zweite Zeile beschreibt ein reales Problem großer Systeme: Ein überlanger Puffer verwandelt einen Lastspitze in eine dauerhafte Verzögerung. Man nennt das ***Bufferbloat****, und die Antwort besteht darin, Puffer bewusst klein zu halten und früh abzuweisen.*

(c) Kanäle gegen Sperren.
Die Aufgabe. Ein Zähler, den mehrere Fäden erhöhen und dessen Stand abgefragt werden kann.
Mit Sperre.

```
mutex m; int wert = 0;

void erhoehen() { sperren(m); wert++;
freigeben(m); }

int lesen() { sperren(m); int v =
wert; freigeben(m); return v; }
```

Mit Kanal.

```
// ein einziger Faden besitzt den Wert
go func() {
  wert := 0
  for {
    select {
    case <-erhoehen:  wert++
    case antwort <- wert:
    }
  }
}()
```

Der Wert gehört genau einem Faden. Niemand sonst kann ihn anfassen, und deshalb kann es keinen Wettlauf geben. Die Synchronisation steckt vollständig in den Kanälen.

Der Vergleich.

	Sperre	**Kanal**
Takte je Erhöhung	≈ 40	≈ 300
Fehleranfälligkeit	hoch	gering
Lesbarkeit	mittel	gut
Skalierung	mit Konflikten schlecht	**ein Faden ist der Engpass**

Kanäle sind hier siebenmal langsamer und deutlich sicherer. Für einen Zähler lohnt sich das nicht, für einen Zustandsautomaten mit vielen Feldern sehr wohl.

Kanäle sind kein Allheilmittel

Die Go-Entwickler haben ihre eigene Empfehlung mehrfach eingeschränkt.

Kanäle für	Weiterreichen von Eigentum, Verteilen von Arbeit, Signalisieren
Sperren für	Schutz einfacher Zustände, Zwischenspeicher, Zähler

> *Die verbreitete Regel „teile Speicher durch Kommunikation“ ist ein Leitsatz, keine Vorschrift. Ein Mutex um einen Zwischenspeicher ist in Go völlig üblich und richtig.*

Auch Verklemmungen bleiben möglich. Zwei Fäden, die jeweils auf den Kanal des anderen warten, sind ebenso festgefahren wie zwei Fäden mit vertauschten Sperren. Die Go-Laufzeitumgebung erkennt immerhin den Fall, dass *alle* Fäden blockiert sind, und bricht mit einer Meldung ab.

Dijkstras Aufsatz[10] enthält die Philosophen; Hoare[15] begründet CSP; Herlihy und Shavit[13] lösen die klassischen Aufgaben mit modernen Mitteln.

Part IV

Ein- und Ausgabe

10 Geräte und Treiber

Wie der Kern ein Gerät anspricht

Eigene Befehle	Getrennter Adressraum für Geräte; auf x86 die Befehle `in` und `out`
Speicherabbildung	Die Register des Geräts erscheinen als Speicheradressen
Direktzugriff	Das Gerät liest und schreibt selbst im Hauptspeicher

Die dritte Zeile ist die wichtigste. Ohne sie müsste der Prozessor jedes Byte selbst bewegen.

Verfahren	**Prozessorlast**	**bei 1 GB/s**
Byte für Byte	sehr hoch	unmöglich
Blockweise durch die CPU	hoch	$\approx 30\,\%$
Direktzugriff	gering	$< 1\,\%$

Beim Direktzugriff teilt der Treiber dem Gerät eine physische Adresse mit, und das Gerät schreibt dorthin, während der Prozessor anderes tut. Am Ende meldet es sich mit einer Unterbrechung.

Direktzugriff umgeht die Seitentabellen

Ein Gerät, das im Hauptspeicher schreiben darf, kann überall schreiben. Es unterliegt nicht dem Schutz aus Kapitel 1.

Fehlerhafter Treiber	Kann durch eine falsche Adresse den Kern zerstören
Böswilliges Gerät	Ein manipuliertes Gerät kann Kernspeicher auslesen
Angriff über die Schnittstelle	Thunderbolt und PCIe erlauben Direktzugriff von außen

Die Antwort darauf ist eine zweite Speicherverwaltungseinheit für Geräte, die IOMMU. Sie übersetzt Geräteadressen genauso, wie die MMU Prozessadressen übersetzt, und begrenzt damit, worauf ein Gerät zugreifen kann.

> *Ohne IOMMU konnte man um 2010 einen gesperrten Rechner über den Firewire-Anschluss auslesen. Es genügte, ein Gerät anzustecken, das nach dem Passwort im Speicher suchte.*

Heute ist die IOMMU auf Arbeitsplatzrechnern meist eingeschaltet, und Schnittstellen mit Direktzugriff verlangen eine Bestätigung durch den Benutzer.

Die Treiberschnittstelle

Der Kern legt fest, welche Funktionen ein Treiber bereitstellen muss. Alles darüber sieht dann für alle Geräte gleich aus.

`open`	Gerät belegen, Zustand aufsetzen
`read`, `write`	Daten übertragen
`ioctl`	Alles, was nicht in das Schema passt
`mmap`	Gerätespeicher in einen Adressraum einblenden
`poll`	Bereitschaft anzeigen
`close`	Freigeben

Das ist die Umsetzung des Grundsatzes „alles ist eine Datei“. Ein Programm kann von einer Datei, einer Tastatur, einer Netzverbindung und einem Messgerät mit demselben Aufruf lesen.

Der Aufruf `ioctl` ist der Auffangbehälter für alles Übrige. Er nimmt eine Nummer und einen Zeiger und tut, was der Treiber will. Das ist praktisch und untergräbt die Einheitlichkeit, denn jedes Gerät hat wieder eigene Nummern.

Gerätetyp	Zugriff	Beispiel
Zeichengeräte	byteweise	Tastatur, Terminal
Blockgeräte	blockweise, adressierbar	Platte, SSD
Netzgeräte	**eigene Schnittstelle**	Ethernet

Netzgeräte fallen aus dem Schema, weil sie nicht in das Dateimodell passen. Sie liefern Pakete statt Bytefolgen und haben kein sinnvolles „Öffnen“.

Warum Treiber die häufigste Fehlerquelle sind

Untersuchungen am Linux-Kern haben wiederholt ergeben, dass Treiber deutlich mehr Fehler je tausend Zeilen enthalten als der übrige Kern.

Bereich	Anteil am Code	Anteil an Fehlern
Treiber	≈ 60 %	≈ 70 %
Dateisysteme	≈ 8 %	≈ 8 %
Netz	≈ 10 %	≈ 8 %
Kern im engeren Sinn	≈ 5 %	≈ 4 %

Wenig Prüfung	Ein Treiber wird oft nur von seinem Autor und nur mit einem Gerät getestet
Nebenläufig	Unterbrechungen können jederzeit eintreffen
Fehlerbehandlung	Der seltene Fehlerpfad wird nie durchlaufen und ist deshalb kaum geprüft
Unzuverlässige Hardware	Geräte verhalten sich nicht immer wie dokumentiert

Die dritte Zeile verdient Beachtung. In vielen Treibern ist mehr Code für die Behandlung von Fehlern zuständig als für den Normalfall, und dieser Code läuft praktisch nie.

Was man dagegen tut.

Treiber im Benutzerbetrieb	FUSE für Dateisysteme, UIO und VFIO für Geräte
Fehlereinstreuung	Der Kern lässt Zuteilungen absichtlich scheitern, damit die Fehlerpfade laufen
Fuzzing	`syzkaller` ruft Systemaufrufe mit zufälligen Argumenten auf
Sicherere Sprache	Seit 2022 lassen sich Linux-Treiber in Rust schreiben

Der Fuzzer `syzkaller` hat seit 2015 mehrere tausend Fehler im Linux-Kern gefunden. Er läuft dauerhaft und meldet jeden Absturz mit einem Programm, das ihn reproduziert.

Die letzte Zeile ist die weitreichendste Änderung. Rust verhindert die häufigsten Fehlerarten in Treibern durch das Typsystem: Zugriffe über das Ende eines Feldes hinaus, Benutzung freigegebenen Speichers, und Wettläufe zwischen Fäden.

Aufgabe 10.1.

(a) Rechnen Sie die Kosten von Ein-/Ausgabe mit und ohne Direktzugriff aus.
(b) Vergleichen Sie blockierende, nichtblockierende und ereignisgesteuerte Ein-/Ausgabe.
(c) Untersuchen Sie Ablaufplanung auf Platten und SSDs.

Lösung Schritt für Schritt.

(a) Direktzugriff.
Der Fall. Übertragung von 1 MB von einer SSD.
Ohne Direktzugriff.

Größe	**Wert**	
Übertragungen zu 4 Byte	262 144	
Takte je Übertragung	≈ 100	Geräteregister lesen
Prozessortakte	$2{,}6 \cdot 10^7$	6,5 ms bei 4 GHz

Mit Direktzugriff.

Schritt	**Takte**	
Puffer vorbereiten	≈ 500	
Auftrag schreiben	≈ 200	
Unterbrechung	≈ 2000	
Prozessortakte	≈ 2700	0,7 µs

$$\text{Faktor } \frac{2{,}6 \cdot 10^7}{2700} \approx 9600.$$

Die Übertragung selbst dauert in beiden Fällen etwa gleich lang, weil das Gerät die Grenze setzt. Der Unterschied besteht darin, ob der Prozessor dabei beschäftigt ist oder nicht.

Der Nebeneffekt auf den Cache.
Das Gerät schreibt in den Hauptspeicher, während der Prozessor womöglich eine ältere Fassung im Cache hat.

Kohärenter Direktzugriff	Das Gerät nimmt am Kohärenzprotokoll teil; die Hardware regelt es
Nichtkohärent	Der Treiber muss die betroffenen Zeilen selbst ungültig machen

Auf x86 ist der Direktzugriff kohärent, bei vielen eingebetteten ARM-Systemen nicht. Ein Treiber, der von der einen auf die andere Plattform portiert wird und die Cachepflege vergisst, liefert sporadisch alte Daten.

(b) Drei Arten der Ein-/Ausgabe.

Art	Verhalten	Programmstil
Blockierend	wartet bis fertig	ein Faden je Verbindung
Nichtblockierend	kehrt sofort zurück	Schleife mit Nachfragen
Ereignisgesteuert	meldet Bereitschaft	**ein Faden für viele**
Asynchron	meldet Fertigstellung	Rückruf oder Warteschlange

Der Vergleich bei 10 000 Verbindungen.

Verfahren	Speicher	Aufwand je Runde
Faden je Verbindung	80 GB reserviert	10^4 Wechsel
`select`	gering	$\mathcal{O}(n)$ je Aufruf
`epoll`	gering	$\mathcal{O}(\text{aktive})$
`io_uring`	gering	fast kein Systemaufruf

Der entscheidende Unterschied zwischen `select` und `epoll` liegt darin, dass `select` bei jedem Aufruf die vollständige Liste aller Deskriptoren übergeben bekommt und durchgeht, während `epoll` die Liste im Kern hält und nur die aktiven zurückgibt.

Verbindungen	davon aktiv	`select`	`epoll`
100	10	100	10
10 000	10	10 000	**10**
100 000	100	nicht möglich	100

Der Aufruf `select` ist außerdem auf 1024 Deskriptoren begrenzt, weil er eine Bitmaske fester Größe benutzt. Das ist ein Entwurf aus dem Jahr 1983 und war damals großzügig bemessen.

Der Unterschied zwischen Bereitschaft und Fertigstellung.

	`epoll`	`io_uring`
Meldet	„du kannst jetzt lesen“	„ich habe gelesen“
Danach	Systemaufruf `read`	**Daten sind schon da**
Aufrufe je Vorgang	2	≈ 0

Bei `io_uring` legt das Programm Aufträge in einen Ringpuffer, den auch der Kern sieht, und holt die Ergebnisse aus einem zweiten. Im günstigsten Fall entfällt der Systemaufruf ganz, weil ein Kernfaden von sich aus nachsieht.

Nach den Spectre-Gegenmaßnahmen von 2018 (Kapitel 1) hat das erheblich an Bedeutung gewonnen, weil Systemaufrufe teurer geworden sind.

(c) Ablaufplanung bei Speichergeräten.
Bei einer Festplatte.
Die Zugriffszeit wird von der Kopfbewegung beherrscht.

Anteil	Zeit	
Positionieren	4–10 ms	**beherrschend**
Umdrehungswarten	2–4 ms	bei 7200/min
Übertragen (4 KB)	0,03 ms	

Es lohnt sich also erheblich, Aufträge nach Spurnummer zu ordnen. Ein Aufzugverfahren, das den Kopf einmal durchlaufen lässt und alles unterwegs erledigt, verdreifacht den Durchsatz.

Verfahren	Regel	Problem
FIFO	Ankunftsreihenfolge	viele Bewegungen
Kürzeste Bewegung	nächstgelegene Spur	Verhungern außen
Aufzug	durchlaufen und umkehren	gut

Bei einer SSD.
Es gibt keine Kopfbewegung. Die Ordnung nach Adressen bringt fast nichts.

Größe	Platte	NVMe-SSD
Zufälliges Lesen (4 KB)	$\approx 150/\mathrm{s}$	$\approx 10^6/\mathrm{s}$
Warteschlangen	1	**65 536**
Aufträge je Schlange	32	65 536

Die mittlere Zeile ist der eigentliche Umbruch. Eine NVMe-SSD hat eine eigene Warteschlange je Prozessorkern. Damit entfällt die zentrale Sperre, die bei Platten den Zugang regelte.

Linux hat die Blockschicht deshalb ab 2013 umgebaut. Das Ergebnis heißt `blk-mq` und arbeitet mit Warteschlangen je Kern statt einer gemeinsamen.

Planer	gedacht für	Bemerkung
`none`	schnelle SSDs	**Standard bei NVMe**
`mq-deadline`	gemischt	Fristen gegen Verhungern
`bfq`	Arbeitsplatz	Gerechtigkeit zwischen Prozessen

Der beste Planer ist manchmal keiner

Bei einer schnellen SSD kostet das Umordnen mehr Zeit, als es einspart. Der Planer `none` reicht die Aufträge einfach durch.

> *Das ist ein wiederkehrendes Muster: Eine Optimierung, die dreißig Jahre lang richtig war, wird durch eine Änderung der Hardware nicht nur überflüssig, sondern schädlich.*

Ein zweites Beispiel derselben Art betrifft das Vorauslesen. Bei Platten lohnt es sich fast immer, den nächsten Block gleich mitzulesen. Bei einer SSD mit zufälligem Zugriffsmuster wird dadurch Bandbreite verschwendet und der Zwischenspeicher verschmutzt.

Deshalb lässt sich das Vorauslesen je Datei einstellen, und Datenbanken schalten es für ihre Datendateien meist ab.

Arpaci-Dusseau und Arpaci-Dusseau[2] behandeln Geräte und Platten ausführlich; Love[18] beschreibt die Linux-Blockschicht; McKusick und Mitautoren[20] zeigen die BSD-Seite.

11 Dateisysteme

Was zu lösen ist

Ein Dateisystem bildet Namen auf Folgen von Blöcken ab. Die Schwierigkeiten liegen in den Randbedingungen.

Größenspanne	Von wenigen Byte bis zu Terabyte, in derselben Struktur
Wachstum	Dateien werden größer und kleiner, ohne dass man es vorher weiß
Verschnitt	Blöcke sind fest, Dateien nicht
Streuung	Blöcke einer Datei sollen benachbart liegen
Absturz	Der Strom kann jederzeit ausfallen (Kap. 12)

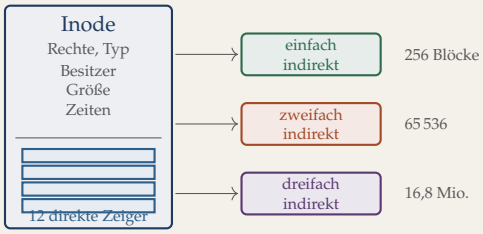
Drei Möglichkeiten, Blöcke zuzuordnen.

Verfahren	Vorteil	Nachteil
Zusammen-hängend	schnell, einfach	Wachstum unmöglich
Verkettet	wächst beliebig	kein wahlfreier Zugriff
Indiziert	beides	Zusatzblöcke nötig

Unix wählte die dritte Möglichkeit und gab ihr eine besondere Form.

Der Inode

Jede Datei hat eine Verwaltungsstruktur, den **Inode**. Er enthält alles über die Datei außer ihrem Namen.



Der Aufbau ist ungleichmäßig, und das mit Absicht. Kleine Dateien brauchen nur die direkten Zeiger und sind mit einem Zugriff erreichbar. Große Dateien kosten mehr, sind aber selten.

Dateigröße	Zugriffe	Anteil der Dateien
$\leq$ 48 KB	1	$\approx$ 85 %
$\leq$ 1 MB	2	$\approx$ 12 %
$\leq$ 256 MB	3	$\approx$ 3 %
darüber	4	$<$ 1 %

Der Name steht nicht im Inode.
Ein Verzeichnis ist eine Datei, die Paare aus Namen und Inode-Nummern enthält. Daraus folgt eine Eigenschaft, die anderswo fehlt.

Dieselbe Datei kann unter mehreren Namen erscheinen, in verschiedenen Verzeichnissen. Der Inode zählt mit, wie viele Namen auf ihn zeigen, und wird erst freigegeben, wenn der Zähler auf null fällt und keine offene Datei mehr darauf verweist.

Löschen	Entfernt den Namen, nicht die Datei. Der Systemaufruf heißt deshalb `unlink`
Offene Datei löschen	Erlaubt; die Datei bleibt zugänglich, bis der letzte Deskriptor geschlossen wird

Die zweite Zeile ist ein verbreiteter Kunstgriff für temporäre Dateien: Man legt sie an, öffnet sie, löscht den Namen sofort, und benutzt sie weiter. Beim Programmende verschwindet sie zuverlässig, auch bei einem Absturz.

Indexsequentielle Dateien

In der kaufmännischen Datenverarbeitung war das Dateimodell ein anderes als in Unix. Eine Datei bestand nicht aus einer Bytefolge, sondern aus *Sätzen* fester Länge, und man wollte sie auf zwei Arten benutzen.

Sequentiell	Alle Sätze der Reihe nach, geordnet nach einem Schlüssel. Für Auswertungen und Listen
Wahlfrei	Ein bestimmter Satz über seinen Schlüssel. Für die Bildschirmmaske

Die Lösung hieß **indexsequentiell**: Zu den Daten kommt eine Indexstruktur, meist ein Baum, der Schlüssel auf Satzadressen abbildet. IBM nannte das Verfahren ISAM, und ähnliche Zugriffsmethoden fanden sich in praktisch allen kaufmännischen Systemen, auch in den Rechnern der mittleren Datentechnik (Kapitel 3).

> *Wer in Business BASIC eine Kundendatei anlegte, bekam damit etwas, das man heute als kleine Datenbank bezeichnen würde: Sätze mit benannten Feldern, ein Schlüssel, ein Index, wahlfreier und sequentieller Zugriff, und Sperren auf Satzebene (Kapitel 6).*

Der Unterschied zur Unix-Sichtweise ist grundsätzlich.

	Unix	Kaufmännisch
Datei ist	Bytefolge	**Satzfolge**
Struktur kennt	das Programm	**das System**
Index	keiner	Teil der Datei
Sperren	ganze Datei	**einzelner Satz**

> *Ritchie und Thompson entschieden sich bewusst für die einfachere Sichtweise. Eine Datei ist eine Folge von Bytes, und alles Weitere ist Sache des Programms. Das machte den Kern klein und erlaubte es, dieselben Werkzeuge auf beliebige Daten anzuwenden.*

Der Preis war, dass jede Anwendung ihre Satzstruktur selbst verwalten musste. Genau diese Lücke füllten später die Datenbanksysteme, und zwar mit denselben Begriffen, die die indexsequentiellen Zugriffsmethoden geprägt hatten: Schlüssel, Index, Satzsperre, Transaktion.

Man kann die Entwicklung deshalb auch so lesen: Was in den kaufmännischen Systemen im Betriebssystem steckte, wanderte bei Unix in eine eigene Schicht darüber.

Eine Spur davon ist geblieben. Die Sperren, die Unix auf Dateiabschnitte anbietet, sind der Rest dieser Denkweise. Der Aufruf `fcntl` mit `F_SETLK` sperrt einen Bytebereich, und wenn dieser Bereich genau einem Satz entspricht, hat man wieder eine Satzsperre.

> *Diese Sperren sind allerdings mit einer unangenehmen Eigenheit behaftet: Sie werden freigegeben, sobald der Prozess irgendeinen Deskriptor auf die Datei schließt, nicht nur den, mit dem er gesperrt hat. Das gilt als einer der schlechtesten Entwürfe in der POSIX-Schnittstelle und wurde später durch* `F_OFD_SETLK` *repariert.*

Dateisysteme im Vergleich

System	Jahr	Verfahren	Besonderheit
FAT	1977	verkettete Tabelle	einfach, verbreitet
ext2	1993	Inodes	kein Journal
ext4	2008	Inodes, Auszüge	Journal
XFS	1994	B-Bäume	sehr große Dateien
ZFS	2005	**Kopie beim Schreiben**	Prüfsummen, Schnappschüsse
Btrfs	2009	ebenso	
F2FS	2012	protokollstrukturiert	für Flashspeicher

Auszüge statt einzelner Blöcke.
Statt jeden Block einzeln zu verzeichnen, merkt man sich Bereiche: Anfangsblock und Länge.

Datei von 1 GB	Verwaltungsdaten	
Blockzeiger (4 KB)	≈ 2 MB	262 144 Einträge
Auszüge	≈ 100 Byte	bei wenigen Bereichen

Der Gewinn entsteht nur, wenn die Datei zusammenhängend liegt. Bei starker Zerstückelung nähert sich der Aufwand wieder dem der Einzelzeiger an.

Kopie beim Schreiben.
ZFS und Btrfs überschreiben nie einen belegten Block. Jede Änderung wird an eine freie Stelle geschrieben, und die Verweise werden anschließend umgehängt.

Vorteil	Der alte Zustand bleibt bis zum Schluss gültig; ein Absturz hinterlässt kein halbes Ergebnis
Vorteil	Schnappschüsse kosten fast nichts; man behält einfach die alten Verweise
Nachteil	Zerstückelung, weil Änderungen überall landen
Nachteil	Der Baum muss bis zur Wurzel neu geschrieben werden

Bei einer Datenbank, die einzelne Sätze in großen Dateien ändert, führt das zu erheblicher Zerstückelung. Deshalb empfehlen Datenbankhersteller für ihre Datendateien meist ein herkömmliches Dateisystem oder das Abschalten dieser Eigenschaft.

Aufgabe 11.1.

(a) Rechnen Sie die maximale Dateigröße bei Inodes aus.
(b) Untersuchen Sie den Verschnitt bei verschiedenen Blockgrößen.
(c) Analysieren Sie, wie ein Pfad aufgelöst wird.

Lösung Schritt für Schritt.

(a) Maximale Dateigröße.
Annahmen: Blockgröße 4 KB, Zeigergröße 4 Byte, also 1024 Zeiger je Block.

Art	Blöcke	Größe
12 direkte	12	48 KB
einfach indirekt	1024	4 MB
zweifach indirekt	$1024^2 = 1{,}05 \cdot 10^6$	4 GB
dreifach indirekt	$1024^3 = 1{,}07 \cdot 10^9$	4 TB
Summe		$\approx$ 4 TB

Der Verwaltungsaufwand für eine große Datei.
Für eine Datei von 4 GB braucht man den zweifach indirekten Zweig:

$$1 + 1024 = 1025 \text{ Indexblöcke} = 4{,}1 \text{ MB}.$$

$$\frac{4{,}1}{4096} = 0{,}1\,\%.$$

Der Platz ist unerheblich. Bedeutsam ist die Zahl der Zugriffe: Um den letzten Block zu erreichen, muss man drei Indexblöcke lesen, und die liegen womöglich weit auseinander.

Warum Auszüge das besser lösen.
Eine zusammenhängende Datei von 4 GB braucht mit Auszügen einen einzigen Eintrag. Der wahlfreie Zugriff auf eine beliebige Stelle kostet dann null zusätzliche Lesevorgänge.

Verfahren	Zugriffe	bei Zerstückelung
Dreifach indirekt	4	4
Auszüge, zusammenhängend	1	—
Auszüge, 1000 Stücke	1–2	Baum nötig

(b) Verschnitt.
Der innere Verschnitt. Der letzte Block einer Datei ist im Mittel halb leer.

Block	Verschnitt je Datei	bei 1 Mio. Dateien	Indexaufwand
512 B	256 B	256 MB	hoch
4 KB	2 KB	2 GB	mittel
16 KB	8 KB	8 GB	gering
64 KB	32 KB	32 GB	sehr gering

Die Verteilung der Dateigrößen.
Sie ist stark schief. Die meisten Dateien sind klein, der meiste Platz steckt in wenigen großen.

Größe	Anteil Dateien	Anteil Platz
< 4 KB	$\approx 55\,\%$	$< 1\,\%$
4 KB bis 1 MB	$\approx 40\,\%$	$\approx 15\,\%$
> 1 MB	$\approx 5\,\%$	$\approx 85\,\%$

Daraus folgt der Kompromiss: Der Block muss klein genug sein, um bei den vielen kleinen Dateien wenig zu verschwenden, und groß genug, um bei den wenigen großen den Index klein zu halten. 4 KB hat sich als guter Wert erwiesen und passt zugleich zur Seitengröße.

Die Lösung für sehr kleine Dateien.
Manche Dateisysteme legen den Inhalt sehr kleiner Dateien unmittelbar im Inode ab, dort wo sonst die Zeiger stehen.

Datei von 60 Byte	Platz	Zugriffe
gewöhnlich	4096 B	2
im Inode	0 zusätzlich	**1**

Der Gewinn liegt weniger im Platz als in der Zugriffszahl. Bei einem Verzeichnis mit vielen winzigen Dateien halbiert sich die Zahl der Lesevorgänge.

(c) Pfadauflösung.
Der Fall. Öffnen von `/home/juergen/text.tex`.

1. Inode des Wurzelverzeichnisses lesen (Nummer 2, bekannt).
2. Datenblöcke der Wurzel lesen, `home` suchen, Inode-Nummer holen.
3. Diesen Inode lesen, Rechte prüfen.
4. Dessen Datenblöcke lesen, `juergen` suchen.
5. Und so fort bis `text.tex`.

Die Kosten ohne Zwischenspeicher.

Bestandteil	**Zugriffe**	
je Pfadbestandteil	2	Inode + Daten
drei Bestandteile	6	
Ziel-Inode	1	
Summe	7	bei SSD $\approx$ 0,7 ms

Sieben Zugriffe für das bloße Öffnen einer Datei. Bei einem Übersetzungslauf, der Tausende von Dateien öffnet, wäre das der beherrschende Aufwand.

Der Verzeichniseintragspuffer.
Der Kern hält deshalb einen Zwischenspeicher, der Pfadbestandteile auf Inodes abbildet.

Fall	**Zugriffe**	**Zeit**
kalt	7	0,7 ms
warm	0	$\approx$ 1 µs

Faktor 700. Deshalb ist der zweite Übersetzungslauf so viel schneller als der erste, auch wenn sich nichts geändert hat.

Die Auflösung ist ein Sicherheitsproblem

Bei jedem Schritt müssen die Rechte geprüft werden, und zwischen Prüfung und Benutzung darf sich nichts ändern.

Zeit	Programm	Angreifer
t_1	prüft: `/tmp/x` gehört mir	
t_2		ersetzt `/tmp/x` durch eine Verknüpfung auf `/etc/passwd`
t_3	öffnet `/tmp/x` und schreibt	

Man nennt das eine **TOCTOU**-Lücke, von *time of check to time of use*. Sie ist ein Wettlauf im Sinne von Kapitel 6, nur zwischen Prozessen und über das Dateisystem.

> *Die Abhilfe besteht darin, nicht zu prüfen und dann zu öffnen, sondern in einem Schritt zu öffnen und dann den Deskriptor zu prüfen. Ein Deskriptor zeigt auf einen Inode und kann nicht umgehängt werden.*

Dafür gibt es eigene Aufrufe: `openat` mit einem Verzeichnisdeskriptor, `fstat` statt `stat`, und Kennzeichen wie `O_NOFOLLOW`, die das Verfolgen von Verknüpfungen verbieten.

Die Regel lautet: Prüfe niemals einen Namen und benutze ihn dann. Öffne und prüfe, was du geöffnet hast.

Arpaci-Dusseau und Arpaci-Dusseau[2] entwickeln Dateisysteme sehr klar; McKusick und Mitautoren[20] beschreiben das BSD-Dateisystem im Detail; Bach[3] zeigt die klassische Unix-Umsetzung.

12 Wenn der Strom ausfällt

Das Problem

Eine einzelne Änderung an einer Datei besteht auf der Platte aus mehreren Schreibvorgängen, die nicht gleichzeitig geschehen können.

1. Einen freien Block aus der Freiliste nehmen.
2. Den Blockzeiger im Inode eintragen.
3. Die Größe im Inode anpassen.
4. Die Daten in den Block schreiben.

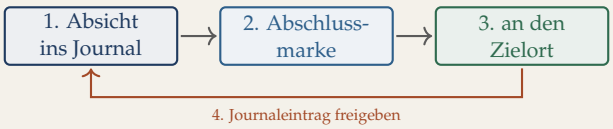
Fällt der Strom zwischen Schritt 2 und Schritt 1 aus, so ist der Block in zwei Dateien eingetragen. Fällt er zwischen 2 und 4 aus, so enthält die Datei alten Müll. Beides ist schlimmer als ein Datenverlust, weil es unbemerkt bleibt.

Ansatz	Vorgehen	Kosten
Nachträgliche Prüfung	`fsck` durchsucht alles	Stunden bei großen Platten
Journal	vorher aufschreiben, was kommt	doppeltes Schreiben
Kopie beim Schreiben	nie überschreiben	Zerstückelung
Weiches Aktualisieren	Reihenfolge sorgfältig wählen	sehr schwer korrekt

Die erste Zeile war bis in die 1990er Jahre üblich. Bei einer Platte von 10 GB dauerte die Prüfung Minuten, bei 10 TB dauert sie Stunden. Das ist der Grund, warum sich Journale durchgesetzt haben.

Das Journal

Der Gedanke ist aus den Datenbanken übernommen: Man schreibt zuerst auf, was man vorhat, und führt es dann aus.



Absturz vor 2	Der Eintrag ist unvollständig und wird verworfen. Die Datei bleibt im alten Zustand
Absturz nach 2	Der Eintrag ist vollständig und wird beim Hochfahren wiederholt

Die Abschlussmarke ist der Kern des Verfahrens. Sie ist ein einzelner Schreibvorgang, der entweder geschieht oder nicht, und sie entscheidet, ob der ganze Eintrag gilt.

Was ins Journal kommt.

Betriebsart	Journal enthält	Bewertung
`journal`	Metadaten **und** Daten	sicher, langsam
`ordered`	nur Metadaten; Daten vorher	**Standard**
`writeback`	nur Metadaten	schnell, Datenmüll möglich

Bei `writeback` kann eine Datei nach einem Absturz die richtige Größe haben und alten Inhalt enthalten. Das Dateisystem ist konsistent, die Datei ist es nicht. Die Zwischenlösung `ordered` schreibt deshalb die Daten immer vor den zugehörigen Metadaten.

Was `fsync` bedeutet

Ein erfolgreicher `write` sagt nur, dass der Kern die Daten übernommen hat. Auf der Platte sind sie damit nicht.

`write`	Daten liegen im Seitenpuffer des Kerns
Rückschreiben	Der Kern schreibt sie später, meist nach einigen Sekunden
`fsync`	Erzwingt das Schreiben und wartet darauf

Vorgang	**Dauer**	
`write` (4 KB)	$\approx$ 2 µs	nur Kopieren
`fsync`, SSD	$\approx$ 200 µs	
`fsync`, Platte	$\approx$ 10 ms	**Faktor 5000**

Deshalb rufen Programme `fsync` sparsam auf, und deshalb verlieren sie bei einem Stromausfall Daten, die sie geschrieben zu haben glaubten. Wer eine Zusage über Dauerhaftigkeit braucht, muss sie ausdrücklich einholen.

Auch `fsync` ist nicht das Ende

Zwischen dem Kern und der Magnetscheibe liegt noch der Zwischenspeicher des Laufwerks.

Schreibpuffer	Das Laufwerk meldet Erfolg, sobald die Daten in seinem eigenen Speicher sind
Sperrbefehl	`fsync` sendet zusätzlich einen Befehl, der das Leeren erzwingt
Unehrliche Geräte	Manche billigen Laufwerke ignorieren diesen Befehl

Die dritte Zeile war in den 2000er Jahren ein reales Problem und hat Datenbankherstellern viel Ärger bereitet. Ein Laufwerk, das den Sperrbefehl bestätigt, ohne ihn auszuführen, ist in Messreihen schneller und im Ernstfall unbrauchbar.

Server-Laufwerke haben deshalb einen Kondensator, der bei Stromausfall genug Energie liefert, um den Puffer zu leeren. Man nennt das **Power Loss Protection**, und es ist der wesentliche Unterschied zwischen einer Server-SSD und einer für den Arbeitsplatz.

Wer es genau wissen will, prüft es mit einem Gerät, das den Strom zu zufälligen Zeitpunkten abschaltet. Das Werkzeug `diskchecker.pl` tut genau das und hat schon manche Überraschung zutage gefördert.

Aufgabe 12.1.

(a) Zeigen Sie, wie man eine Datei sicher ersetzt.
(b) Rechnen Sie die Kosten des Journals aus.
(c) Untersuchen Sie optimistische Nebenläufigkeit.

Lösung Schritt für Schritt.

(a) Eine Datei sicher ersetzen.
Der falsche Weg.

```
f = fopen("daten.txt", "w");     Datei wird geleert
fwrite(...); fclose(f);
```

Ein Absturz zwischen Öffnen und Schließen hinterlässt eine leere oder halbe Datei. Der alte Inhalt ist unwiederbringlich weg, obwohl er vollständig war.

Der richtige Weg.

1. In eine neue Datei `daten.txt.neu` schreiben.
2. `fsync` auf diese Datei.
3. `rename("daten.txt.neu", "daten.txt")`.
4. `fsync` auf das **Verzeichnis**.

Warum jeder Schritt nötig ist.

Schritt 2	Ohne ihn kann die Umbenennung dauerhaft sein, während der Inhalt es nicht ist. Man hätte eine leere Datei unter dem richtigen Namen
Schritt 3	`rename` ist auf POSIX-Dateisystemen **unteilbar**. Es gibt keinen Zeitpunkt, an dem kein `daten.txt` existiert
Schritt 4	Der Verzeichniseintrag ist selbst eine Änderung, die im Puffer stehen kann

Schritt 4 wird fast immer vergessen. Die Umbenennung landet dann im Journal, aber die Zusage darüber wurde nie erbeten, und bei einem Stromausfall kann sie verlorengehen.

Die Unteilbarkeit von `rename` ist die Grundlage.

Zeitpunkt	**Leser sieht**	
vorher	alten Inhalt	vollständig
währenddessen	**alten oder neuen**	nie etwas dazwischen
danach	neuen Inhalt	vollständig

*Dieses Muster, unter dem Namen **schreiben und umbenennen** bekannt, ist die Standardlösung für jede Aktualisierung von Konfigurationsdateien und Zustandsdateien.*

Der Streit von 2009

Als ext4 breit eingesetzt wurde, verloren viele Programme nach einem Absturz ihre Einstellungen. Sie hatten das Muster benutzt, aber die `fsync`-Aufrufe weggelassen.

> *Unter ext3 hatte das trotzdem funktioniert, weil dieses Dateisystem alle fünf Sekunden alles zurückschrieb. Ext4 verzögerte die Zuteilung länger, um bessere Entscheidungen zu treffen, und legte damit die fehlenden Aufrufe offen.*

Die Diskussion war heftig. Die Anwendungsentwickler hielten das Verhalten für einen Fehler, die Dateisystementwickler hielten die Programme für falsch.

Formal	Die Dateisystementwickler hatten recht; POSIX verspricht nichts ohne `fsync`
Praktisch	Tausende von Programmen verließen sich auf das alte Verhalten

Die Auflösung war ein Kompromiss: ext4 erkennt das Muster „neue Datei, dann umbenennen“ und schreibt die Daten in diesem Fall vorher zurück, auch ohne ausdrückliche Aufforderung.

Das ist bemerkenswert, weil hier ein Dateisystem eine Zusage macht, die die Schnittstelle nicht kennt, weil zu viele Programme sie stillschweigend voraussetzen.

(b) Die Kosten des Journals.
Der Aufwand. Jeder Metadatenblock wird zweimal geschrieben.

Betriebsart	Schreibmenge	Durchsatz
kein Journal	1,00	100 %
`writeback`	1,05	≈ 95 %
`ordered`	1,08	≈ 92 %
`journal`	**2,00**	≈ 55 %

Der Aufschlag ist bei den ersten drei Zeilen gering, weil Metadaten nur einen kleinen Teil der Schreibmenge ausmachen. Erst wenn auch die Daten ins Journal gehen, verdoppelt sich alles.

Warum das Journal trotzdem schneller sein kann.
Die Einträge liegen hintereinander an einer Stelle der Platte. Bei einer Festplatte ist das sequentielles Schreiben.

Zugriffsart	**Durchsatz**	
zufällig, 4 KB	0,6 MB/s	Platte
sequentiell	150 MB/s	

Viele kleine verstreute Änderungen werden im Journal zu einem einzigen langen Schreibvorgang gebündelt. Bei einer Last mit vielen kleinen Schreibvorgängen kann das Journal die Sache beschleunigen, obwohl es doppelt schreibt.

Die Wiederherstellungszeit.

Verfahren	**10 TB**	
`fsck`	2–10 Stunden	alles durchsuchen
Journal abspielen	$\approx$ 1 Sekunde	**nur den Ring**

Das ist der eigentliche Grund für Journale. Bei einem Server ist der Unterschied zwischen einer Sekunde und fünf Stunden Ausfallzeit entscheidend.

(c) Optimistische Nebenläufigkeit.
Der Gedanke. Statt zu sperren, prüft man beim Zurückschreiben, ob sich inzwischen etwas geändert hat.

1. Beim Lesen wird eine Versionsnummer oder ein Zeitstempel mitgelesen.
2. Die Bearbeitung geschieht ohne jede Sperre.
3. Beim Schreiben wird geprüft, ob die Version noch dieselbe ist.
4. Wenn nicht, wird der Vorgang abgewiesen und muss wiederholt werden.

```
UPDATE kunde SET anschrift = ?,
version = version + 1
WHERE nr = ?  AND version = ?;
- betroffene Zeilen = 0 bedeutet:
jemand war schneller
```

Der Vergleich mit der Satzsperre aus Kapitel 6.

	Pessimistisch	**Optimistisch**
Sperre	über die ganze Bearbeitung	**keine**
Konflikt	wird verhindert	wird **erkannt**
Bemerkt es	der Zweite beim Öffnen	der Zweite beim Speichern
Verlorene Arbeit	keine	die Eingabe
Blockade	möglich, dauerhaft	keine

Der Unterschied liegt darin, wann der Benutzer die schlechte Nachricht bekommt. Bei der Satzsperre gleich beim Öffnen, bei der optimistischen Variante nach getaner Arbeit.

Wann welches Verfahren passt.

Pessimistisch	Wenige Bearbeiter, hohe Konfliktwahrscheinlichkeit, lange Eingaben
Optimistisch	Viele Bearbeiter, seltene Konflikte, Weboberflächen ohne Sitzungszustand

Bei einer Webanwendung gibt es kein zuverlässiges „Schließen“. Wer eine Sperre setzt und den Browser zuklappt, würde den Satz für immer blockieren. Deshalb ist dort das optimistische Verfahren praktisch alternativlos.

Die Rechnung, wann es sich lohnt.
Sei p die Wahrscheinlichkeit eines Konflikts und A der Aufwand einer Bearbeitung.

p	**optimistisch**	**pessimistisch**	**besser**
0,001	1,001 A	A + Sperre	optimistisch
0,05	1,05 A	A + Sperre	optimistisch
0,5	2,0 A	A + Warten	pessimistisch

Die Faustregel lautet: Unter etwa zehn Prozent Konfliktrate ist das optimistische Verfahren besser, darüber das pessimistische. Genau dieselbe Abwägung steckt hinter den sperrfreien Datenstrukturen aus Kapitel 7.

Arpaci-Dusseau und Arpaci-Dusseau[2] behandeln Journale und Konsistenz sehr klar; Rosenblum und Ousterhout[25] führten das protokollstrukturierte Dateisystem ein; McKusick und Mitautoren[20] beschreiben das weiche Aktualisieren.

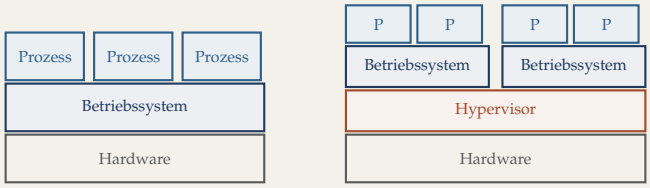
Part V

Größere Zusammenhänge

13 Virtualisierung und Container

Dieselbe Idee, eine Ebene höher

Ein Betriebssystem spiegelt jedem Prozess einen eigenen Rechner vor. Ein Hypervisor tut dasselbe mit ganzen Betriebssystemen.



gewöhnlich virtualisiert

Warum das lange nicht ging.
GERALD POPEK und ROBERT GOLDBERG formulierten 1974 die Bedingungen, unter denen sich ein Rechner virtualisieren lässt. Die wichtigste lautet: Jeder Befehl, der sich im Benutzerbetrieb anders verhält als im Kernbetrieb, muss eine Ausnahme auslösen.

> *Nur dann kann der Hypervisor eingreifen und das gewünschte Verhalten vortäuschen. Die x86-Architektur erfüllte diese Bedingung nicht: Sie hatte siebzehn Befehle, die im Benutzerbetrieb stillschweigend etwas anderes taten.*

Verfahren	Jahr	Vorgehen
Binärübersetzung	1999	VMware ersetzt die kritischen Befehle beim Laden
Paravirtualisierung	2003	Xen ändert das Gastsystem
Hardwarehilfe	**2005**	Intel VT-x, AMD-V: ein neuer Betriebsmodus

> *Die dritte Zeile löste das Problem, indem sie eine Ebene unterhalb des Kernbetriebs einzog. Das Gastbetriebssystem läuft in seinem gewohnten Modus und weiß nicht, dass darunter noch jemand sitzt.*

Was die Virtualisierung kostet

Vorgang	Aufschlag	Bemerkung
Rechnen im Gast	≈ 0 %	läuft unmittelbar
Systemaufruf im Gast	≈ 0 %	bleibt im Gast
Speicherzugriff	2–5 %	zweistufige Übersetzung
Ein-/Ausgabe	10–50 %	je nach Verfahren
Gerätezugriff durchgereicht	≈ 2 %	mit IOMMU

Die zweistufige Adressübersetzung.
Der Gast führt eigene Seitentabellen, und der Hypervisor führt eine zweite Ebene darüber.

Ebene	übersetzt	
Gast-Seitentabelle	virtuell → Gast-physisch	vom Gastkern
Zweite Ebene	Gast-physisch → echt	vom Hypervisor

Bei einem TLB-Fehlgriff müssen beide Bäume durchlaufen werden. Bei vier Ebenen je Baum sind das im ungünstigsten Fall 24 Speicherzugriffe statt vier.

Das ist der Grund, warum große Seiten (Band 4, Kapitel 9) in virtualisierten Umgebungen noch mehr bringen als sonst.
Die Ein-/Ausgabe ist das eigentliche Problem.

Verfahren	Aufschlag	Vorgehen
Emulation	sehr hoch	jedes Geräteregister abgefangen
Paravirtualisiert	mittel	Gast kennt einen einfachen Treiber (virtio)
Durchgereicht	gering	Gast spricht das Gerät unmittelbar an
SR-IOV	gering	Ein Gerät stellt sich als mehrere dar

Die letzte Zeile ist der heutige Standard bei Netzkarten im Rechenzentrum. Die Karte bringt selbst mehrere virtuelle Ausführungen mit, von denen jede einem Gast unmittelbar zugewiesen werden kann.

Container

Ein Container ist keine virtuelle Maschine. Es gibt nur einen Kern, und die Trennung geschieht innerhalb des Betriebssystems.

	Virtuelle Maschine	Container
Kerne	einer je Gast	**einer für alle**
Startzeit	10–60 s	≈ 0,1 s
Speicheraufschlag	200 MB +	wenige MB
Isolation	stark	schwächer
Fremdes Betriebssystem	möglich	nein

Die beiden Bausteine unter Linux.

Namensräume	Was ein Prozess *sieht*: Prozesse, Dateisystem, Netz, Benutzer, Rechnername
Kontrollgruppen	Was ein Prozess *verbrauchen darf*: Prozessorzeit, Speicher, Ein-/Ausgabe

Beide Bausteine gab es einzeln schon Jahre vorher. Docker erfand 2013 nichts Neues, sondern verpackte Vorhandenes so, dass es benutzbar wurde, und fügte ein Format für Abbilder hinzu.

Jahr	Technik	Bemerkung
1979	`chroot`	Wurzelverzeichnis verlegen
2000	FreeBSD Jails	vollständigere Trennung
2002	Namensräume	in Linux, schrittweise
2007	Kontrollgruppen	von Google beigesteuert
2013	**Docker**	**Werkzeug und Format**
2014	Kubernetes	Verwaltung vieler Container

Schichtweise Abbilder

Ein Containerabbild besteht aus übereinandergelegten Schichten, von denen jede nur die Unterschiede zur vorigen enthält.

Schicht	Größe	geteilt mit
Grundsystem	80 MB	allen Abbildern
Laufzeitumgebung	120 MB	vielen
Bibliotheken	40 MB	einigen
Anwendung	5 MB	keinem

Zwanzig Container mit derselben Grundlage belegen nicht zwanzigmal 245 MB, sondern einmal 240 plus zwanzigmal 5. Der Kern blendet die gemeinsamen Schichten nur einmal ein.

Technisch geschieht das mit einem **Überlagerungsdateisystem**. Es zeigt die Vereinigung mehrerer Verzeichnisbäume; Schreibvorgänge landen in der obersten, veränderbaren Schicht.

Wird eine Datei aus einer unteren Schicht geändert, so wird sie zunächst nach oben kopiert. Das ist dasselbe Verfahren wie beim Kopieren bei Bedarf aus Kapitel 2, nur auf Dateiebene.

Aufgabe 13.1.

(a) Vergleichen Sie Container und virtuelle Maschinen quantitativ.
(b) Untersuchen Sie, wie sicher die Trennung durch Container ist.
(c) Analysieren Sie, was bei Überbuchung geschieht.

Lösung Schritt für Schritt.

(a) Der quantitative Vergleich.
Der Fall. Hundert kleine Dienste auf einem Server mit 64 GB Speicher.

Größe	VMs	Container
Speicher je Einheit	512 MB	50 MB
davon Betriebssystem	≈ 300 MB	≈ 0
Gesamtspeicher	51 GB	**5 GB**
Startzeit	30 s	0,1 s
Dichte je Server	≈ 100	≈ 1000

Der Faktor zehn bei der Dichte ist der wirtschaftliche Grund für die Verbreitung von Containern. Bei hundert Diensten spart man neunzig Betriebssysteme, die alle dasselbe tun.

Die Startzeit ist der zweite Grund.

Bei 30 Sekunden	Man hält Reserven bereit, weil Nachschub zu lange dauert
Bei 0,1 Sekunden	Man startet nach Bedarf und hält keine Reserve

Das verändert die Betriebsweise grundlegend. Erst damit wurde es sinnvoll, Kapazität nach der aktuellen Last zu regeln, und darauf beruht die Arbeitsweise von Kubernetes.

(b) Wie sicher ist die Trennung?
Die Angriffsfläche.

	VM	Container
Schnittstelle zum Wirt	≈ 30 Befehle	**über 350 Systemaufrufe**
Ausbruch bedeutet	Hypervisor überwinden	**Kernfehler ausnutzen**
Gemeldete Ausbrüche	wenige	regelmäßig

Die erste Zeile ist der Kern der Sache. Ein Container hat den vollen Systemaufrufsatz des Kerns vor sich. Jeder Fehler in einem dieser Aufrufe ist ein möglicher Weg nach draußen.

Die Gegenmaßnahmen.

seccomp	Erlaubt nur eine Auswahl von Systemaufrufen; Dockers Voreinstellung sperrt etwa 60
Benutzer-namensraum	Der Wurzelbenutzer im Container ist außen ein gewöhnlicher
Fähigkeiten ablegen	Rechte einzeln entziehen statt alles oder nichts
Zwangs-zugriffs-kontrolle	SELinux oder AppArmor als zusätzliche Schicht

Die zweite Zeile ist die wirksamste und wird am seltensten benutzt, weil sie manche Anwendungen stört. Ohne sie ist der Wurzelbenutzer im Container derselbe wie außen, und ein Ausbruch bedeutet sofort volle Rechte.

Die Zwischenlösungen.

Ansatz	Vorgehen	Aufschlag
gVisor	Kern im Benutzerbetrieb fängt Aufrufe ab	10–50 %
Kata Containers	sehr kleine VM je Container	≈ 10 %
Firecracker	Hypervisor mit minimaler Ausstattung	Start in 125 ms

Firecracker wurde von Amazon für den Dienst Lambda entwickelt. Es lässt fast alles weg, was ein Hypervisor sonst kann, und erreicht damit Startzeiten in der Größenordnung von Containern bei der Isolation einer virtuellen Maschine.

Die Faustregel für die Praxis

Eigener Code	Container genügen; die Trennung dient der Ordnung, nicht der Abwehr
Fremder Code	Virtuelle Maschine oder eine der Zwischenlösungen

Wer Container als Sicherheitsgrenze gegen böswilligen Code benutzt, verlässt sich darauf, dass in über dreihundert Systemaufrufen kein ausnutzbarer Fehler steckt. Diese Annahme hat sich mehrfach als falsch erwiesen.

(c) Überbuchung.
Der Fall. Ein Server mit 64 GB, darauf Container mit zusammen 100 GB Zusage.

Anforderung	Was ein Container garantiert bekommt; der Planer verteilt danach
Grenze	Was er höchstens nehmen darf; darüber greift der Kern ein

	Prozessor	**Speicher**
Über der Grenze	gedrosselt	**beendet**
Folge	langsamer	**Absturz**

Der Unterschied ist grundlegend. Prozessorzeit ist teilbar und nachholbar, Speicher nicht. Wer zu viel Speicher will, bekommt keine Verzögerung, sondern das Ende.

Die Drosselung hat eine unangenehme Eigenschaft.
Die Prozessorgrenze wird in Zeitfenstern von meist 100 ms durchgesetzt. Ein Container, der sein Kontingent in 20 ms verbraucht, steht die restlichen 80 ms still.

Grenze	Wirkung	Antwortzeit
1,0 Kerne	voll ausgelastet	gleichmäßig
0,2 Kerne	20 ms von 100	Ausreißer bis 80 ms

Ein Dienst mit einer mittleren Antwortzeit von 5 ms kann durch Drosselung einzelne Antworten mit 85 ms erzeugen. In Messreihen fällt das nur bei den hohen Perzentilen auf.

Die verbreitete Empfehlung lautet deshalb, für antwortzeitkritische Dienste eine Anforderung zu setzen und auf die Grenze zu verzichten. Der Dienst bekommt dann garantiert seinen Anteil und darf darüber hinaus freie Kapazität nutzen.

Das setzt allerdings voraus, dass man den Nachbarn traut. In einer gemeinsam genutzten Umgebung ist die Grenze unverzichtbar, und die Ausreißer sind der Preis dafür.

Barham und Mitautoren[4] beschreiben Xen und die Paravirtualisierung; Merkel[21] stellt Docker vor; Smith und Nair sowie Anderson und Dahlin[1] ordnen die Virtualisierung ein.

14 Schutz und Isolation

Acht Grundsätze von 1975

JEROME SALTZER und MICHAEL SCHROEDER stellten 1975 Grundsätze für den Schutz von Information auf. Sie sind fünfzig Jahre alt und werden bis heute zitiert, weil sie nicht an eine Technik gebunden sind.

Sparsamkeit	Der Entwurf soll so einfach sein, dass man ihn prüfen kann
Voreinstellung sperrt	Erlaubt ist, was ausdrücklich erlaubt wurde
Vollständige Prüfung	Jeder Zugriff wird geprüft, nicht nur der erste
Offener Entwurf	Die Sicherheit darf nicht an der Geheimhaltung des Verfahrens hängen
Trennung der Rechte	Zwei Schlüssel sind besser als einer
Geringstes Recht	Jeder bekommt nur, was er braucht
Wenig Gemeinsames	Geteilte Betriebsmittel sind Wege für Angriffe
Annehmbarkeit	Was zu umständlich ist, wird umgangen

Der letzte Punkt wird am häufigsten verletzt. Eine Sicherheitsvorkehrung, die die Arbeit behindert, wird abgeschaltet, und dann ist sie schlechter als keine, weil man sich auf sie verlässt.

Der siebte Punkt ist derjenige, den Spectre und Meltdown (Band 4, Kapitel 14) auf eine Weise bestätigt haben, die 1975 niemand vorhersehen konnte. Caches und Sprungvorhersager sind geteilte Betriebsmittel, und genau daraus entstanden die Kanäle.

Wie Unix Rechte verwaltet

Wer	Rechte	Bits
Besitzer	lesen, schreiben, ausführen	3
Gruppe	dasselbe	3
Andere	dasselbe	3

Das ist bemerkenswert sparsam und für viele Fälle zu grob. Man kann nicht ausdrücken, dass genau zwei bestimmte Benutzer Zugriff haben sollen, ohne eine Gruppe anzulegen.

Zugriffslisten	Feinere Rechte je Benutzer; POSIX-ACLs, seit den 1990er Jahren
Fähigkeiten	Die Rechte des Wurzelbenutzers in etwa vierzig einzelne aufgeteilt
Zwangskontrolle	SELinux, AppArmor: Regeln, die auch der Besitzer nicht umgehen kann

Das Problem mit dem Wurzelbenutzer.
In der ursprünglichen Fassung darf er alles. Ein Programm, das nur eine einzige besondere Berechtigung braucht, bekommt sie alle.

Programm	braucht	bekam früher
`ping`	Rohe Netzpakete	**alles**
`passwd`	Schreiben in eine Datei	alles
Webserver auf Port 80	Port unter 1024	alles

Ein Fehler in einem dieser Programme bedeutete die vollständige Übernahme des Systems. Genau das verletzt den Grundsatz des geringsten Rechts, und die Fähigkeiten sind die Antwort darauf.

`CAP_NET_RAW`	rohe Netzpakete, für `ping`
`CAP_NET_BIND_SERVICE`	Ports unter 1024
`CAP_SYS_ADMIN`	ein Sammelbecken, praktisch alles

Die letzte Zeile ist eine bekannte Schwäche. `CAP_SYS_ADMIN` umfasst so viele verschiedene Berechtigungen, dass es faktisch dem Wurzelbenutzer entspricht. Es gilt als der neue Sammelposten für alles, was man nicht sauber einordnen wollte.

Was heute die Grenze bildet

Die klassische Vorstellung war, dass Benutzer voreinander geschützt werden. Auf einem Arbeitsplatzrechner gibt es aber nur einen Benutzer, und die Bedrohung kommt von den Programmen, die er ausführt.

	klassisch	**heute**
Grenze zwischen	Benutzern	**Anwendungen**
Bedroht ist	andere Benutzer	**die eigenen Daten**
Werkzeug	Rechte an Dateien	Sandkästen, Rechteabfragen

Ein Textverarbeitungsprogramm läuft mit allen Rechten des Benutzers und kann daher sämtliche seiner Dateien lesen, verändern und versenden. Das Rechtesystem hilft dagegen nicht, weil es alles erlaubt, was der Benutzer darf.

System	Ansatz	Bemerkung
Mobilbetriebs-systeme	je Anwendung eine Be-nutzerkennung	von Anfang an so gebaut
macOS	Sandkasten, Zu-griffsabfragen	seit 2012
Windows	AppContainer	
Linux	Flatpak, Snap	nachträglich, lückenhaft

Die letzte Zeile ist am schwierigsten, weil die vorhandenen Anwendungen davon ausgehen, überall zugreifen zu dürfen. Ein Sandkasten, der sie einsperrt, bricht sie.

Der Ausweg: Rechte über Dialoge vergeben.
Statt vorab festzulegen, welche Dateien ein Programm sehen darf, öffnet man einen Dialog des Systems und übergibt nur die gewählte Datei.

Alt	Das Programm zeigt einen eigenen Dialog und liest die Datei selbst
Neu	Das System zeigt den Dialog und reicht einen Deskriptor an das Programm weiter

Das Programm bekommt Zugriff auf genau eine Datei und erfährt nichts über den Rest. Der Benutzer merkt keinen Unterschied, weil der Dialog aussieht wie immer.

*Man nennt das ein **Portal**. Es ist ein gutes Beispiel für Saltzers achten Grundsatz: Die Sicherheitsvorkehrung ist wirksam, weil sie niemanden stört.*

Aufgabe 14.1.

(a) Untersuchen Sie Programme mit erhöhten Rechten und ihre Gefahren.
(b) Analysieren Sie, wie ein Sandkasten aufgebaut wird.
(c) Diskutieren Sie, was Isolation seit 2018 kostet.

Lösung Schritt für Schritt.

(a) Erhöhte Rechte.
Der Mechanismus. Eine Datei mit gesetztem `setuid`-Bit läuft mit den Rechten ihres Besitzers, nicht des Aufrufenden.

Kennung	Bedeutung	
reale	wer aufgerufen hat	bleibt
effektive	**wonach geprüft wird**	wird zum Besitzer
gespeicherte	zum Zurück-wechseln	

Warum das gefährlich ist.
Das Programm läuft mit fremden Rechten und wird vollständig vom Aufrufenden gesteuert.

Argumente	Beliebig lang und beliebiger Inhalt
Umgebung	Vollständig unter Kontrolle des Angreifers
Deskriptoren	Können umgebogen sein; auch 0, 1 und 2
Signale	Können zu ungünstigen Zeitpunkten kommen
Grenzen	Können so gesetzt sein, dass Aufrufe scheitern

Die dritte Zeile hat historisch mehrfach zu Übernahmen geführt. Ein Programm schließt Deskriptor 1, öffnet eine Datei und schreibt eine Fehlermeldung dorthin, die es für den Bildschirm hält.

Der Rückzug aus diesem Mechanismus.

Fähigkeiten	Statt aller Rechte nur die benötigte; `ping` braucht seit 2011 kein `setuid` mehr
Hilfsdienste	Ein Dienst mit Rechten nimmt Aufträge über einen Socket entgegen
Rechte früh ablegen	Was man nicht mehr braucht, gibt man zurück

Die zweite Zeile hat den Vorteil, dass der Dienst eine sehr schmale Schnittstelle hat und nicht vom Aufrufenden gesteuert wird. Er entscheidet selbst, was er zulässt.

(b) Ein Sandkasten.
Die Bausteine.

1. Eigener Namensraum für Dateien: Nur wenige Verzeichnisse sind sichtbar.
2. Eigener Netznamensraum: Kein Netzzugang, oder nur über einen Vermittler.
3. Seccomp: Nur eine Auswahl von Systemaufrufen ist erlaubt.
4. Alle Fähigkeiten abgelegt.
5. `no_new_privs`: Auch ein `setuid`-Programm kann keine Rechte mehr gewinnen.
6. Speicher- und Prozessorgrenzen über Kontrollgruppen.

Wie man den Aufrufsatz bestimmt.
Man lässt das Programm unter Beobachtung laufen und zeichnet auf, welche Aufrufe es benutzt.

Programm	**benutzte Aufrufe**	**von etwa 350**
`cat`	≈ 15	4 %
Bildbetrachter	≈ 60	17 %
Webbrowser	≈ 150	43 %

Die Angriffsfläche schrumpft entsprechend. Wenn ein Programm nur fünfzehn Aufrufe braucht, sind die Fehler in den übrigen dreihundertfünfunddreißig für einen Angreifer unerreichbar.

Die Falle bei der Aufzeichnung.
Ein Programm benutzt in seltenen Fällen Aufrufe, die im Testlauf nicht vorkommen. Wird es später gesperrt, stürzt es ab, und zwar beim Anwender.

Beim Einführen	Verletzungen nur melden, nicht sperren
Nach Beobachtung	Auf Sperren umstellen
Bei Änderungen	Die Liste neu bestimmen; eine neue Bibliotheksfassung kann andere Aufrufe benutzen

Die dritte Zeile ist der Grund, warum solche Listen laufend gepflegt werden müssen. Eine Aktualisierung der C-Bibliothek, die von `open` auf `openat` umstellt, bricht jeden Sandkasten, der nur `open` erlaubt.

(c) Was Isolation kostet.
Die Bilanz seit 2018.

Maßnahme	**Kosten**	**gegen**
KPTI	5–30 %	Meltdown
Retpoline	5–20 %	Spectre v2
Puffer leeren beim Wechsel	2–10 %	MDS
Mehrfädigkeit abschalten	bis 30 %	geteilte Kerne
zusammen	bis **40** %	

Bei Lasten mit vielen Systemaufrufen, also Datenbanken und Netzdiensten, liegt der Verlust am oberen Ende. Bei reiner Rechenarbeit ist er kaum messbar.

Wann man abwägen darf.

Mehrere Mieter	Alle Gegenmaßnahmen nötig; ein Kunde darf keinen anderen auslesen
Ein Mieter, kein fremder Code	Manche Maßnahmen können abgeschaltet werden
Rechenknoten im Cluster	Oft abgeschaltet, weil nur eigener Code läuft

Diese Abwägung setzt voraus, dass man genau weiß, welcher Code auf der Maschine läuft. Sobald irgendwo ein Browser oder ein Interpreter fremden Code ausführt, gilt sie nicht mehr.

Der Umbruch der letzten Jahre

Bis 2018 galt der Prozessor als vertrauenswürdige Grundlage, auf der das Betriebssystem seine Grenzen zog. Diese Annahme ist gefallen.

	vorher	**heute**
Grenze durchsetzen	Seitentabellen	**plus Zeitverhalten**
Vertrauensbasis	Prozessor + Kern	Prozessor mit Vorbehalt
Neue Lücken	selten	**regelmäßig**

Zwischen 2018 und 2026 erschien praktisch jedes Jahr eine neue Angriffsart auf die Mikroarchitektur. Die Namen wechseln, das Muster bleibt: Ein geteiltes Betriebsmittel erlaubt Rückschlüsse über eine Grenze hinweg.

Genau davor warnt Saltzers siebter Grundsatz von 1975, unter der Überschrift *least common mechanism*. Er wurde jahrzehntelang als Empfehlung für sauberen Entwurf gelesen und nicht als Warnung vor Seitenkanälen.

Die praktische Folgerung für die Gegenwart lautet: Wo wirklich Trennung gebraucht wird, trennt man die Hardware. Ein eigener Kern, ein eigener Rechner, oder wenigstens die Zusicherung, dass kein fremder Code auf demselben Kern läuft.

Saltzer und Schroeder[26] sind bemerkenswert gut gealtert; Anderson und Dahlin[1] behandeln Schutzmechanismen ausführlich; Tanenbaum und Bos[28] geben die Übersicht über Rechtemodelle.

15 Der Stand im Jahr 2026

Was gelöst ist und was nicht

Frage	Stand	Bemerkung
Ablaufplanung, allgemein	gelöst	CFS und EEVDF genügen
Speicherverwaltung	gelöst	seit Jahrzehnten stabil
Dateisystemkonsistenz	gelöst	Journal, Kopie beim Schreiben
Virtualisierung	gelöst	mit Hardwarehilfe
Treiberfehler	in Arbeit	Rust im Kern
Ein-/Ausgabe-Aufwand	in Arbeit	io_uring
Echtzeit unter Linux	in Arbeit	PREEMPT_RT seit 2024 im Hauptzweig
Containerisolation	in Arbeit	Zwischenlösungen
Nebenläufige Programme richtig zu schreiben	offen	**das Kernproblem**
Seitenkanäle	offen	jedes Jahr Neues
Kernfehler grundsätzlich	offen	30 Mio. Zeilen

Die drei Zeilen im unteren Abschnitt haben gemeinsam, dass sie keine technische Lösung erwarten lassen. Sie werden eingedämmt, nicht behoben.

Fünf Entwicklungen

Erstens: Rust im Kern.
Seit 2022 lassen sich Teile des Linux-Kerns in Rust schreiben. Bis 2026 sind mehrere Treiber und Teilsysteme dort angekommen.

Was es verhindert	Zugriffe über Feldgrenzen, Benutzung freigegebenen Speichers, Wettläufe zwischen Fäden
Was es nicht verhindert	Verklemmungen, Logikfehler, falsche Annahmen über Hardware

Untersuchungen an Microsoft- und Android-Code haben übereinstimmend ergeben, dass etwa siebzig Prozent der sicherheitsrelevanten Fehler auf Speicherfehler zurückgehen. Genau diese Klasse verschwindet.

Der Übergang ist trotzdem mühsam. Es gibt Widerstand von Entwicklern, die zwei Sprachen im Kern für einen Fehler halten, und die Verbindung zwischen den C-Datenstrukturen und dem Rust-Typsystem ist aufwendig zu pflegen.

Zweitens: eBPF.
Man kann geprüfte Programme in den Kern laden, die dort an bestimmten Stellen ausgeführt werden.

Beobachtung	Messen, was der Kern tut, ohne ihn zu verändern
Netz	Pakete filtern und umleiten, bevor der Netzstapel sie sieht
Sicherheit	Eigene Regeln für Systemaufrufe
Ablaufplanung	Seit 2024 lassen sich eigene Planer laden

Ein Prüfer stellt vor dem Laden sicher, dass das Programm terminiert und nur auf erlaubte Speicherbereiche zugreift. Das ist eine bemerkenswerte Anwendung der Grenzen aus Band 2: Weil das Halteproblem unentscheidbar ist, lehnt der Prüfer im Zweifel ab.

Die letzte Zeile der Tabelle ist die weitreichendste. Ein Ablaufplaner im Kern war jahrzehntelang der Inbegriff dessen, was man nicht anfasst. Jetzt kann man einen eigenen laden, ausprobieren und wieder entfernen.

Drittens: io_uring und die Kosten der Grenze.
Nach 2018 wurde der Systemaufruf teuer, und die Antwort besteht darin, ihn zu vermeiden.

Schnittstelle	Aufrufe je Vorgang	Jahr
`read/write`	1	1974
`epoll`	2	2002
`io_uring`	≈ 0	2019

Der Preis ist eine erheblich größere Angriffsfläche. Mehrere Anbieter haben io_uring in ihren Containerumgebungen abgeschaltet, nachdem darin Lücken gefunden wurden.

Viertens: Isolation als Hauptthema.
Was 2018 als Zwischenfall begann, ist zur dauerhaften Lage geworden. Seither erscheint regelmäßig eine neue Angriffsart auf geteilte Hardwarezustände.

Folge für den Entwurf	Neue Prozessoren bekommen ausdrückliche Trennmechanismen statt nachträglicher Abhilfen
Folge für den Betrieb	Wer echte Trennung braucht, trennt die Hardware
Folge für die Kosten	Der Aufschlag von zwanzig bis vierzig Prozent ist dauerhaft eingepreist

Fünftens: Echtzeit im Hauptzweig.
Die Erweiterung PREEMPT_RT wurde 2024 nach mehr als zwanzig Jahren Entwicklung in den Linux-Hauptzweig aufgenommen.

Größe	gewöhnlich	mit PREEMPT_RT
Übliche Verzögerung	≈ 50 µs	≈ 20 µs
Schlechtester Fall	Millisekunden	**unter 100 µs**
Durchsatz	100 %	≈ 95 %

Der Gewinn steht in der zweiten Zeile. Nicht der Mittelwert wird besser, sondern die obere Schranke, und darauf kommt es bei Werkzeugmaschinen, Audiogeräten und Fahrzeugsteuerungen an.

Was aus der mittleren Datentechnik geblieben ist

Die Rechner der mittleren Datentechnik sind verschwunden, ihre Aufgabenstellungen nicht. Es lohnt sich, die Linien nachzuzeichnen.

damals	heute	Kapitel
Satzsperre	Zeilensperre in der Datenbank	6
Sperrgranularität	unverändert dieselbe Frage	6
Indexsequentiell	B-Baum-Index	11
Bildschirmplätze	Sitzungen im Webserver	3
Dezimalrechnung	`DECIMAL` in SQL	Band 0
Mehrplatzbetrieb	Mehrmandantenfähigkeit	14

Bemerkenswert ist die zweite Zeile. Die Frage, wie fein man sperrt, ist in fünfzig Jahren nicht einfacher geworden. Sie wird heute nur an anderer Stelle gestellt.

Und eine Erfahrung aus jener Zeit hat unmittelbar zu einem heute verbreiteten Verfahren geführt: Weil eine Sperre, die ein Mensch hält, beliebig lange dauern kann, verzichtet man auf sie und prüft stattdessen beim Schreiben, ob sich etwas geändert hat.

Die optimistische Nebenläufigkeit aus Kapitel 12 ist damit keine neue Erfindung, sondern die Antwort auf ein Problem, das man in der Auftragsbearbeitung sehr genau kannte.

Drei Einsichten

Erstens: Die Fiktion ist eine Leistung, keine Selbstverständlichkeit.
Jedes Programm glaubt, es habe den Rechner für sich. Diese Vorspiegelung erfordert Seitentabellen, Zeitgeberunterbrechungen, Zustandswechsel und eine sorgfältige Buchführung.

Wo sie undicht wird, merkt man es an unerklärlichen Verzögerungen, an Programmen, die auf einer anderen Maschine anders laufen, und an Seitenkanälen. Es ist derselbe Gedanke wie in Band 4, eine Ebene höher.

Zweitens: Nebenläufigkeit ist kein Zusatz, sondern eine andere Denkweise. Ein sequentielles Programm kann man Zeile für Zeile lesen. Bei einem nebenläufigen genügt das nicht, denn zwischen je zwei Zeilen kann alles Mögliche geschehen.

	sequentiell	**nebenläufig**
Abläufe	einer	**exponentiell viele**
Testen	wirksam	unzureichend
Nötig	Sorgfalt	**Argumentation**

Deshalb sind die wirksamsten Fortschritte auf diesem Gebiet nicht Werkzeuge, sondern Denkweisen: der Monitor von 1974, die Nachrichten von 1978, das Eigentumsmodell von Rust. Sie machen ganze Fehlerklassen unmöglich, statt sie zu suchen.

Drittens: Fast alles ist ein Kompromiss, und die Lage ändert sich. Zeitscheibenlänge, Blockgröße, Puffergröße, Journalbetriebsart, Sperrgranularität, Container gegen virtuelle Maschine: Jede dieser Entscheidungen hat keine richtige Antwort, sondern eine, die zur Last passt.

Und die Antwort altert. Der Plattenplaner war dreißig Jahre lang richtig und ist bei einer NVMe-SSD schädlich. Das Vorauslesen war immer sinnvoll und ist es bei zufälligem Zugriff nicht mehr. Wer eine Optimierung übernimmt, ohne ihre Voraussetzungen zu prüfen, übernimmt einen Fehler. Gegen diese Art von Irrtum hilft kein Verfahren, sondern nur geübtes Urteil, und die beste Sammlung davon stammt von Butler Lampson *aus dem Jahr 1983*[17].

Band 6 wendet sich den Programmiersprachen und Übersetzern zu, also der Frage, wie aus dem Text, den ein Mensch schreibt, die Befehle werden, die in Band 4 ausgeführt und in diesem Band verwaltet wurden.

Ein Betriebssystem ist eine Ansammlung von Dingen, die nicht in eine Sprache passen. Es sollte deshalb keine geben.

— Dan Ingalls, *Design Principles Behind Smalltalk, 1981.*
Der Satz ist zugespitzt und trifft einen wunden Punkt: Vieles, was im Betriebssystem steht, steht dort nur,

weil es anderswo keinen Platz gefunden hat.

Part VI

Anhang

A Zahlen und Kenngrößen

Zeiten im Überblick

Vorgang	Zeit	Takte bei 4 GHz
Funktionsaufruf	1 ns	4
Sperre nehmen, frei	5 ns	20
Cachefehlgriff, L3	10 ns	40
Hauptspeicherzugriff	65 ns	260
Atomarer Befehl, umstritten	100 ns	400
Systemaufruf	170 ns	680
Fadenwechsel	1 µs	4000
Prozesswechsel	3 µs	12 000
Kleiner Seitenfehler	3 µs	12 000
SSD-Zugriff	100 µs	$4 \cdot 10^5$
`fsync`, SSD	200 µs	$8 \cdot 10^5$
Plattenzugriff	8 ms	$3{,}2 \cdot 10^7$
`fsync`, Platte	10 ms	$4 \cdot 10^7$

Zwischen der ersten und der letzten Zeile liegt der Faktor zehn Millionen. Wer eine Optimierung erwägt, sollte zuerst nachsehen, in welcher Zeile er sich befindet.

Kenngrößen

Größe	typisch	Bemerkung
Seitengröße	4 KB	auch 2 MB, 1 GB
Blockgröße Dateisystem	4 KB	
Zeitscheibe	1–10 ms	lastabhängig
Zielverzögerung CFS	≈ 24 ms	
Kernstapel je Faden	8–16 KB	
Stapel im Benutzerbetrieb	8 MB	reserviert
Direkte Zeiger im Inode	12	
Systemaufrufe (Linux)	≈ 350	
Linux-Kern	≈ 30 Mio. Zeilen	davon 60 % Treiber

Vier Bedingungen und acht Grundsätze

Coffman (Verklemmung)	Saltzer/Schroeder (Schutz)
Ausschluss	Sparsamkeit
Halten und Warten	Voreinstellung sperrt
Kein Entzug	Vollständige Prüfung
Zyklisches Warten	Offener Entwurf
	Trennung der Rechte
	Geringstes Recht
	Wenig Gemeinsames
	Annehmbarkeit

Links genügt es, eine Bedingung zu brechen. Rechts sollte man alle acht beachten.

B Landkarte des Bandes

Wo was steht

Kap.	Thema	Verweise
1	Kern, Systemaufrufe	Band 4 (Betriebsarten)
2	Unterbrechungen	Band 4 (Ausnahmen)
3	Prozesse und Fäden	Band 4 (Mehrkern)
4	Ablaufplanung	Band 3 (Optimalität)
5	Speicherverwaltung	Band 4 (Seitentabellen)
6	Wettläufe	Band 4 (Speichermodelle)
7	Semaphore, Monitore	Band 6 (Sprachmittel)
8	Verklemmungen	Band 1 (Graphen, Zyklen)
9	Klassiker	Band 11 (Netzwerkanalyse)
10	Geräte	Band 9 (Netze)
11	Dateisysteme	Band 1 (B-Bäume), Band 10
12	Dauerhaftigkeit	Band 10 (Transaktionen)
13	Virtualisierung	Band 2 (Simulation)
14	Isolation	Band 8 (Kryptographie)
15	Stand 2026	Band 2 (Halteproblem, eBPF)

Sechs Verbindungen zur Mathematikreihe

Wartezeiten	Warteschlangentheorie, Poisson-Prozesse
Verklemmungsgraphen	Graphentheorie, Zyklensuche
Ablaufplanung	Optimierung unter Nebenbedingungen
Verdrängung	Stapelalgorithmen, Bélády
Seitenfehlerraten	Wahrscheinlichkeitsrechnung
Sperrkonflikte	Kombinatorik der Verschränkungen

C Glossar

Begriffe

Ablaufplaner	Wählt aus, welcher bereite Prozess läuft
Arbeitsmenge	Die Seiten, die ein Prozess in letzter Zeit gebraucht hat
Bedingungsvariable	Warteschlange im Monitor, ohne Gedächtnis
Drehsperre	Sperre, bei der der Wartende aktiv prüft
Faden	Ausführungsstrang; teilt den Adressraum
Inode	Verwaltungsstruktur einer Datei, ohne Namen
Journal	Protokoll geplanter Änderungen
Kontrollgruppe	Begrenzung des Verbrauchs
Kritischer Abschnitt	Code, den nur einer zugleich betreten darf
Livelock	Alle aktiv, niemand kommt voran
Monitor	Sprachkonstruktion mit eingebautem Ausschluss
Namensraum	Begrenzung dessen, was sichtbar ist
Prioritätsumkehr	Wichtiger Faden wartet auf unwichtigen
Satzsperre	Sperre auf einem Datensatz statt der Datei
Semaphor	Zähler mit unteilbarem Warten und Freigeben
Seitenfehler	Ausnahme bei nicht abgebildeter Seite
Seitenflattern	Alle Prozesse verdrängen sich gegenseitig
Systemaufruf	Kontrollierter Übergang in den Kernbetrieb
TOCTOU	Wettlauf zwischen Prüfung und Benutzung
Verklemmung	Zyklisches Warten, niemand kommt voran
Wettlauf	Ergebnis hängt von der Reihenfolge ab
Zeitscheibe	Zeitraum, den ein Prozess am Stück bekommt

D Zeittafel

Von 1956 bis 2026

1956	Erstes Stapelverarbeitungssystem
1961	Zeitscheibenbetrieb (CTSS); virtueller Speicher (Atlas)
1964	Multics beginnt
1965	**Dijkstra: wechselseitiger Ausschluss, Semaphore**
1968	Dijkstra: Cooperating Sequential Processes
1968	Nixdorf Computer AG gegründet
1969	Bell Labs verlässt Multics; Unix beginnt
1971	Coffman u. a.: vier Bedingungen
1973	Unix-Kern in C neu geschrieben
1974	**Ritchie/Thompson: Unix-Aufsatz; Hoare: Monitore**
1974	Popek/Goldberg: Bedingungen für Virtualisierung
1975	Saltzer/Schroeder: acht Grundsätze
1976	Nixdorf 8870 mit NIROS
1978	Hoare: CSP; Lamport: Ordnung von Ereignissen
1979	`chroot`
1981	Peterson: einfache Ausschlusslösung
1983	Lampson: Hints for Computer System Design
1987	MINIX
1990	Nixdorf geht an Siemens
1991	Linux beginnt
1992	Tanenbaum-Torvalds-Streit; LFS
1994	Bonwick: Slab-Zuteiler
1996	Ousterhout: Why Threads Are a Bad Idea
1997	Mars Pathfinder: Prioritätsumkehr
2000	FreeBSD Jails
2002	`epoll`; Namensräume in Linux
2003	**Xen: Paravirtualisierung**
2005	Intel VT-x, AMD-V; ZFS
2007	CFS; Kontrollgruppen
2009	seL4: Kern mit Korrektheitsbeweis
2013	**Docker**
2014	Kubernetes
2018	**Spectre und Meltdown**
2019	io_uring
2020	eBPF breit im Einsatz
2022	Rust im Linux-Kern
2023	EEVDF löst CFS ab
2024	PREEMPT_RT im Hauptzweig
2026	Isolation bleibt das beherrschende Thema

E Kommentierte Quellen

[1] Anderson, T., Dahlin, M.: *Operating Systems: Principles and Practice*. 2. Aufl., Recursive Books 2014.

Ein Lehrbuch, das mehr Wert auf Begründungen legt als die meisten. Besonders gut bei Nebenläufigkeit und Schutzmechanismen. Etwas anspruchsvoller als Silberschatz.

[2] Arpaci-Dusseau, R., Arpaci-Dusseau, A.: *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books.

***Die beste Empfehlung für den Einstieg**, und frei im Netz verfügbar. Gegliedert in Virtualisierung, Nebenläufigkeit und Persistenz. Kurze Kapitel, viele Beispiele, ein angenehm gesprächiger Ton. Wenn Sie nur ein Buch lesen, dann dieses.*

[3] Bach, M.: *The Design of the UNIX Operating System*. Prentice Hall 1986.

Die klassische Darstellung des System-V-Kerns. Vierzig Jahre alt und immer noch lehrreich, weil sie zeigt, wie überschaubar ein Betriebssystem einmal war.

[4] Barham, P., u. a.: *Xen and the Art of Virtualization*. SOSP 2003.

Der Aufsatz, der die Paravirtualisierung populär machte, kurz bevor die Hardware das Problem auf andere Weise löste. Historisch wichtig und klar geschrieben.

[5] Bonwick, J.: *The Slab Allocator: An Object-Caching Kernel Memory Allocator*. USENIX 1994.

Zwölf Seiten mit einer einzigen guten Idee, die überall übernommen wurde. Ein Musterbeispiel dafür, wie ein Systemaufsatz aussehen sollte.

[6] Bovet, D., Cesati, M.: *Understanding the Linux Kernel*. 3. Aufl., O'Reilly 2005.

Geht bis auf die Quelltextebene hinunter. Bezieht sich auf einen alten Kern und ist trotzdem das gründlichste Buch dieser Art. Zum Nachschlagen, nicht zum Durchlesen.

[7] Coffman, E., Elphick, M., Shoshani, A.: *System Deadlocks*. ACM Computing Surveys 3(2), 1971.

Die vier Bedingungen. Ein kurzer Aufsatz, der ein Problem so vollständig aufgeklärt hat, dass seither wenig hinzugekommen ist.

[8] Corbató, F., Vyssotsky, V.: *Introduction and Overview of the Multics System*. AFIPS 1965.

Die Vision, an der Multics scheiterte und aus der Unix hervorging. Lesenswert wegen des Ehrgeizes: Rechnen sollte eine Versorgungsleistung werden wie Strom.

[9] Dijkstra, E.: *Solution of a Problem in Concurrent Programming Control*. CACM 8(9), 1965.

Eine Seite. Die erste korrekte Lösung des Ausschlussproblems ohne Hardwarehilfe. Der Beginn eines ganzen Gebiets.

[10] Dijkstra, E.: *Cooperating Sequential Processes*. 1968.

Semaphore, Erzeuger und Verbraucher, die speisenden Philosophen. Fast alles, was in Teil III dieses Bandes steht, geht auf diesen Text zurück.

[11] Brinch Hansen, P.: *Operating System Principles*. Prentice Hall 1973.

Der Erste, der Monitore in einer Sprache umsetzte. Sein Buch ist strenger und formaler als die üblichen Lehrbücher und hat viele Begriffe geprägt.

[12] Heinz Nixdorf MuseumsForum, Paderborn: Sammlung und Archiv zur Geschichte der Datenverarbeitung.

Das größte Computermuseum der Welt und zugleich das Archiv zur Geschichte von Nixdorf und der deutschen mittleren Datentechnik. Wer zu Baureihen wie der 8870, zu NIROS oder zu COMET Genaueres sucht, findet dort Geräte, Handbücher und Firmenunterlagen. Für einen Band über Betriebssysteme ist das die einschlägige Quelle zu einem Kapitel, das in der englischsprachigen Literatur nicht vorkommt.

[13] Herlihy, M., Shavit, N.: *The Art of Multiprocessor Programming*. 2. Aufl., Morgan Kaufmann 2020.

Das maßgebliche Buch zur nebenläufigen Programmierung mit gemeinsamem Speicher. Behandelt sperrfreie Datenstrukturen gründlich und ehrlich, also einschließlich der Gründe, sie zu meiden.

[14] Hoare, C.A.R.: *Monitors: An Operating System Structuring Concept*. CACM 17(10), 1974.

Die Verlagerung des Ausschlusses vom Programmierer in die Sprache. Jede moderne Sprache hat eine Form davon, meist ohne den Namen zu nennen.

[15] Hoare, C.A.R.: *Communicating Sequential Processes*. CACM 21(8), 1978.

Der andere Weg: keine gemeinsamen Daten, nur Nachrichten. Hat Occam, Erlang und Go geprägt. Formal und trotzdem gut lesbar.

[16] Lamport, L.: *Time, Clocks, and the Ordering of Events in a Distributed System*. CACM 21(7), 1978.

Der meistzitierte Aufsatz der verteilten Systeme. Die Beziehung „geschieht vor“ ist die Grundlage jedes Wettlaufsuchers, auch derjenigen, die auf einer einzigen Maschine arbeiten.

[17] Lampson, B.: *Hints for Computer System Design*. SOSP 1983.

Eine Sammlung von Faustregeln aus jahrzehntelanger Erfahrung, geordnet und mit Beispielen belegt. Kein Lehrbuch, sondern gesammelte Urteilskraft. Sehr zu empfehlen, wenn man die Technik schon kennt.

[18] Love, R.: *Linux Kernel Development*. 3. Aufl., Addison-Wesley 2010.

Der beste Einstieg in den Linux-Kern. Erklärt die Entwurfsentscheidungen und nicht nur die Datenstrukturen. Bezieht sich auf einen älteren Kern, was den Wert kaum mindert.

[19] McKenney, P.: *Is Parallel Programming Hard, And, If So, What Can You Do About It?*

Frei verfügbar und laufend aktualisiert. Der Autor hat RCU in den Linux-Kern gebracht. Sehr gründlich, teilweise schwer, und die beste Darstellung von Speichermodellen, die es gibt.

[20] McKusick, M., u. a.: *The Design and Implementation of the FreeBSD Operating System*. 2. Aufl., Addison-Wesley 2014.

Das Gegenstück zu Love für die BSD-Seite. Besonders gut bei Dateisystemen; die Beschreibung des weichen Aktualisierens findet man nirgends sonst so klar.

[21] Merkel, D.: *Docker: Lightweight Linux Containers for Consistent Development and Deployment*. Linux Journal 2014.

Der Artikel, mit dem Container in die Breite gingen. Technisch enthält er nichts Neues, und genau das ist die Lehre: Verpackung kann wichtiger sein als Erfindung.

[22] Ousterhout, J.: *Why Threads Are a Bad Idea (for most purposes)*. USENIX 1996.

Ein Foliensatz, in zwanzig Minuten gelesen, mit einer bis heute wirksamen Streitschrift. Man muss ihm nicht zustimmen, um von der Argumentation zu profitieren.

[23] Peterson, G.: *Myths About the Mutual Exclusion Problem*. Information Processing Letters 12(3), 1981.

Zwei Seiten, die eine einfachere Lösung liefern als alles Vorherige. Zeigt nebenbei, dass verbreitete Annahmen über die Notwendigkeit komplizierter Verfahren falsch waren.

[24] Ritchie, D., Thompson, K.: *The UNIX Time-Sharing System*. CACM 17(7), 1974.

Elf Seiten über ein System, das die Welt verändert hat. Bemerkenswert ist, wie viel Raum den Entwurfsgedanken gewidmet ist und wie wenig den Merkmalen.

[25] Rosenblum, M., Ousterhout, J.: *The Design and Implementation of a Log-Structured File System*. SOSP 1992.

Alles wird sequentiell geschrieben, nichts überschrieben. Für Festplatten war das zu radikal; für Flashspeicher ist es heute genau richtig.

[26] Saltzer, J., Schroeder, M.: *The Protection of Information in Computer Systems*. Proceedings of the IEEE 63(9), 1975.

Die acht Grundsätze. Fünfzig Jahre alt und ohne Alterserscheinungen. Der siebte, least common mechanism*, liest sich seit 2018 wie eine Vorhersage der Seitenkanalangriffe.*

[27] Silberschatz, A., Galvin, P., Gagne, G.: *Operating System Concepts*. 10. Aufl., Wiley 2018.

Das verbreitetste Lehrbuch, oft „das Dinosaurierbuch" genannt. Vollständig und etwas trocken. Als Nachschlagewerk und zur Prüfungsvorbereitung gut geeignet.

[28] Tanenbaum, A., Bos, H.: *Modern Operating Systems*. 5. Aufl., Pearson 2022.

Der Gegenpol zu Silberschatz: meinungsfreudig, unterhaltsam und mit Fallstudien zu Unix, Linux, Windows und Android. Tanenbaums Urteile sind pointiert, was das Lesen erleichtert und gelegentlich zum Widerspruch reizt.

Wenn Sie nur drei Bücher kaufen

[2]	**Arpaci-Dusseau**	Zum Lernen. Frei verfügbar, klar, mit den besten Erklärungen des Fachs
[13]	**Herlihy/Shavit**	Für die Nebenläufigkeit, wenn Sie mehrfädige Programme schreiben
[28]	**Tanenbaum/Bos**	Zum Nachschlagen und zum Vergnügen

Wer danach noch etwas möchte: [17] Lampson für die Urteilskraft und [26] Saltzer/Schroeder für den Entwurf sicherer Systeme. Beide sind kurz und beide halten, was fünfzig Jahre Abstand versprechen.